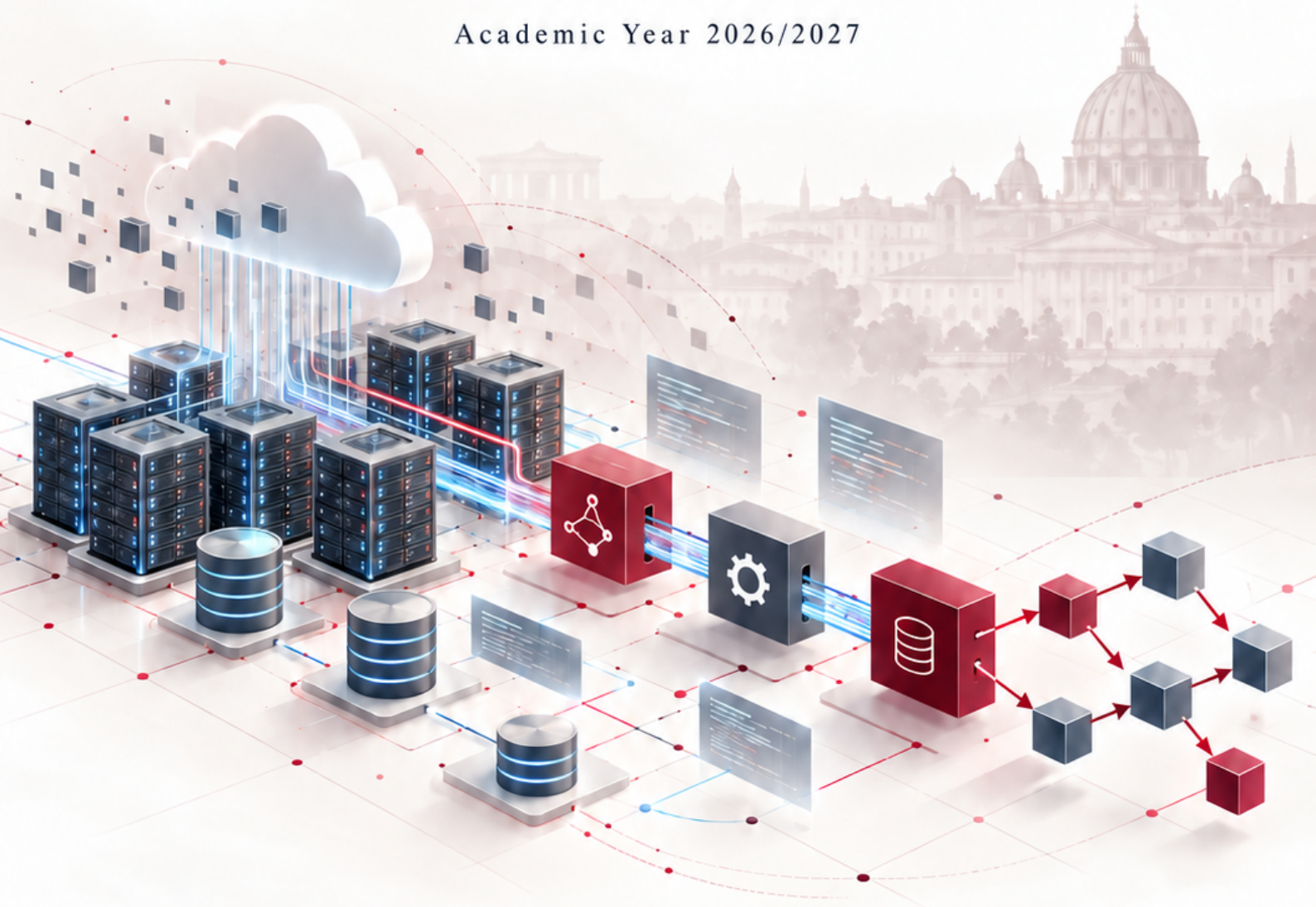
Master's Degree in Computer Science

BIG DATA COMPUTING

Course Notes

Academic Year 2026/2027



Prepared by

Matt Clifford Trinidad Ramos

Contents

Course Information **ix**

1 Scale-Out Paradigm and Distributed Storage **1**
1.1 Why Scale Out? . . . 1
1.1.1 The Data Growth Problem . . . 1
1.1.2 Single-Machine Limits . . . 2
1.1.3 Vertical Scaling . . . 2
1.1.4 Horizontal Scaling . . . 2
1.1.5 Vertical versus Horizontal Scaling . . . 3
1.1.6 The New Failure Model . . . 3
1.1.7 Cost per Byte and Cost per FLOP . . . 3
1.2 Foundations of Distributed Storage . . . 4
1.2.1 The Distributed File System Abstraction . . . 4
1.2.2 Throughput versus Latency . . . 4
1.2.3 Huge Files and Sequential Access . . . 4
1.2.4 Write-Once, Read-Many . . . 4
1.2.5 The Block / Chunk Abstraction . . . 4
1.2.6 Why Fixed-Size Blocks? . . . 4
1.2.7 Metadata and Data Separation . . . 5
1.2.8 Single Point of Coordination . . . 5
1.2.9 Namespace Management . . . 5
1.2.10 Network Topology Awareness . . . 5
1.2.11 Why Replication Is Necessary . . . 5
1.3 The Google File System (GFS) . . . 6
1.3.1 Motivation and Original Context . . . 6
1.3.2 Architecture Overview . . . 6
1.3.3 The GFS Master . . . 6
1.3.4 Why a Single Master? . . . 7
1.3.5 Chunkservers . . . 7
1.3.6 Chunk Size . . . 7
1.3.7 Replication Factor . . . 7
1.3.8 Master–Chunkserver Heartbeats . . . 7
1.3.9 GFS Read Path . . . 7
1.3.10 GFS Write Path . . . 8
1.3.11 Primary and Secondary Replicas . . . 8
1.3.12 Mutation Ordering and Consistency . . . 9
1.3.13 Chunk Leases . . . 9
1.3.14 Garbage Collection . . . 9
1.3.15 Operation Log and Checkpoints . . . 9
1.3.16 Shadow Masters . . . 9
1.3.17 Limitations . . . 9
1.4 Data Locality and Replication Strategy . . . 10

1.4.1 The Principle of Data Locality . . . 10
1.4.2 Why Network Bandwidth Is Scarce . . . 10
1.4.3 Rack-Aware Replica Placement . . . 10
1.4.4 Replication Factor Trade-offs . . . 10
1.4.5 Failure Domains . . . 11
1.4.6 Worked Example: Replication and Data-Loss Probability . . . 11
1.4.7 Re-Replication after Failure . . . 12
1.4.8 Balancing Storage Utilization . . . 12
1.5 Hadoop Distributed File System (HDFS) . . . 12
1.5.1 Origins and Relation to GFS . . . 12
1.5.2 Architecture . . . 13
1.5.3 NameNode . . . 13
1.5.4 DataNodes . . . 13
1.5.5 Block Size . . . 13
1.5.6 FsImage and EditLog . . . 13
1.5.7 NameNode Startup . . . 13
1.5.8 Secondary NameNode . . . 14
1.5.9 High Availability . . . 14
1.5.10 Heartbeats and Block Reports . . . 14
1.5.11 HDFS Read Path . . . 14
1.5.12 HDFS Write Pipeline . . . 14
1.5.13 Replica Placement Policy . . . 15
1.5.14 Federation . . . 15
1.5.15 GFS and HDFS: Terminology and Architecture . . . 15
1.5.16 Why Blocks Suit Batch Processing . . . 16
1.5.17 Append-Only, No Random Writes . . . 16
1.6 Failure Handling and Fault Tolerance . . . 16
1.6.1 Failure Is the Default Assumption . . . 16
1.6.2 Types of Failures . . . 17
1.6.3 Detecting Failure: Heartbeats and Timeouts . . . 17
1.6.4 Recovery from Chunkserver / DataNode Failure . . . 17
1.6.5 Recovery from Master / NameNode Failure . . . 17
1.6.6 Checksums and Silent Corruption . . . 17
1.6.7 Real-World Failure Rates at Scale . . . 18
1.6.8 Modeling Disk Failures in One Week . . . 18
1.6.9 Probability of Exactly k Failures . . . 19
1.6.10 Expected Number of Failures . . . 19
1.6.11 Variance and Standard Deviation . . . 19
1.6.12 From AFR to Failures per Week . . . 20
1.6.13 Probability of at Least One Failure . . . 20
1.6.14 Example: Chunkserver Dies Mid-Read . . . 21
1.7 From Storage to Computation . . . 21
1.7.1 GFS and HDFS Side by Side . . . 21
1.7.2 Storage Enables Compute . . . 21
1.7.3 MapReduce on Top of Distributed Storage . . . 22
1.7.4 Limits of the GFS/HDFS Model . . . 22
1.7.5 Three Pillars of Scale-Out Storage . . . 22
1.8 Key Terms . . . 22
1.9 Chapter Summary . . . 22

1.10 Quick Review . . . 24
1.11 Further Reading . . . 26

2 The MapReduce Execution Model 27
2.1 The MapReduce Abstraction . . . 27
2.1.1 The Problem Being Solved . . . 27
2.1.2 Functional Programming Roots . . . 27
2.1.3 The Programming Model . . . 28
2.1.4 Map Semantics . . . 28
2.1.5 Reduce Semantics . . . 28
2.1.6 Worked Example: Word Count . . . 28
2.1.7 Second Example: Inverted Index . . . 29
2.1.8 What the Abstraction Hides . . . 29
2.1.9 Determinism and Side Effects . . . 29
2.2 Execution Architecture . . . 30
2.2.1 From Program to Cluster Job . . . 30
2.2.2 The Master / JobTracker . . . 30
2.2.3 Input Splits Become Map Tasks . . . 30
2.2.4 Data Locality in Scheduling . . . 31
2.2.5 Number of Map Tasks versus Machines . . . 31
2.2.6 Intermediate Output Stays Local . . . 31
2.2.7 Partitioning Intermediate Keys . . . 31
2.2.8 Choosing the Number of Reduce Tasks . . . 31
2.2.9 Execution Pipeline . . . 32
2.2.10 Combiners . . . 32
2.2.11 Sharded Output . . . 32
2.3 Shuffle, Sort, and the Reduce Phase . . . 33
2.3.1 The Shuffle Phase . . . 33
2.3.2 Each Mapper Partitions Its Own Output . . . 33
2.3.3 Why Shuffle Is Expensive . . . 33
2.3.4 Shuffle Network Volume . . . 33
2.3.5 Sorting by Intermediate Key . . . 34
2.3.6 Why Sorting Simplifies Reduce . . . 34
2.3.7 Reduce Output . . . 34
2.3.8 Why Shuffle Often Dominates Runtime . . . 34
2.3.9 Mitigating Shuffle Cost . . . 34
2.3.10 End-to-End Word Count Trace . . . 34
2.4 Fault Tolerance and Stragglers . . . 35
2.4.1 Computation Introduces New Failure Modes . . . 35
2.4.2 Worker Failure Detection . . . 36
2.4.3 Failed Map Tasks . . . 36
2.4.4 Failed Reduce Tasks . . . 36
2.4.5 Why Re-Execution Is Safe . . . 36
2.4.6 Idempotency and Atomic Commit . . . 36
2.4.7 Master Failure . . . 36
2.4.8 The Straggler Problem . . . 37
2.4.9 Causes of Stragglers . . . 37
2.4.10 Speculative Execution . . . 37
2.4.11 Numeric Example . . . 37

2.4.12 Speculation Trade-off . . . 38
2.5 Synthesis . . . 38
2.5.1 Full MapReduce Pipeline . . . 38
2.5.2 Storage versus Compute Layer . . . 38
2.5.3 Strengths of MapReduce . . . 38
2.5.4 Limitations . . . 39
2.5.5 Iterative Pipelines . . . 39
2.6 Key Terms . . . 39
2.7 Chapter Summary . . . 39
2.8 Quick Review . . . 41
2.9 Further Reading . . . 42

3 Apache Spark and Dataflow Graph Architectures 43
3.1 Beyond Two-Stage Pipelines . . . 43
3.1.1 The MapReduce Ceiling Revisited . . . 43
3.1.2 Why Chaining MapReduce Jobs Is Expensive . . . 43
3.1.3 Arbitrary Computation Graphs . . . 44
3.1.4 What “Memory-Centric” Means . . . 44
3.1.5 Motivating Workloads . . . 44
3.1.6 MapReduce and Spark Side by Side . . . 45
3.2 Resilient Distributed Datasets . . . 45
3.2.1 RDD as the Core Abstraction . . . 45
3.2.2 Why Immutability Matters . . . 45
3.2.3 Partitions: The Unit of Parallelism . . . 46
3.2.4 RDD Partitions versus Storage Blocks . . . 46
3.2.5 RDDs and Underlying Storage . . . 46
3.2.6 Logical Handle versus Materialized Data . . . 46
3.2.7 The “Resilient” Property . . . 47
3.2.8 RDD Interface Overview . . . 47
3.2.9 RDDs versus Regular Distributed Arrays . . . 47
3.2.10 Caching and Persistence . . . 47
3.3 Transformations and Actions . . . 48
3.3.1 Two Categories of RDD Operations . . . 48
3.3.2 Common Transformations . . . 48
3.3.3 Common Actions . . . 49
3.3.4 Worked Example: Word Count with RDDs . . . 49
3.3.5 Why the Split Creates an Optimization Opportunity . . . 49
3.4 Lazy Evaluation and the Execution DAG . . . 50
3.4.1 What Laziness Means . . . 50
3.4.2 Building the Logical Execution Plan . . . 50
3.4.3 Optimization Opportunities Enabled by Laziness . . . 50
3.4.4 Operator Fusion within a Stage . . . 50
3.4.5 From Logical DAG to Physical Jobs . . . 51
3.4.6 Worked Example: Building and Triggering a Multi-Step DAG . . . 51
3.5 Narrow versus Wide Dependencies . . . 52
3.5.1 Dependencies between RDDs . . . 52
3.5.2 Dependency Diagrams . . . 52
3.5.3 Why Dependency Type Determines Stage Boundaries . . . 52
3.5.4 Worked DAG: Identifying Narrow and Wide Operations . . . 53

3.5.5 Performance Implications of Wide Dependencies . . . 53
3.5.6 Minimizing Wide Dependencies . . . 53
3.6 Lineage Tracking and Fault Tolerance . . . 54
3.6.1 The Fault-Tolerance Problem . . . 54
3.6.2 What Lineage Records . . . 54
3.6.3 Lineage Stores a Recipe, Not Data Copies . . . 54
3.6.4 How a Lost Partition Is Recomputed . . . 54
3.6.5 Comparison with MapReduce Re-Execution . . . 55
3.6.6 Comparison with GFS/HDFS Replication . . . 55
3.6.7 Why Replicating Every Intermediate RDD Is Expensive . . . 55
3.6.8 Long Lineage and Checkpointing . . . 55
3.6.9 Recovery Reuses the DAG Scheduler . . . 56
3.7 Synthesis: From Transformations to Cluster Tasks . . . 56
3.7.1 The Full Spark Execution Model . . . 56
3.7.2 Role of the DAG Scheduler . . . 56
3.7.3 Data Locality Reappears . . . 57
3.7.4 Strengths of the Spark/RDD Model . . . 57
3.7.5 Remaining Limitations . . . 57
3.7.6 Beyond the Original RDD Model . . . 57
3.8 Key Terms . . . 57
3.9 Chapter Summary . . . 57
3.10 Quick Review . . . 59
3.11 Further Reading . . . 60

4 Geometry of High-Dimensional Spaces 61
4.1 Why Low-Dimensional Intuition Fails . . . 61
4.1.1 The Problem with Geometric Intuition . . . 61
4.1.2 Vectors, Norms, and the Two Reference Bodies . . . 61
4.1.3 Why Big Data Naturally Produces Large d . . . 62
4.2 Volume Concentration in the Hypercube . . . 62
4.2.1 Volume of the Unit Cube . . . 62
4.2.2 A Thin Shell Near the Boundary . . . 63
4.2.3 The Diagonal Grows with Dimension . . . 63
4.3 Volume Concentration in the Hypersphere . . . 64
4.3.1 Volume of a d-Dimensional Ball . . . 64
4.3.2 A Numerical View of the Vanishing Volume . . . 64
4.3.3 Ball versus Circumscribing Cube . . . 64
4.3.4 A Thin Spherical Shell . . . 65
4.4 Concentration Near the Equator . . . 65
4.4.1 From Surface Concentration to an Equatorial Question . . . 65
4.4.2 Why Slices Shrink Away from the Equator . . . 65
4.4.3 A Concentration Bound . . . 66
4.4.4 Random Unit Vectors Become Nearly Orthogonal . . . 66
4.5 Distance Concentration and the Curse of Dimensionality . . . 67
4.5.1 Random Points in the Unit Cube . . . 67
4.5.2 Expected Squared Distance . . . 67
4.5.3 Relative Spread Shrinks . . . 67
4.5.4 Chebyshev Makes the Concentration Explicit . . . 68
4.5.5 Nearest and Farthest Become Relatively Similar . . . 68

4.6 Metric-Space Breakdown and Practical Consequences . . . 69
4.6.1 What “Breakdown” Means . . . 69
4.6.2 Why Euclidean Distance Is Vulnerable . . . 69
4.6.3 Other Metrics . . . 69
4.6.4 Nearest-Neighbor Search . . . 69
4.6.5 Clustering and Classification . . . 69
4.7 Synthesis . . . 70
4.7.1 The Chain of Results . . . 70
4.7.2 Connection Back to the Infrastructure Layer . . . 70
4.8 Key Terms . . . 70
4.9 Chapter Summary . . . 71
4.10 Quick Review . . . 71
4.11 Further Reading . . . 72

5 Dimensionality Reduction and Random Projections 73
5.1 The Dimensionality-Reduction Problem . . . 73
5.1.1 The Core Goal . . . 73
5.1.2 Exact Isometry . . . 73
5.1.3 Approximate Isometry . . . 74
5.1.4 Data-Dependent versus Data-Oblivious Reduction . . . 74
5.2 Random Projections . . . 74
5.2.1 The Construction . . . 74
5.2.2 Why Gaussian Rows Are Isotropic . . . 75
5.2.3 Why the Normalization Preserves Expected Squared Length . . . 75
5.3 The Johnson–Lindenstrauss Lemma . . . 76
5.3.1 Formal Statement . . . 76
5.3.2 Parameter Dependence . . . 76
5.3.3 Worked Numerical Example . . . 76
5.4 Proof of the JL Lemma . . . 77
5.4.1 Step 1: Reduce One Pair to a Unit Vector . . . 77
5.4.2 Step 2: Identify the Chi-Squared Distribution . . . 77
5.4.3 Step 3: Exponential Concentration via the MGF . . . 78
5.4.4 Step 4: Union Bound over All Pairs . . . 79
5.4.5 Why the Proof Is Constructive . . . 79
5.5 Practical Implications and Limits . . . 80
5.5.1 JL as a Preprocessing Step . . . 80
5.5.2 Sparse Random Projections . . . 80
5.5.3 Fast JL Transform . . . 80
5.5.4 What JL Does Not Guarantee . . . 80
5.5.5 When JL Can Be Unhelpful . . . 81
5.5.6 Accuracy–Compression Trade-off . . . 81
5.6 Synthesis . . . 81
5.6.1 From Curse to Cure . . . 81
5.7 Key Terms . . . 81
5.8 Chapter Summary . . . 82
5.9 Quick Review . . . 82
5.10 Further Reading . . . 83

6 Vector Representations of Text and Embeddings 84
6.1 Text as High-Dimensional Data . . . 84

6.1.1 Why Text Must Become Vectors . . . 84
6.1.2 Bag of Words . . . 84
6.1.3 One-Hot Word Vectors . . . 85
6.1.4 Vocabulary Size at Web Scale . . . 85
6.2 Sparse Representations: TF–IDF . . . 85
6.2.1 From Counts to Weighted Counts . . . 85
6.2.2 Term Frequency . . . 85
6.2.3 Inverse Document Frequency . . . 86
6.2.4 The TF–IDF Formula . . . 86
6.2.5 Practical Variants . . . 86
6.2.6 Cosine Similarity . . . 86
6.2.7 Extreme Sparsity . . . 86
6.3 Scalability Limits of Sparse TF–IDF Spaces . . . 87
6.3.1 Sparse Does Not Mean Low-Dimensional . . . 87
6.3.2 Corpus-Scale Storage and Search . . . 87
6.3.3 Zipf's Law . . . 87
6.3.4 Heaps' Law . . . 87
6.4 The Distributional Hypothesis . . . 88
6.4.1 Meaning through Context . . . 88
6.4.2 Context Windows . . . 88
6.4.3 From Co-Occurrence Counts to Learned Embeddings . . . 88
6.5 From Sparse to Dense: Word Embeddings . . . 88
6.5.1 The Dense Representation . . . 88
6.5.2 Distributed Representation . . . 89
6.5.3 Semantic Geometry . . . 89
6.5.4 Analogy Arithmetic . . . 89
6.6 The Word2Vec Formulation . . . 89
6.6.1 CBOW and Skip-Gram . . . 89
6.6.2 The Skip-Gram Objective . . . 90
6.6.3 Input and Output Vectors . . . 90
6.6.4 The Full-Softmax Bottleneck . . . 90
6.6.5 Negative Sampling . . . 90
6.7 Training Word2Vec at Scale . . . 91
6.7.1 Per-Update Cost . . . 91
6.7.2 Billion-Word Corpora . . . 92
6.7.3 Subsampling Frequent Words . . . 92
6.7.4 Data-Parallel Corpus Sharding . . . 92
6.7.5 Shared Parameters and Asynchronous Updates . . . 92
6.8 Tokenization at Scale . . . 92
6.8.1 Tokenization Is a Representation Decision . . . 92
6.8.2 Word-Level Tokenization and OOV . . . 93
6.8.3 Subword Tokenization . . . 93
6.8.4 Byte-Pair Encoding . . . 93
6.8.5 WordPiece . . . 93
6.8.6 Tokenization Trade-Offs . . . 94
6.8.7 Distributed Tokenization . . . 94
6.9 Synthesis . . . 94
6.9.1 The Representation Pipeline . . . 94
6.9.2 Sparse versus Dense Recap . . . 94

6.10 Key Terms 95
6.11 Chapter Summary 95
6.12 Quick Review 96
6.13 Further Reading 98

Combined Quick Reference **99**

Course Information

Course at a glance

Course	Big Data Computing, M.Sc. in Computer Science
Academic year	2026/2027
Lecturer	Gabriele Tolomei
Class schedule	Wednesday 08:00–11:00 (Room 1L); Thursday 10:00–12:00 (Room 2L)
Office	Room 106, first floor, Building E, Viale Regina Elena 295
Course website	https://gtolomei.github.io/teaching/bdc.html

Assessment

The final grade is based on an oral seminar about a recent, high-impact research paper selected by the student and approved by the instructor. The course material identifies top-tier venues such as SIGMOD, VLDB, OSDI, SOSP, NeurIPS, ICML, and KDD as representative sources.

Seminar format

Format	Presentation	Q&A	Total
Individual	~12 min	~3 min	~15 min
Pair	~25 min	~5 min	~30 min

For pair presentations, presentation time and Q&A should be balanced between the two students. The course material states that honors (30 cum laude) require exceptional mastery of the topic, deep critical insight, and flawless academic communication.

Course Organization

- **Big Data Infrastructure:** scale-out storage, MapReduce, Spark/DAG execution, streaming, and fault tolerance.
- **High-Dimensional Data Representations:** high-dimensional geometry, Johnson–Lindenstrauss reduction, columnar layouts, and embeddings.
- **Classical Non-Learning Tasks:** indexing, sampling, Approximate Query Processing, sketches (HyperLogLog, Count-Min), and Locality-Sensitive Hashing.
- **Scale-Out Learning Systems:** large-scale ERM, parallel/asynchronous SGD, Parameter Server, Federated Learning, and memory-bandwidth optimization.

Suggested references. Leskovec, Rajaraman, and Ullman, *Mining of Massive Datasets*; Blum, Hopcroft, and Kannan, *Foundations of Data Science*; Kleppmann, *Designing Data-Intensive Applications*; Manning, Raghavan, and Schütze, *Introduction to Information Retrieval*; Shalev-Shwartz and Ben-David, *Understanding Machine Learning*; plus research papers referenced throughout the course material.

CHAPTER 1

Scale-Out Paradigm and Distributed Storage

A single machine is constrained by finite processor, memory, storage, network, power, and cooling resources. Big data infrastructure therefore replaces the assumption of one sufficiently large and reliable machine with a cluster of cooperating machines whose aggregate capacity can grow over time. This change in scale also changes the failure model: individual disks, nodes, and network components are expected to fail while the overall system continues to operate.

Distributed storage provides the substrate for this model. Large files are divided into independently managed blocks or chunks, metadata is separated from bulk data transfer, and replicas are placed across machines and racks so that data remains available after ordinary failures. Google File System (GFS) and Hadoop Distributed File System (HDFS) instantiate these principles and expose the physical placement information later used by distributed computation frameworks.

Key Concept

At scale, reliability is a property of the distributed software rather than of any individual machine. Commodity components may fail; the system survives by detecting those failures, maintaining redundant data, and reconstructing lost replicas automatically.

1.1 Why Scale Out?

1.1.1 The Data Growth Problem

Data growth can outpace the capacity growth of an individual machine. Increasing the capacity of one disk or one server every few years does not solve workloads whose data volume grows on a much shorter timescale. Representative examples include web crawls containing hundreds of billions of pages and petabytes of raw HTML, telemetry and clickstream data accumulating at terabytes per day, and scientific or enterprise datasets that can reach exabyte scale.

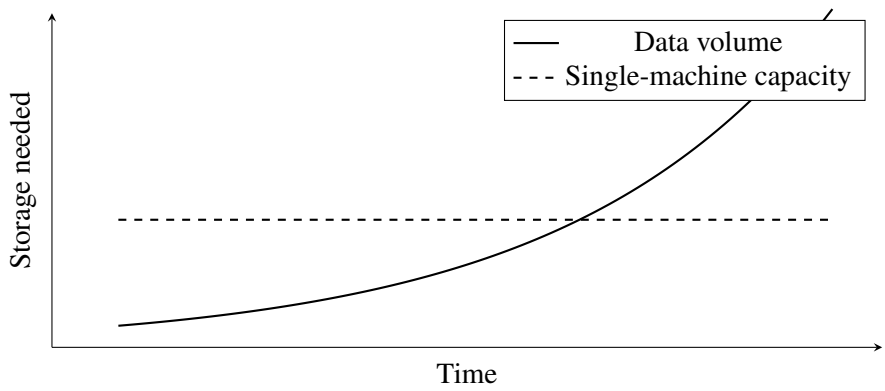


Figure 1.1: Conceptual representation of the data-growth problem. The curves are schematic and do not encode measured experimental data.

Figure 1.1 captures the qualitative scaling argument: if demand grows faster than the capacity of an individual machine, periodically replacing that machine only moves the ceiling.

1.1.2 Single-Machine Limits

A powerful server still faces finite physical resources:

- **CPU:** finite cores per socket and finite sockets per motherboard;
- **RAM:** finite DIMM slots and finite addressable memory;
- **Disk:** finite drive bays and finite controller bandwidth;
- **Network:** finite NIC bandwidth entering and leaving one machine.

These are hard ceilings. A larger machine may postpone the limit, but it does not remove the existence of a limit.

1.1.3 Vertical Scaling

Definition – Vertical Scaling / Scale Up

Vertical scaling means increasing the capacity of a single machine by adding or upgrading CPU cores, RAM, disks, and other resources within the same node.

Scale up is conceptually attractive because the software model often remains unchanged: the same machine simply has more resources. Its weakness appears at the high end, where price does not grow linearly with capacity.

Table 1.1: Illustrative vertical-scaling cost curve.

Configuration	Relative capacity	Relative cost
Commodity server	1×	1×
High-end server	4×	~ 10 × +
Top-of-line mainframe-class	16×	~ 100 × +

Premium hardware introduces a steep price premium. Beyond cost, vertical scaling eventually reaches physical limits imposed by motherboard capacity, power, cooling, and machine design. Vertical scaling therefore buys time, but it does not provide a long-term strategy for web-scale or exabyte-scale workloads.

1.1.4 Horizontal Scaling

Definition – Horizontal Scaling / Scale Out

Horizontal scaling means increasing total system capacity by adding more independent machines to a cluster rather than making a single machine larger.

The workload is distributed over many relatively modest nodes. Capacity grows by adding nodes, which makes horizontal scaling the foundational strategy behind big data systems.

A key design philosophy in systems such as GFS and HDFS is to build clusters from inexpensive commodity machines rather than from a few premium, highly reliable servers. Individual node failure is expected. Reliability is therefore provided by the distributed software rather than entrusted entirely to the hardware.

Key Concept

Scale-out infrastructure replaces the assumption of highly reliable individual hardware with software-level fault tolerance across many inexpensive components.

1.1.5 Vertical versus Horizontal Scaling

Table 1.2: Comparison between vertical and horizontal scaling across the main design dimensions.

Aspect	Vertical (Scale Up)	Horizontal (Scale Out)
Approach	Bigger single machine	More machines
Ceiling	Hard physical limit	Limited by coordination, network, and workload scalability
Cost growth	Superlinear at the high end	Roughly linear with commodity hardware
Failure handling	Hardware reliability	Software-level fault tolerance
Complexity	Lower; same software model	Higher; distributed coordination is required

The advantage of scale out is therefore not that distribution is simpler. It is that distribution avoids the hard capacity ceiling of one machine and allows the system to expand using commodity resources.

1.1.6 The New Failure Model

At cluster scale, hardware failure becomes a continuous background process. Disks fail, machines crash, network links become temporarily unavailable, and entire racks can lose power. A system designed under the assumption that failure is rare will encounter its "rare" cases continuously when deployed across thousands of components.

1.1.7 Cost per Byte and Cost per FLOP

Two economic dimensions become especially important at scale:

- **Cost per byte:** commodity disks make raw storage inexpensive, so replication and durability overhead become important.
- **Cost per FLOP:** commodity CPUs make raw computation inexpensive, so coordination and data movement become important.

In both cases, the challenge shifts away from the unit price of hardware and toward the efficiency with which the system coordinates many cheap components.

Review Questions

1. What is the fundamental difference between vertical and horizontal scaling?
2. Why does vertical scaling eventually hit a hard physical ceiling?
3. Why do big data systems prefer many commodity machines over a few premium machines?
4. Why must failure be treated as a normal event in scale-out systems?

1.2 Foundations of Distributed Storage

1.2.1 The Distributed File System Abstraction

Definition – Distributed File System

A distributed file system (DFS) presents a unified file-system interface while the underlying data is physically distributed across many machines. Files appear as part of one logical namespace even though their contents are split into blocks, replicated, and stored on multiple servers.

The DFS hides physical distribution behind ordinary file operations such as open, read, write, and close. The design target is petabyte-scale storage that remains usable through a familiar file abstraction.

1.2.2 Throughput versus Latency

Big data DFSs are designed primarily for *throughput*: the amount of data transferred per unit time across the cluster. They tolerate higher latency for individual operations because batch processing over very large datasets benefits more from aggregate bandwidth than from millisecond-scale response to isolated requests.

Remark – Design target

High latency is not itself desirable; the deliberate optimization target is aggregate throughput rather than minimum per-request latency for large batch workloads.

1.2.3 Huge Files and Sequential Access

The expected workload contains files ranging from gigabytes to terabytes and predominantly sequential access. Random small reads and writes are not the primary target. This assumption influences block size, metadata design, replication, and the write model.

1.2.4 Write-Once, Read-Many

A common lifecycle is that a file is written once, possibly through appends, then closed and read many times by analytical jobs. Arbitrary in-place updates are rare or disallowed. Relaxing support for random in-place modification removes a significant consistency and coordination burden.

1.2.5 The Block / Chunk Abstraction

Definition – Block / Chunk

A block or chunk is a storage unit with a configured maximum size into which a large file is divided; the final unit may be shorter. Blocks are stored, replicated, placed, and recovered independently, and different blocks of the same logical file may reside on different machines.

The fixed-size abstraction is one of the most important design decisions in distributed storage. It simplifies metadata, load balancing, replication, and later parallel computation.

1.2.6 Why Fixed-Size Blocks?

Tracking blocks rather than arbitrary byte ranges simplifies the metadata model. Uniform blocks are also easier to move and rebalance, and each block can be replicated independently using a uniform mechanism.

Big data DFSs therefore choose blocks that are orders of magnitude larger than conventional local file-system blocks. A small file does not reserve an entire large block on disk: the main small-file costs are metadata and limited parallelism.

Table 1.3: Block-size trade-offs in distributed storage.

Block size	Advantages	Disadvantages
Small (e.g. 4 KB)	Fine-grained placement; low waste for small files	Huge metadata overhead at scale
Large (e.g. 64–128 MB)	Low metadata overhead; efficient sequential I/O	Small-file metadata overhead; coarser load balancing

1.2.7 Metadata and Data Separation

A central architectural decision is to separate information about *where data is* from the *actual bytes*. The metadata service tracks the namespace and file-to-block or block-to-location mappings; data servers store block contents on local disks.

This separation allows metadata lookup and bulk I/O to be optimized independently.

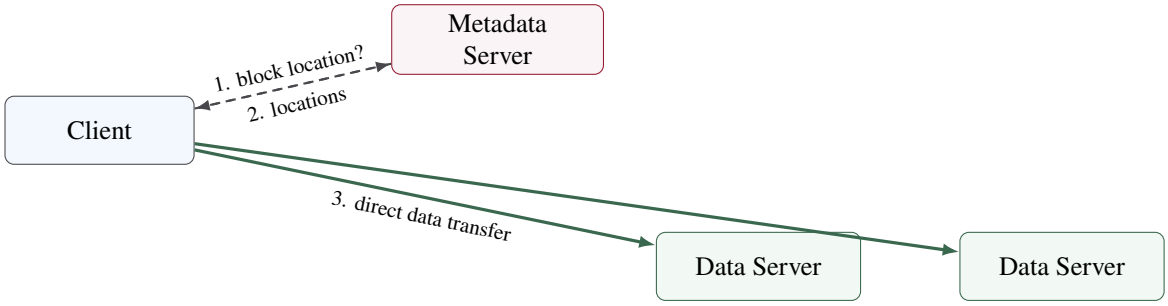


Figure 1.2: Generic DFS interaction model. Metadata is obtained from a coordinator, while bulk data transfer occurs directly between the client and data servers.

The metadata server is deliberately removed from the large data path. This avoids turning the coordinator into a bandwidth bottleneck.

1.2.8 Single Point of Coordination

A single metadata coordinator offers a globally consistent namespace view and avoids distributed consensus for every metadata update. The trade-off is equally direct: the coordinator may become both a single point of failure and a scalability bottleneck.

1.2.9 Namespace Management

The metadata service maintains the directory hierarchy, the mapping from files to blocks, and the mapping from blocks to replicas. This metadata is kept primarily in memory to support fast lookups.

1.2.10 Network Topology Awareness

Clusters are physically hierarchical. Machines belong to racks, racks are connected by switches, and communication within a rack is typically less costly than communication across racks. Placement and scheduling therefore consider physical topology rather than treating every machine as equally distant.

1.2.11 Why Replication Is Necessary

RAID can protect against some disk failures inside one machine, but it cannot protect against the loss of the entire node, rack, or network path. Distributed storage therefore replicates data across machines and, ideally, across racks.

Key Concept

Replication is not merely a performance optimization in a scale-out DFS. It is a primary durability mechanism that allows the logical file system to survive ordinary hardware failures.

Review Questions

1. Why do big data DFSs use large block sizes instead of small ones?
2. Why is metadata kept separate from the actual data servers?
3. In the client interaction model, what does the metadata server *not* do?
4. Why is RAID-style protection insufficient at cluster scale?

1.3 The Google File System (GFS)

1.3.1 Motivation and Original Context

The Google File System (GFS), published in 2003, was designed for Google's internal workloads: large web crawls, logs, and index files stored on thousands of commodity machines. It assumes frequent component failures, huge files, sequential reads, and append-heavy writes rather than general-purpose POSIX semantics.

GFS later became the direct architectural inspiration for HDFS.

1.3.2 Architecture Overview

GFS uses one Master for metadata and many Chunkservers for actual data.

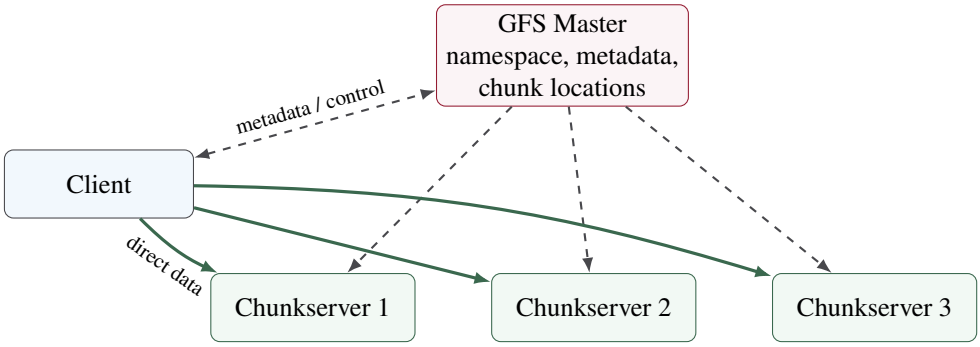


Figure 1.3: GFS architecture. The Master coordinates metadata, while Chunkservers store and serve the actual chunk data directly to clients.

1.3.3 The GFS Master

The Master is the single metadata-managing node. It maintains:

- the file-system namespace;
- file-to-chunk mappings;
- chunk-to-Chunkserver locations;
- chunk lease assignments;
- coordination of chunk creation, replication, deletion, and garbage collection.

Master metadata is kept in memory for speed. Chunk-location information is not persisted in the same

way as namespace mutations; it can be reconstructed by polling Chunkservers at startup.

The trade-off is that the namespace is bounded by the amount of Master memory.

1.3.4 Why a Single Master?

The design deliberately avoids a distributed metadata cluster. One Master greatly simplifies global decisions and avoids consensus on every metadata mutation. The risks are a potential bottleneck and a potential single point of failure, both of which require explicit mitigation.

1.3.5 Chunkservers

Chunkservers store GFS chunks as regular files on their local Linux file systems. They serve chunk reads and writes directly to clients and periodically report their state and chunk inventories to the Master.

1.3.6 Chunk Size

GFS uses 64 MB chunks. Large chunks reduce the number of chunks per file, decrease Master metadata, reduce client–Master lookup frequency, and encourage sustained sequential transfer. The accepted disadvantage is inefficiency for small files.

1.3.7 Replication Factor

The default replication factor is three. Replicas are placed on different Chunkservers with rack awareness, and the replication factor may be adjusted per file.

1.3.8 Master–Chunkserver Heartbeats

Heartbeats allow Chunkservers to report liveness, status, and chunk inventory. The Master can piggyback commands, such as replicate or delete, on heartbeat responses.

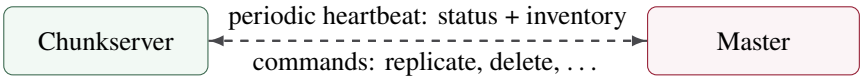


Figure 1.4: Master–Chunkserver heartbeat interaction in GFS.

1.3.9 GFS Read Path

For a read, the client first determines which chunk contains the requested file offset, obtains the chunk handle and replica locations from the Master, then communicates directly with a suitable Chunkserver.

1. The client computes the chunk index for the requested file offset.
2. The client asks the Master for the chunk handle and Chunkserver locations.
3. The Master returns the handle and the locations; the client may cache them briefly.
4. The client contacts the nearest replica directly.
5. The Chunkserver streams the data directly to the client.

Key Concept

The Master is on the metadata path, not on the bulk-data path. This separation keeps the metadata coordinator from becoming a data-transfer bottleneck.

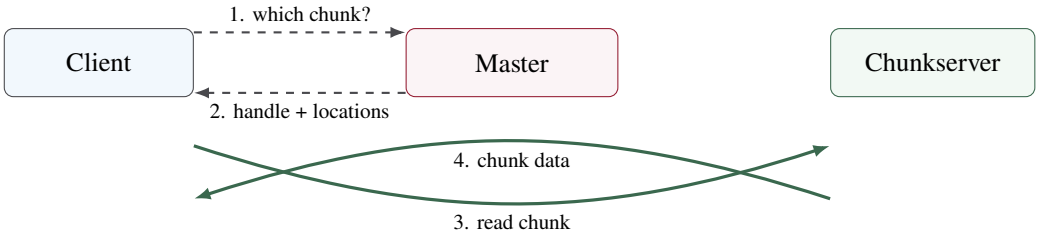


Figure 1.5: GFS read path. The Master supplies metadata; bulk data bypasses the Master and flows directly between the client and a Chunkserver.

1.3.10 GFS Write Path

GFS separates the transfer of bytes from the ordering of mutations. The original GFS design pipelines the data through a chain of replicas chosen to exploit network proximity; the primary is special for *control and ordering*, not because all data must pass through it.

1. The client asks the Master which Chunkservers hold the replicas and which replica currently holds the lease.
2. The client starts pushing the data to a nearby replica; the replicas forward the data along a carefully chosen chain until every replica has buffered it.
3. After all replicas have received the data, the client sends the write request to the primary replica.
4. The primary assigns a serial order to the mutation and forwards the ordered request to the secondary replicas.
5. The secondaries apply the mutation and acknowledge the primary; the primary then acknowledges the client.

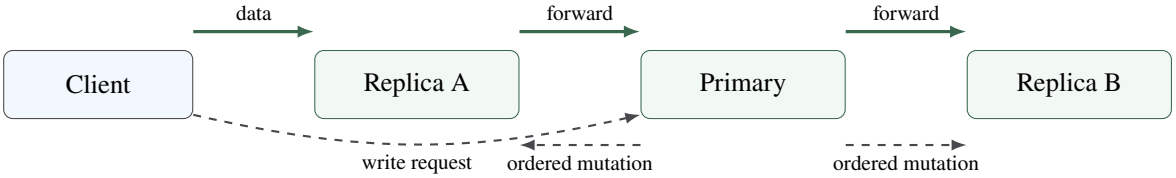


Figure 1.6: GFS write control and data flow. Data is pipelined through replicas, while mutation ordering is controlled by the lease-holding primary.

Remark – Data flow versus control flow

A simplified description often says that the client pushes data to all replicas. In the original GFS design, the replicas form a network-aware pipeline: the client initiates the transfer and each Chunkserver forwards data to the next. The conceptual point is unchanged: data movement is decoupled from the primary's ordering of mutations.

1.3.11 Primary and Secondary Replicas

The lease mechanism designates one replica as primary for a bounded period. The primary decides the serial order of concurrent mutations. Secondary replicas apply those mutations in the order dictated by the primary.

This delegates ordering to one replica rather than requiring the Master to order every write.

1.3.12 Mutation Ordering and Consistency

Successful writes use a common mutation order across replicas. Concurrent successful writes may leave a consistent but undefined region containing fragments of different writes; failed mutations can leave replicas inconsistent. Record append guarantees an atomic append at least once, so applications must handle possible duplicate records and padding. This is a relaxed consistency model rather than strict POSIX semantics.

1.3.13 Chunk Leases

The Master grants a time-bounded lease to a replica. The lease may be extended while the replica is actively handling mutations. If the primary becomes unreachable, a new lease can be granted after the old lease expires.

1.3.14 Garbage Collection

Deletion is lazy. A deleted file is renamed to a hidden name and retained for several days. Actual chunk storage is reclaimed later during normal Master–Chunkserver communication. This reduces synchronous cluster-wide deletion work and provides a short recovery window for accidental deletions.

1.3.15 Operation Log and Checkpoints

Namespace mutations are appended to a persistent operation log that is replicated to multiple machines. Periodically, the Master writes a compact checkpoint of its state. Recovery consists of loading the latest checkpoint and replaying subsequent log entries.

1.3.16 Shadow Masters

Shadow masters continuously apply the operation log and maintain read-only replicas of Master state. They can serve read-only metadata requests, but they do not automatically assume full write coordination after a primary Master failure.

1.3.17 Limitations

The original single-Master design introduces three major limitations:

- the namespace must fit in one machine's memory;
- high metadata-operation rates can overload the Master;
- Master failover is not instantaneous.

These limitations motivate later HDFS mechanisms such as Federation and explicit high-availability configurations.

Review Questions

1. What two types of information does the GFS Master manage, and why is metadata kept in memory?
2. Walk through the GFS read path: what is the Master's role and what is the Chunkserver's role?
3. What is the function of the chunk lease mechanism in the write path?
4. How does GFS survive a Master crash without losing namespace metadata?

1.4 Data Locality and Replication Strategy

1.4.1 The Principle of Data Locality

At big data scale, moving large datasets over the network is expensive. The preferred approach is therefore to move computation to the machine that already stores the required data, or at least to a nearby machine.

Definition – Data Locality

Data locality is the strategy of scheduling computation on or near the machine that already stores the required input block or chunk, thereby reducing network data movement.

Key Concept

Move computation to data, not data to computation. The storage layer exposes block locations so that the computation layer can exploit physical locality.

1.4.2 Why Network Bandwidth Is Scarce

Data locality reduces contention, but there is no universal ordering between disk and network speeds. The relevant resources are:

Table 1.4: Storage and network constraints relevant to data locality.

Resource	Main constraint
Local disk read	Storage-device bandwidth and competing local I/O
Same-rack network	Host links and switch capacity shared by active flows
Cross-rack network	Rack uplinks and backbone capacity; possible oversubscription

The bottleneck depends on the hardware and concurrent workload. Cross-rack traffic can be especially costly when rack uplinks are oversubscribed; local reads avoid consuming those links.

1.4.3 Rack-Aware Replica Placement

Replica placement balances two competing goals:

- **durability:** replicas should span independent failure domains;
- **efficiency:** excessive cross-rack placement increases network traffic during writes.

A common compromise is to keep some replicas in the same rack while placing at least one replica in a different rack.

1.4.4 Replication Factor Trade-offs

Definition – Replication Factor

The replication factor is the number of copies maintained for each chunk or block.

Increasing replication improves durability but consumes more storage and adds write traffic. The acknowledgement policy and handling of failed replicas determine when a write is accepted; the configured replication factor need not always be met immediately.

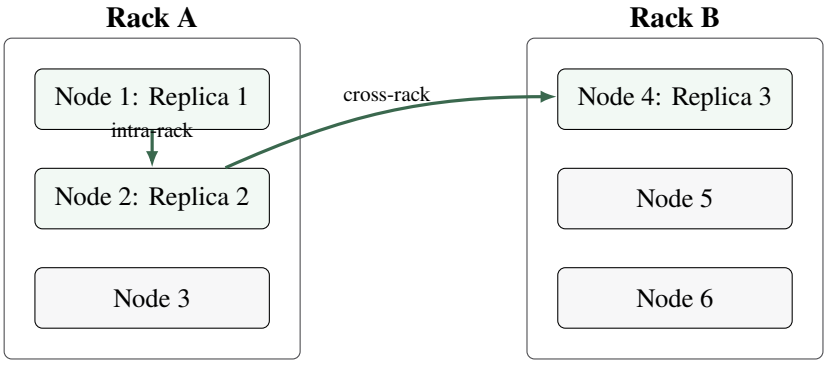


Figure 1.7: Rack-aware replication with two replicas in one rack and one replica in another rack. The placement balances durability against cross-rack write traffic.

Table 1.5: Replication factor, storage overhead, and simplified durability interpretation.

Replication factor	Storage overhead	Durability
1 (no replication)	1×	No redundancy; a failure can lose the only copy
2	2×	Tolerates one replica failure
3 (typical default)	3×	Tolerates two simultaneous replica failures in the simplified model

1.4.5 Failure Domains

Replication must consider correlated failures at multiple scales:

Table 1.6: Failure domains relevant to distributed storage.

Failure domain	Scope of impact
Disk	One local copy of one or more chunks
Node	All chunks stored on the failed machine
Rack	Multiple machines sharing a switch or power source
Datacenter	An entire facility, including power or network loss

Spreading replicas across failure domains is important precisely because failures are not always independent.

1.4.6 Worked Example: Replication and Data-Loss Probability

Consider a simplified one-year model with no repair or re-replication, in which each replica is permanently lost with independent probability $p = 0.02$. For r replicas, the probability of losing all copies by the end of that year is

$$P(\text{total loss}) = p^r. \tag{1.1}$$

For $p = 0.02$,

Table 1.7: Simplified independent-failure illustration for $p = 0.02$.

Replication factor r	$P(\text{loss}) = p^r$
1	0.02
2	0.0004
3	0.000008

Remark – Simplified independence model

This no-repair example illustrates the multiplicative effect of independent replicas. It is not an estimate of annual data-loss risk in a self-healing cluster: real durability also depends on repair time, overlapping failures, and shared failure domains.

1.4.7 Re-Replication after Failure

A failed node does not leave the cluster permanently under-replicated. The recovery process is:

1. the Master or NameNode detects the failed node through missed heartbeats;
2. affected blocks are marked under-replicated;
3. the system selects surviving replicas;
4. new copies are scheduled on healthy nodes;
5. the desired replication factor is restored automatically.

1.4.8 Balancing Storage Utilization

Replica placement and re-replication also consider disk utilization. A balanced cluster avoids concentrating too much data on a small subset of machines, thereby reducing hotspots and premature capacity exhaustion.

Review Questions

1. Why is moving computation to data preferable to moving data to computation?
2. How does rack-aware placement balance durability against write cost?
3. Under the simplified independent-failure model, why does moving from two to three replicas reduce loss probability so strongly?
4. What happens after a Chunkserver or DataNode permanently fails?

1.5 Hadoop Distributed File System (HDFS)

1.5.1 Origins and Relation to GFS

HDFS was designed as an open-source distributed storage system directly inspired by the 2003 GFS paper. It was built within Apache Hadoop to support large-scale, fault-tolerant batch processing and originally served the MapReduce execution model.

Its central architecture is nearly the same as GFS: one metadata-managing node, many data-storing nodes, large fixed-size blocks, and replication.

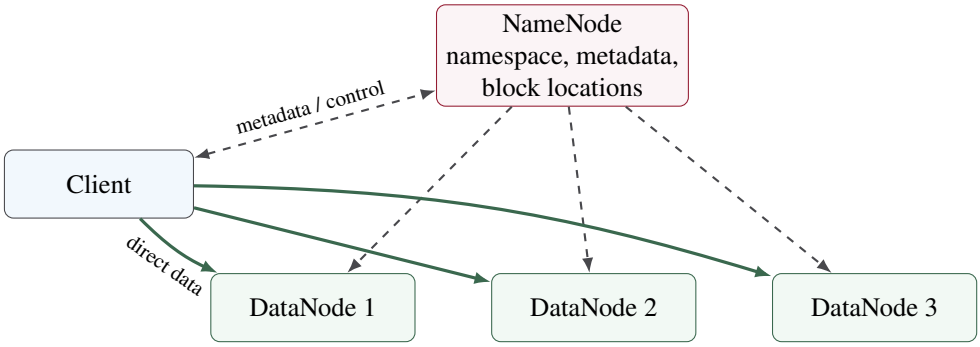


Figure 1.8: HDFS architecture. The NameNode plays the metadata role of the GFS Master, while DataNodes correspond to GFS Chunkservers.

1.5.2 Architecture

1.5.3 NameNode

The NameNode maintains the namespace, permissions, file-to-block mappings, block-to-DataNode locations, and coordination decisions for replication, deletion, and rebalancing. The metadata is kept in memory for fast access.

1.5.4 DataNodes

DataNodes store HDFS blocks as files on their local underlying file systems and serve block reads and writes directly to clients. They periodically send heartbeats and block reports to the NameNode.

1.5.5 Block Size

Table 1.8: GFS chunk size and HDFS block size.

System	Default size	Typical range
GFS chunk	64 MB	Fixed at 64 MB
HDFS block	128 MB	Configurable, commonly 128–256 MB

Both systems use large units for the same reasons: reduced metadata and efficient sequential transfer.

1.5.6 FsImage and EditLog

HDFS persists NameNode metadata using two structures:

- **FsImage:** a complete point-in-time snapshot of the namespace;
- **EditLog:** an append-only log of namespace mutations since the most recent FsImage.

This is analogous to GFS's checkpoint plus operation log.

1.5.7 NameNode Startup

On restart, the NameNode loads the most recent FsImage and replays EditLog entries recorded since that snapshot. The resulting in-memory namespace reconstructs the state that existed before the restart. A new checkpoint can then bound future replay time.

1.5.8 Secondary NameNode

Remark – Secondary NameNode misconception

The Secondary NameNode is *not* a hot standby and is not the component that automatically takes over when the primary NameNode fails. Its role is to merge FsImage and EditLog periodically into a new checkpoint, thereby offloading checkpointing work.

1.5.9 High Availability

Production HDFS can use Active/Standby NameNodes. One Active node handles client requests while one or more Standby nodes remain synchronized through a shared, consistent edit log. Automatic failover can promote a Standby when the Active node fails.

1.5.10 Heartbeats and Block Reports

HDFS separates two periodic mechanisms:

- **heartbeats:** frequent liveness messages;
- **block reports:** less frequent full inventories of blocks stored on a DataNode.

Together, they allow the NameNode to maintain an accurate view of node health and block distribution.

1.5.11 HDFS Read Path

1. The client asks the NameNode for the locations of the blocks that form the target file.
2. The NameNode returns block identifiers and an ordered list of DataNodes for each block.
3. The client connects directly to the nearest available DataNode.
4. The selected DataNode streams block data directly to the client.

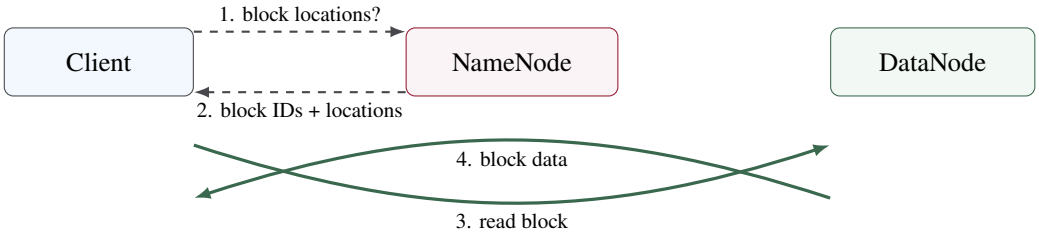


Figure 1.9: HDFS read path. As in GFS, the metadata lookup is separated from the direct block-data transfer.

1.5.12 HDFS Write Pipeline

HDFS also uses a node-to-node replication pipeline, but its write protocol is organized differently from GFS: the client streams packets through a DataNode chain and acknowledgements return through that chain, whereas GFS separately uses a lease-holding primary to serialize mutations after the data has been propagated.

1. The client asks the NameNode to create a file; the NameNode checks permissions and allocates a block.
2. The NameNode returns DataNodes that form the replication pipeline.
3. The client streams data to the first DataNode.

4. Each DataNode forwards the data to the next DataNode.
5. Acknowledgements return in the reverse direction after the pipeline has processed the packets. An ordinary acknowledgement does not imply a disk-device sync; `hsync()` provides that stronger synchronization request.

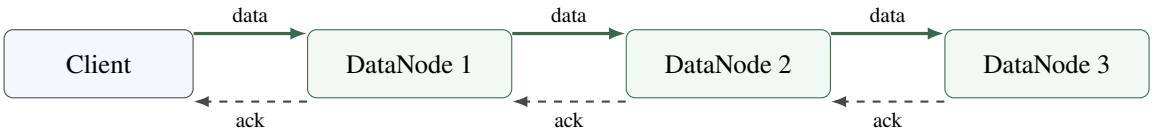


Figure 1.10: HDFS write pipeline. Data flows through a chain of DataNodes and acknowledgements return in the reverse direction.

1.5.13 Replica Placement Policy

For replication factor three, HDFS uses the following rack-aware placement pattern:

1. first replica on the local or a nearby DataNode;
2. second replica on a different rack;
3. third replica on a different node in the same rack as the second.

This preserves cross-rack durability while limiting the number of cross-rack transfers.

Rack awareness must be configured explicitly. Without a mapping from DataNodes to racks, HDFS may treat the cluster as a single rack and lose the intended protection against rack-level failures.

1.5.14 Federation

HDFS Federation addresses NameNode metadata scalability by introducing multiple independent NameNodes. Each manages a distinct portion of the namespace, while DataNodes register with and store blocks for all NameNodes in the federation.

Each NameNode still has its own in-memory limit, but aggregate namespace capacity scales with the number of NameNodes.

1.5.15 GFS and HDFS: Terminology and Architecture

Table 1.9: Terminology mapping between GFS and HDFS.

Concept	GFS term	HDFS term
Metadata node	Master	NameNode
Data-storing node	Chunkserver	DataNode
Storage unit	Chunk	Block
Default unit size	64 MB	128 MB
Recovery / checkpoint support	Shadow masters (read-only replicas)	Secondary NameNode (checkpoint helper)

Remark – No exact one-to-one equivalent

GFS shadow masters and the HDFS Secondary NameNode solve different supporting problems. Shadow masters maintain read-only replicas of Master state, whereas the Secondary NameNode

> periodically merges the FsImage and EditLog to create checkpoints; neither should be interpreted as a simple hot-standby equivalent of the other.

Table 1.10: Architectural comparison between GFS and HDFS.

Aspect	GFS	HDFS
Metadata coordinator	Master	NameNode
Data node	Chunkserver	DataNode
Storage unit	Chunk	Block
Default unit size	64 MB	128 MB
Write coordination	Pipelined data propagation; primary/lease serializes mutations	Client-to-DataNode pipeline with reverse acknowledgements
Metadata persistence	Checkpoint + operation log	FsImage + EditLog
Metadata scaling	Single-Master limitations	Federation adds multiple NameNodes
Availability	Shadow masters plus recovery/promotion mechanisms	Explicit Active/Standby HA configuration

1.5.16 Why Blocks Suit Batch Processing

Blocks are independently processable units. Their physical locations can be exposed to a scheduler, which enables one task to be placed near the block it processes. This storage abstraction therefore leads directly to MapReduce-style parallelism.

1.5.17 Append-Only, No Random Writes

HDFS follows the same write-once-read-many philosophy introduced earlier. Append operations are supported, but arbitrary in-place modification of existing file contents is not part of the classic model. This restriction simplifies consistency and replication.

Review Questions

1. Map NameNode, DataNode, and block to their GFS counterparts.
2. What is the actual role of the Secondary NameNode?
3. How do the HDFS and GFS write protocols differ in control and ordering, even though both use pipelined data propagation?
4. How does Federation address the NameNode metadata bottleneck?
5. Why does block-based storage naturally support parallel batch computation?

1.6 Failure Handling and Fault Tolerance

1.6.1 Failure Is the Default Assumption

With thousands of disks and machines, failures are a continuous background process. Replication, heartbeats, leases, logs, checkpoints, and checksums exist so that the system can degrade and recover gracefully rather than depend on the absence of failure.

1.6.2 Types of Failures

Failures can be divided into three broad categories:

- **Transient failures:** temporary network interruptions or brief pauses;
- **Permanent failures:** disk death, hardware failure, or node decommissioning;
- **Correlated failures:** failures affecting multiple machines simultaneously, such as rack power or switch loss.

1.6.3 Detecting Failure: Heartbeats and Timeouts

Each data node sends periodic heartbeats to the metadata node. If no heartbeat arrives before a timeout, the node is presumed dead.

Table 1.11: Heartbeat timeout trade-off.

Timeout	Detection behaviour	Main risk
Short	Fast failure detection	False positives for slow but healthy nodes; unnecessary re-replication
Long	Fewer false positives	Slower detection of real failures; longer period of reduced redundancy

There is no universally correct timeout; it must be chosen for the cluster's workload and network conditions.

1.6.4 Recovery from Chunkserver / DataNode Failure

The recovery chain is:

$$\text{missed heartbeat} \Longrightarrow \text{node presumed dead} \Longrightarrow \text{blocks marked under-replicated}$$
$$\Longrightarrow \text{new replicas scheduled} \Longrightarrow \text{replication factor restored.}$$

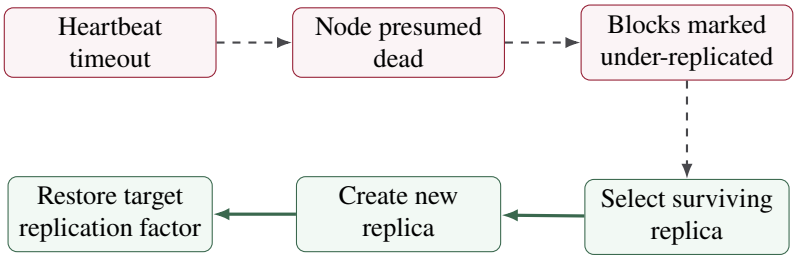


Figure 1.11: Simplified self-healing chain after a data-storage node failure.

1.6.5 Recovery from Master / NameNode Failure

Without high availability, the cluster cannot accept new operations until the metadata node is restarted and its state reconstructed from a checkpoint plus log. With HA configured, a synchronized Standby can be promoted to Active, greatly reducing interruption.

1.6.6 Checksums and Silent Corruption

Crashes are not the only failure mode. A disk may return corrupted data without reporting a hardware error. Both GFS and HDFS therefore store checksums alongside chunks or blocks.

A read verifies the checksum. If a mismatch is detected, another valid replica is used and the corrupted

replica can be replaced.

$$\begin{aligned}\text{read block} &\Longrightarrow \text{verify checksum} \Longrightarrow \text{detect mismatch}\\ &\Longrightarrow \text{read valid replica} \Longrightarrow \text{regenerate replica.}\end{aligned}$$

Key Concept

Heartbeats detect missing nodes; block reports describe data placement; checksums detect silent corruption; re-replication restores the desired redundancy. Together these mechanisms produce the self-healing behaviour of the storage system.

1.6.7 Real-World Failure Rates at Scale

Disk failure rates are often summarized using an Annualized Failure Rate (AFR). The supplied material uses a representative range of roughly 1% to 5% AFR, depending on disk age, model, and operating conditions.

The worked example uses 10,000 disks and a 2% annual failure rate.

1.6.8 Modeling Disk Failures in One Week

Consider

$$N = 10{,}000$$

disks with annual failure rate

$$\text{AFR} = 2\% = 0.02.$$

Let p denote the probability that one disk fails during one week. Under the simplifying assumption that failures are uniformly distributed throughout the year,

$$p \approx \frac{0.02}{52} \approx 0.0003846. \tag{1.2}$$

For each disk, define the indicator variable

$$X_i = \begin{cases} 1, & \text{if disk } i \text{ fails during the week,} \\ 0, & \text{otherwise.} \end{cases} \tag{1.3}$$

Then

$$X_i \sim \text{Bernoulli}(p).$$

The total number of failures in the week is

$$X = \sum_{i=1}^{N} X_i. \tag{1.4}$$

Under the stated assumption that the disk failures are independent and identically distributed,

$$X \sim \text{Binomial}(N, p), \qquad N = 10{,}000, \quad p \approx 0.0003846. \tag{1.5}$$

1.6.9 Probability of Exactly k Failures

For a binomial random variable,

$$\Pr(X = k) = \binom{N}{k} p^k (1-p)^{N-k}. \tag{1.6}$$

For exactly four failures in one week,

$$\Pr(X = 4) = \binom{10{,}000}{4} (0.0003846)^4 (1 - 0.0003846)^{9996} \approx 19.5\%.$$

Thus, under this simplified model, exactly four failures in a week is not an unusual outcome.

1.6.10 Expected Number of Failures

For a binomial random variable,

$$\mathbb{E}[X] = Np. \tag{1.7}$$

Therefore,

$$\mathbb{E}[X] = 10{,}000 \cdot \frac{0.02}{52} \approx 3.85.$$

An important interpretation is that

$$\mathbb{E}[X] \approx 3.85 \qquad \nRightarrow \qquad X = 3.85.$$

The random variable X takes integer values,

$$X \in \{0, 1, 2, 3, \ldots, N\},$$

whereas 3.85 is the long-run average.

1.6.11 Variance and Standard Deviation

For a binomial random variable,

$$\text{Var}(X) = Np(1-p). \tag{1.8}$$

With these values,

$$\text{Var}(X) = 10{,}000(0.0003846)(1 - 0.0003846) \approx 3.84.$$

Hence,

$$\sigma_X = \sqrt{\text{Var}(X)} \approx 1.96.$$

The weekly number of failures therefore fluctuates naturally around its expectation.

1.6.12 From AFR to Failures per Week

The complete chain is

$$\text{AFR} = 0.02, \qquad p_{\text{week}} \approx \frac{0.02}{52} \approx 0.0003846,$$

$$X \sim \text{Binomial}(10{,}000, 0.0003846),$$

and

$$\mathbb{E}[X] = Np \approx 3.85.$$

This is the central scaling effect: a small annual probability for one disk becomes a regular stream of failures when the number of disks is very large.

1.6.13 Probability of at Least One Failure

The probability of at least one failure is most conveniently computed using the complement:

$$\Pr(X \geq 1) = 1 - \Pr(X = 0). \tag{1.9}$$

For a binomial random variable,

$$\Pr(X = 0) = (1 - p)^N,$$

so

$$\Pr(X \geq 1) = 1 - (1 - p)^N. \tag{1.10}$$

Substituting these values,

$$\Pr(X \geq 1) = 1 - (1 - 0.0003846)^{10{,}000} \approx 97.9\%.$$

Thus, under the stated simplifying assumptions, a cluster of 10,000 disks experiences at least one disk failure in roughly 98% of weeks.

Remark – Model assumptions

The calculation deliberately uses a simplified iid model and the approximation $p \approx \text{AFR}/52$. Its purpose is to quantify the scaling effect, not to provide a complete empirical model of correlated or age-dependent disk failures.

1.6.14 Example: Chunkserver Dies Mid-Read

Example – Chunkserver failure during an active read

Suppose a client is reading a chunk from Chunkserver A and A crashes during the transfer.
Client-visible sequence:

1. the current transfer fails or times out;
2. the client re-queries the Master for alternative replica locations;
3. the client retries the read against another Chunkserver, such as B or C;
4. the application-level read succeeds using the surviving replica.

Background recovery sequence:

1. the Master notices A's missed heartbeats;
2. chunks formerly stored on A are marked under-replicated;
3. re-replication is scheduled from surviving copies to healthy nodes.

Review Questions

1. Why must heartbeat timeouts balance detection speed against false positives?
2. What happens after a DataNode permanently fails?
3. Why are checksums necessary even when data is replicated?
4. Why does a cluster of thousands of disks experience failures on a weekly rather than yearly timescale?
5. In the mid-read failure example, which steps are visible to the client and which happen in the background?

1.7 From Storage to Computation

1.7.1 GFS and HDFS Side by Side

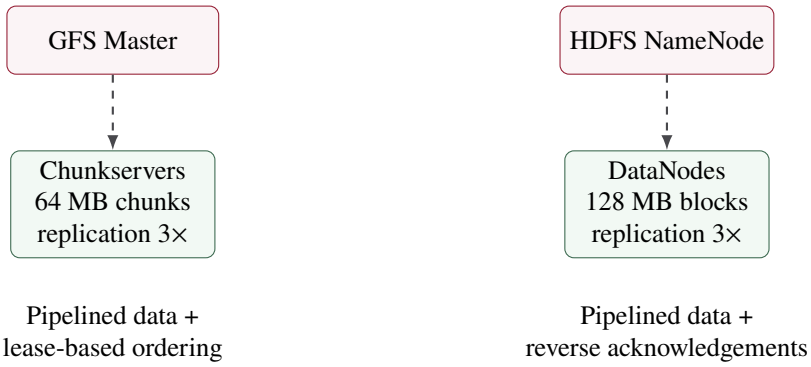


Figure 1.12: High-level GFS/HDFS architectural recap. Both systems separate metadata coordination from replicated data storage and use pipelined replication, while their write-control mechanisms differ.

The two systems share the same core architectural idea even though terminology and some mechanics differ.

1.7.2 Storage Enables Compute

The storage design is significant because it directly enables distributed computation:

$$\text{fixed-size blocks} \Longrightarrow \text{parallel units of computation},$$

$$\text{known block locations} \Longrightarrow \text{data-local scheduling},$$

$$\text{replicated, fault-tolerant storage} \Longrightarrow \text{reliable substrate for distributed computation.}$$

A computation framework does not need to invent its own storage layer. It can schedule work over data already distributed and protected by GFS or HDFS.

1.7.3 MapReduce on Top of Distributed Storage

MapReduce builds a computation layer directly on top of this storage foundation. A MapReduce job reads data stored as blocks or chunks, and Map tasks can be placed on or near the machines that hold those inputs. Outputs are written back to the distributed file system and inherit its replication and fault-tolerance mechanisms.

The following chapter develops the Map, Shuffle, Sort, and Reduce execution model in detail; the essential connection is that computation inherits the storage layer's block structure, locality information, and fault-tolerant substrate.

1.7.4 Limits of the GFS/HDFS Model

This architecture has several important limitations:

- optimization for large files and sequential access makes the model a poor fit for very large numbers of small files or random-access workloads;
- a centralized metadata architecture imposes namespace scalability limits, even when HA and Federation mitigate different aspects of the problem;
- write-once-read-many semantics are restrictive for workloads requiring frequent in-place updates.

Later topics such as Spark, streaming, and lakehouse architectures address different parts of this design space.

1.7.5 Three Pillars of Scale-Out Storage

The architecture can be distilled into three structural principles:

1. **Horizontal scaling over commodity hardware:** many inexpensive, individually unreliable machines rather than a few expensive machines.
2. **Block-based storage with replication:** fixed-size units simplify placement, parallelism, durability, and recovery.
3. **Separated metadata management and data transfer:** a coordinating node tracks locations and namespace state, while bulk data flows directly between clients and data-storing nodes.

1.8 Key Terms

1.9 Chapter Summary

Table 1.12: Key terminology for scale-out distributed storage.

Term	Meaning
Scale up	Increase the capacity of one machine.
Scale out	Increase capacity by adding machines to a cluster.
Distributed File System	Logical file system backed by storage distributed across many machines.
Chunk / Block	Fixed-size unit of distributed file storage; 64 MB in GFS and 128 MB by default in the HDFS material.
Master / NameNode	Metadata-managing coordinator.
Chunkserver / DataNode	Node storing and serving actual data.
Replication factor	Number of copies maintained for each chunk or block.
Heartbeat	Periodic liveness signal from a data node to the coordinator.
Block report	HDFS report containing the blocks held by a DataNode.
Data locality	Scheduling computation near the data it consumes.
Rack awareness	Placement strategy that accounts for physical failure and bandwidth domains.
Lease	Time-bounded designation of a GFS replica as primary for ordering mutations.
FsImage	Point-in-time snapshot of the HDFS namespace.
EditLog	Append-only log of HDFS namespace mutations.
Re-replication	Automatic reconstruction of lost or corrupted replicas.
Checksum	Integrity value used to detect silent data corruption.

Table 1.13: Compact summary of the central relationships in scale-out distributed storage.

Concept	Main role	Important relationship / trade-off
Scale Up	Grow one machine	Simple software model, but high-end cost grows steeply and physical limits remain.
Scale Out	Grow a cluster	Capacity grows by adding nodes, but distributed coordination becomes necessary.
Commodity hardware	Cheap cluster components	Reliability moves from individual hardware into distributed software mechanisms.
Large blocks / chunks	Storage unit	Reduce metadata and favour sequential I/O, but are inefficient for small-file workloads.
Metadata coordinator	Track namespace and locations	Central coordination is simple and fast but may become a bottleneck and availability concern.
Direct data transfer	Keep bulk bytes off metadata node	Prevents the coordinator from becoming a throughput bottleneck.
Replication	Provide redundancy	Improves durability at the cost of storage capacity and write traffic.

Concept	Main role	Important relationship / trade-off
Rack awareness	Spread replicas across failure domains	Balances correlated-failure protection against cross-rack bandwidth cost.
Data locality	Place computation near data	Reduces network transfer of large input datasets.
GFS Master	Manage GFS metadata	Uses in-memory metadata, operation log, checkpoints, leases, and Chunkserver reports.
GFS Chunkserver	Store GFS chunks	Serves data directly and reports chunk state to the Master.
HDFS NameNode	Manage HDFS metadata	Uses in-memory state plus FsImage/EditLog; HA and Federation address availability and scaling separately.
HDFS DataNode	Store HDFS blocks	Serves blocks directly and sends heartbeats plus block reports.
GFS write path	Order mutations	Client pushes data to replicas; a leased primary serializes mutation order.
HDFS write path	Replicate data	Data streams through a DataNode pipeline and acknowledgements return backwards.
Heartbeat	Detect suspected failure	Short timeouts detect quickly but risk false positives; long timeouts delay recovery.
Checksum	Detect silent corruption	Complements replication by detecting bad copies that still appear reachable.
Re-replication	Restore redundancy	Converts surviving replicas into replacement copies after failure or corruption.
FsImage + EditLog	Persist HDFS namespace	Snapshot plus mutation log reconstructs the NameNode's in-memory state.
Secondary NameNode	Checkpoint helper	Merges FsImage and EditLog; it is not the HA standby.
Federation	Scale HDFS namespace	Multiple independent NameNodes increase aggregate namespace capacity.
High Availability	Reduce metadata-node downtime	Active/Standby NameNodes allow failover with synchronized edits.
Block-based storage	Foundation for batch processing	Independent blocks support parallel tasks and data-local scheduling.

1.10 Quick Review

1. Big data infrastructure begins from the assumption that no single machine is infinitely large, fast, or reliable.

2. Vertical scaling enlarges one machine; horizontal scaling adds machines.

3. Scale up is simple but eventually reaches both economic and physical ceilings.
4. Scale out relies on software-level coordination and fault tolerance over commodity hardware.
5. At cluster scale, component failure is normal rather than exceptional.
6. Big data DFSs optimize aggregate throughput rather than minimum single-request latency.
7. Their intended workload contains very large files, sequential access, and write-once-read-many behaviour.
8. Fixed-size blocks or chunks simplify metadata, placement, replication, load balancing, and recovery.
9. Metadata is separated from bulk data so that the coordinator performs lookups rather than carrying file contents.
10. GFS uses a Master and Chunkservers; HDFS uses a NameNode and DataNodes.
11. GFS uses 64 MB chunks; HDFS uses a 128 MB default block size in this material.
12. GFS keeps Master metadata in memory for speed and persists namespace mutations through an operation log and checkpoints.
13. GFS clients obtain chunk metadata from the Master but transfer data directly with Chunkservers.
14. GFS writes decouple data transfer from mutation ordering; a leased primary orders writes for secondary replicas.
15. HDFS persists namespace state through FsImage and EditLog.
16. The Secondary NameNode is a checkpointing helper, not a hot standby.
17. HDFS HA uses Active and Standby NameNodes; Federation uses multiple NameNodes to scale namespace capacity.
18. Both GFS and HDFS pipeline replica data; GFS additionally uses a lease-holding primary to serialize mutations, while HDFS uses a DataNode pipeline with reverse acknowledgements.
19. Replication improves durability but consumes storage and increases write traffic.
20. Rack-aware placement spreads replicas across failure domains while limiting cross-rack communication.
21. Data locality reduces network traffic by scheduling computation near the blocks it needs.
22. Heartbeats detect missing nodes; block reports describe stored data; checksums detect silent corruption.
23. Re-replication restores the intended replication factor after failure or corruption.
24. A short heartbeat timeout improves detection speed but increases false positives; a long timeout has the opposite trade-off.
25. Under the simplified model, 10,000 disks with 2% AFR yield an expected 3.85 failures per week and roughly 97.9% probability of at least one weekly failure.
26. The block abstraction naturally supports parallel batch computation and data-local Map scheduling.
27. MapReduce is built on top of the storage mechanisms introduced here rather than replacing them.

1.11 Further Reading

Further reading includes:

- Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). *The Google File System*. SOSP, especially Sections 2.5–2.7 and 3.1–3.3.
- Apache Hadoop. *HDFS Architecture*.
- Apache Hadoop. *FSDataOutputStream API*, including `hflush()` and `hsync()`.

CHAPTER 2

The MapReduce Execution Model

Distributed storage solves the problem of retaining datasets that exceed the capacity of a single machine, but stored data still needs to be processed. A computation framework must partition work, place tasks near data, move intermediate results, coordinate aggregation, and recover from worker failures without forcing every programmer to implement those mechanisms manually.

MapReduce addresses this problem with a deliberately small programming interface. Users provide Map and Reduce functions, while the runtime transforms those functions into many parallel tasks, schedules them across a cluster, routes intermediate records by key, and reconstructs work after failures. The result is a batch-processing model in which simple deterministic computations can be executed across thousands of unreliable machines.

Key Concept

MapReduce separates application logic from distributed execution. The programmer specifies how records are transformed and aggregated; the runtime supplies task scheduling, communication, grouping, fault recovery, and data-local placement.

2.1 The MapReduce Abstraction

2.1.1 The Problem Being Solved

Before a reusable framework, every large-scale batch-processing program had to solve the same distributed problems:

- split work across machines;
- detect and recover from worker failure;
- move intermediate results between machines;
- combine partial results;
- coordinate completion.

MapReduce moves these concerns into the framework so that application programmers can concentrate on task-specific logic.

2.1.2 Functional Programming Roots

The model borrows two familiar higher-order operations:

$$\text{map}(f, L): \quad \text{apply } f \text{ independently to each element of } L,$$

and

$$\text{reduce}(g, L): \quad \text{combine elements of } L \text{ into an aggregate result using } g.$$

The absence of dependencies among independent map applications is especially important: independent records can be processed in parallel and in arbitrary order.

2.1.3 The Programming Model

From the user's perspective, a MapReduce job is primarily defined by two functions:

$$\text{map}(k, v) \longrightarrow [(k'_1, v'_1), \ldots, (k'_m, v'_m)] \tag{2.1}$$

and

$$\text{reduce}\big(k', [v'_1, \ldots, v'_n]\big) \longrightarrow [\text{output values}]. \tag{2.2}$$

The programmer does not explicitly write network protocols, worker scheduling, retry logic, or task placement.

Definition – Map Function

The Map function processes one input $(key, value)$ pair independently and emits zero, one, or many intermediate $(key, value)$ pairs.

Definition – Reduce Function

The Reduce function receives one intermediate key together with the complete collection of intermediate values associated with that key and emits the final output value or values for that key.

2.1.4 Map Semantics

Each Map invocation has no visibility into unrelated input records and cannot rely on mutable state produced by other Map calls. That independence allows the runtime to schedule millions of Map invocations across workers in any order.

2.1.5 Reduce Semantics

All values belonging to the same intermediate key are grouped before the Reduce call. Different keys are independent and can therefore be processed by different Reduce tasks in parallel.

2.1.6 Worked Example: Word Count

The canonical example counts every word across a large collection of documents.

Suppose the input consists of document identifier and document text pairs. The Map logic is

$$\text{map}(\text{document_id}, \text{document_text})$$

with the operational rule

$$\text{for each word } w: \qquad \text{emit}(w, 1).$$

No global counting occurs during Map. The Map function merely reports one observation per encountered word.

Reduce receives one word and the complete list of counts produced for that word:

$$\text{reduce}(w, [1, 1, \ldots, 1])$$

and computes

$$\text{total} = \sum_i 1, \qquad \text{emit}(w, \text{total}).$$

Example – Word Count

For documents containing the word `the` several times, each Map task emits (`the`, 1) independently. The framework later guarantees that every value associated with `the` reaches the same Reduce call, where the values are summed.

2.1.7 Second Example: Inverted Index

The same abstraction can build an inverted index:

$$\text{map}(\text{document_id}, \text{document_text})$$

emits

$$(w, \text{document_id})$$

for each word w. Reduce receives all document identifiers associated with a word and emits a unique, sorted list.

The execution model does not change; only the intermediate values and aggregation logic differ.

2.1.8 What the Abstraction Hides

The programmer does not specify:

- which worker runs a particular Map or Reduce call;
- how intermediate data moves physically;
- how to recover from worker crashes;
- how values are grouped by key.

These responsibilities belong to the MapReduce runtime.

2.1.9 Determinism and Side Effects

Determinism is a central assumption for fault tolerance. Given the same input, a re-executed task must reproduce the same output.

Definition – Deterministic, Side-Effect-Free Task

A Map or Reduce computation is suitable for transparent re-execution when its result is determined by its input and it does not rely on or mutate external state in a way that changes the observable result of a retry.

If a worker fails, the runtime can rerun the task elsewhere. This retry strategy is safe because the repeated computation is expected to produce the same logical output.

Key Concept

MapReduce fault tolerance relies on the ability to re-execute work. Deterministic and side-effect-free user functions make "just run the task again" a valid recovery strategy.

Review Questions

1. What are the signatures and roles of the Map and Reduce functions?
2. In word count, what does Map emit, and why does it not perform the global count?
3. Why must Map and Reduce be deterministic and side-effect-free?
4. Which distributed-systems concerns are hidden from the programmer?

2.2 Execution Architecture

2.2.1 From Program to Cluster Job

Turning the user's functions into a distributed job involves a pipeline:

1. split input into independent pieces;
2. create and schedule Map tasks;
3. partition and shuffle intermediate output;
4. group and sort intermediate keys;
5. run Reduce tasks;
6. write final output.

2.2.2 The Master / JobTracker

A central coordinator, called the Master in Google's original implementation or the JobTracker in Hadoop MapReduce v1, manages the computation:

- creates Map and Reduce tasks;
- assigns them to workers;
- tracks progress;
- detects worker failure;
- reschedules work after failure.

Conceptually, this resembles the coordination role of the GFS Master or HDFS NameNode, but it manages *computation* rather than the storage namespace.

2.2.3 Input Splits Become Map Tasks

MapReduce creates one Map task per logical input split. Splits often follow GFS/HDFS block boundaries, but they are not identical to physical blocks: the input format, compression, and record boundaries affect splitting.

$$\text{distributed storage block} \implies \text{natural unit of Map parallelism.}$$

This coupling is intentional: MapReduce and GFS were designed together.

2.2.4 Data Locality in Scheduling

The coordinator attempts to schedule each Map task on the worker storing its input. If no perfectly local worker is available, it prefers a worker in the same rack before falling back to a more distant machine.

Key Concept

The principle "move computation to data" becomes a concrete scheduling rule in MapReduce: place Map tasks near the blocks they process.

2.2.5 Number of Map Tasks versus Machines

The number of Map tasks may be far larger than the number of workers. For example, thousands of blocks can produce thousands of Map tasks on a cluster containing only hundreds of machines. Tasks therefore run in successive waves.

This over-decomposition improves load balancing and later helps straggler mitigation: work is divided into units small enough to redistribute.

2.2.6 Intermediate Output Stays Local

Map output is temporary and is written to the local disk of the worker that executed the Map task rather than directly to the distributed file system.

The reason is economic: intermediate data is required only until the Reduce stage. Replicating it through HDFS/GFS would introduce unnecessary storage and network overhead.

2.2.7 Partitioning Intermediate Keys

All intermediate pairs sharing the same key must reach the same Reduce task. A partition function maps each intermediate key to a reducer.

The default strategy is hash partitioning:

$$\text{partition}(k) = \text{hash}(k) \bmod R, \tag{2.3}$$

where R is the number of Reduce tasks.

Because the function is deterministic, identical keys emitted by different Map tasks map to the same reducer.

Definition – Partition Function

A partition function assigns every intermediate key to exactly one Reduce-task partition. It therefore determines where the values associated with a key are routed during shuffle.

2.2.8 Choosing the Number of Reduce Tasks

Unlike the number of Map tasks, which is determined by input splits, the number of Reduce tasks is typically user-controlled.

A common rule of thumb is to choose a reducer count on the order of the number of available workers, sometimes a small multiple of it.

Table 2.1: Trade-off in choosing the number of Reduce tasks.

Choice	Consequence
Too few Reduce tasks	Each reducer receives a large share of keys, reducing parallelism and risking an overloaded "hot" reducer.
Too many Reduce tasks	More output files, more shuffle coordination, and diminishing returns once reducer parallelism exceeds available worker capacity.

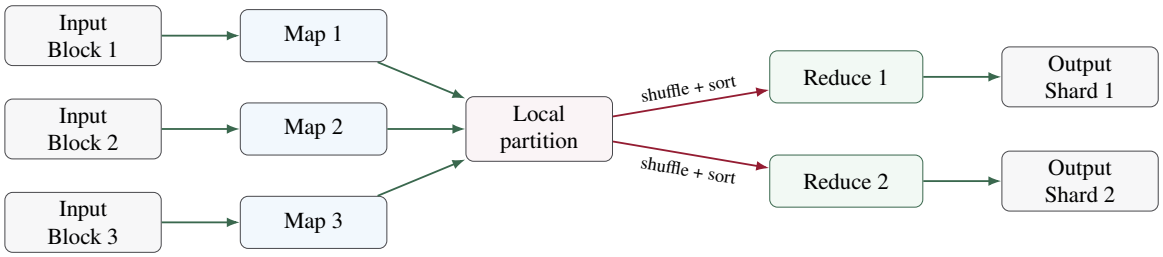


Figure 2.1: MapReduce execution overview: input blocks generate Map tasks; intermediate data is partitioned, shuffled, sorted, and reduced into output shards.

2.2.9 Execution Pipeline

2.2.10 Combiners

Definition – Combiner

A combiner is an optional local pre-aggregation step applied to a Map task's output before shuffle, with the goal of reducing the amount of intermediate data transferred across the network.

For word count, instead of sending many separate $(w, 1)$ pairs across the network, a combiner can aggregate them locally.

A combiner may run zero, one, or multiple times, so correctness must not depend on it running exactly once. Local aggregation must preserve the final result under regrouping; associative, commutative operations such as integer addition are suitable. Averages require carrying a sum and count, rather than averaging partial averages.

2.2.11 Sharded Output

Each Reduce task writes its own output file, producing a distributed collection of shards such as

`part-r-00000, part-r-00001, ...`

rather than one monolithic file. A downstream MapReduce job can then consume these shards as distributed input.

Review Questions

1. How do logical input splits relate to physical storage blocks and Map tasks?
2. How does data locality affect Map-task placement?
3. What problem does a combiner solve, and where does it run?
4. What trade-off governs the number of Reduce tasks?

2.3 Shuffle, Sort, and the Reduce Phase

2.3.1 The Shuffle Phase

Shuffle transfers intermediate Map output to the correct reducers. Each Reduce task must receive all pairs belonging to its assigned keys, regardless of which Map worker produced them.

This creates an all-to-all communication pattern: every mapper may need to communicate with every reducer.

Definition – Shuffle

Shuffle is the distributed transfer of partitioned intermediate data from Map tasks to the Reduce tasks responsible for the corresponding key partitions.

2.3.2 Each Mapper Partitions Its Own Output

Before network transfer, each Map task applies the partition function to emitted keys. Its local output is divided into R regions, one per reducer. Reducers later fetch only the regions assigned to them.

2.3.3 Why Shuffle Is Expensive

With M Map tasks and R Reduce tasks, the communication pattern can involve up to

$$M \times R \tag{2.4}$$

distinct map-to-reduce transfers.

Unlike the Map phase, this traffic generally cannot be made local for every producer–consumer pair.

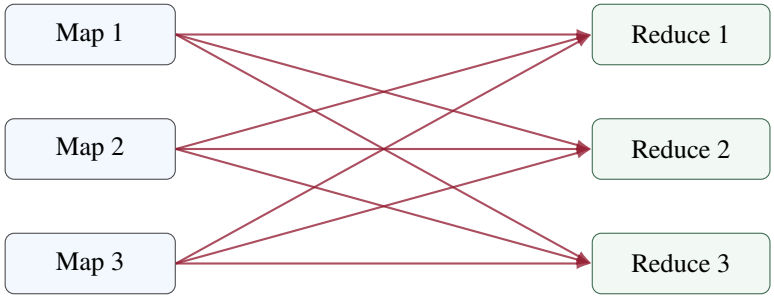


Figure 2.2: The all-to-all structure of shuffle. Every Map task may contribute data to every Reduce task.

2.3.4 Shuffle Network Volume

Let D be the intermediate-data volume after combining and compression. If most partitions are fetched remotely and retries are ignored, shuffle transfers roughly D bytes over the network. Local fetches reduce network traffic; retries can increase it.

Across MR logical map-to-reduce pairs, the average partition size is

$$\text{average bytes per map-to-reduce pair} = \frac{D}{MR}. \tag{2.5}$$

This is not a worst-case bound or a physical-link bandwidth estimate. Skew can concentrate up to D bytes in one pair, and a physical network link may carry many pairs. The key contrast is with the Map phase, where input data can often be read locally.

2.3.5 Sorting by Intermediate Key

After a reducer receives its assigned partitions from all mappers, it sorts the data by intermediate key. Sorting places equal keys next to one another and allows the framework to present each reducer invocation with a complete group of values for one key.

2.3.6 Why Sorting Simplifies Reduce

The user-defined Reduce function need not implement its own distributed grouping logic. The framework guarantees that all values for the current key are presented together.

Conceptually, the reducer processes

$$(k, [v_1, v_2, \ldots, v_n])$$

one key group at a time.

Key Concept

Sorting is not merely an implementation detail: it is the mechanism that turns distributed intermediate records into contiguous key groups with a simple Reduce interface.

2.3.7 Reduce Output

Final Reduce output is written back to GFS/HDFS, unlike temporary Map output. Each Reduce task creates one output shard, which is then stored and replicated like ordinary distributed-file-system data.

2.3.8 Why Shuffle Often Dominates Runtime

Shuffle combines two expensive resources:

- **network I/O:** all-to-all movement of the intermediate dataset;
- **disk I/O:** Map output is written locally, read back for transfer, and often written or merged again on the Reduce side.

For many batch jobs, shuffle rather than user computation is the dominant cost.

2.3.9 Mitigating Shuffle Cost

Two practical techniques reduce this cost:

- **combiners**, which reduce intermediate volume before transfer;
- **compression**, which trades CPU time for reduced network and disk I/O.

At cluster scale, this CPU-for-I/O trade can be favourable because network movement is expensive.

2.3.10 End-to-End Word Count Trace

Consider:

Doc A : “`the cat sat`”

and

Doc B : “`the dog ran`”.

The Map phase produces

$$\text{Map(Doc A)} \rightarrow (\texttt{the}, 1), (\texttt{cat}, 1), (\texttt{sat}, 1),$$

$$\text{Map(Doc B)} \rightarrow (\texttt{the}, 1), (\texttt{dog}, 1), (\texttt{ran}, 1).$$

Assume two reducers and a hash partition in which `the` and `cat` map to Reduce 1, while `sat`, `dog`, and `ran` map to Reduce 2.

After shuffle and sort:

$$R_1 : (\texttt{cat}, [1]), (\texttt{the}, [1, 1]),$$

$$R_2 : (\texttt{dog}, [1]), (\texttt{ran}, [1]), (\texttt{sat}, [1]).$$

Reduce then emits

$$R_1 : (\texttt{cat}, 1), (\texttt{the}, 2)$$

and

$$R_2 : (\texttt{dog}, 1), (\texttt{ran}, 1), (\texttt{sat}, 1).$$

Example – Why identical keys must share a reducer

The two occurrences of `the` originate from different documents and therefore different Map executions, but deterministic partitioning sends both values to the same reducer. Only then can the framework provide the complete group $(\texttt{the}, [1, 1])$ to one Reduce call.

Review Questions

1. Why is shuffle an all-to-all communication pattern?
2. Why does sorting simplify the Reduce implementation?
3. Why is shuffle frequently the dominant cost of a MapReduce job?
4. Why must both occurrences of the same key be routed to the same Reduce task?

2.4 Fault Tolerance and Stragglers

2.4.1 Computation Introduces New Failure Modes

Distributed storage must already tolerate disk and node failures. Active computation introduces additional failure modes:

- a worker crashes while running a task;
- a task remains alive but becomes abnormally slow;
- the job coordinator itself fails.

2.4.2 Worker Failure Detection

Workers periodically send heartbeats containing liveness and progress information. If a worker misses heartbeats beyond a timeout, the Master marks it as failed and must recover affected work.

2.4.3 Failed Map Tasks

If a Map worker fails, the task is rescheduled on a healthy worker. Even a previously completed Map task may need to be rerun if its intermediate output was stored only on the failed worker's local disk.

The determinism requirement guarantees that rerunning the Map task reconstructs equivalent intermediate output.

2.4.4 Failed Reduce Tasks

A failed Reduce task is also rescheduled, but the recovery path differs. If the necessary Map outputs are still available, the replacement reducer can fetch its partitions again. The original Map computations need not all be repeated.

Table 2.2: Recovery distinction between failed Map and Reduce tasks.

Failure	Recovery	Why
Map task / worker	Re-execute Map on another worker	Intermediate Map output lived on the failed worker's local disk and may be lost.
Reduce task / worker	Re-execute Reduce and re-fetch intermediate partitions	Map outputs can still be fetched from surviving Map workers.

2.4.5 Why Re-Execution Is Safe

Pure deterministic functions make a retry functionally indistinguishable from successful first execution. Without this property, the framework could silently obtain different results after a failure.

2.4.6 Idempotency and Atomic Commit

A subtle case appears when multiple attempts of the same task coexist, either due to retries or speculation. The framework must ensure that only one attempt becomes authoritative.

An atomic-commit pattern addresses this: attempts write to temporary locations, and a coordinated atomic rename or commit makes exactly one result official.

Key Concept

Re-execution solves computation failure only if the framework also controls output commitment. Multiple task attempts may run, but only one attempt's output may become the official result.

2.4.7 Master Failure

The MapReduce Master is itself a point of failure. In the original MapReduce design, Master failure is handled by restarting the entire job.

This is a deliberate simplicity trade-off in Google's original implementation, whose Master coordinates one job. Hadoop v1's JobTracker is a long-lived service managing multiple jobs; YARN separates cluster resource management from a per-application ApplicationMaster.

2.4.8 The Straggler Problem

Definition – Straggler

A straggler is a task that remains alive and responsive but executes much more slowly than comparable tasks, thereby delaying overall job completion.

Heartbeats do not classify a straggler as failed because the worker is still responding. If 999 tasks finish quickly and one task takes much longer, the entire job still waits for the slowest task.

2.4.9 Causes of Stragglers

Common causes include:

- hardware degradation;
- CPU, disk, or network contention;
- data skew, especially when one reducer receives an unusually large key group.

2.4.10 Speculative Execution

Definition – Speculative Execution

Speculative execution launches a backup copy of an unusually slow task on another available worker. The first attempt to finish becomes the accepted result, and the slower copy is discarded.

Near the end of a job, the Master identifies tasks running significantly slower than their peers and launches backup attempts if spare cluster capacity exists.

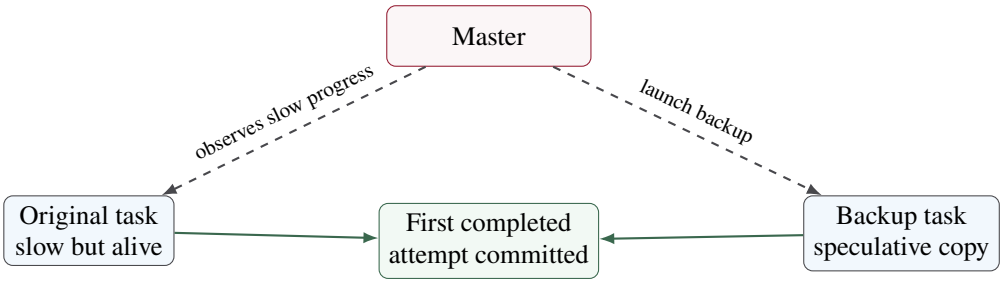


Figure 2.3: Speculative execution: the runtime duplicates a slow task and accepts the first successful attempt.

2.4.11 Numeric Example

Suppose a Map stage has 1000 tasks, enough slots to start them together, and spare capacity for a backup. Normal tasks take approximately 10 seconds, but one straggler takes 200 seconds. Assume the backup starts at $t = 10$ seconds and takes another 10 seconds; ignore scheduling and commit overhead.

Table 2.3: Illustrative effect of speculative execution on completion time.

Scenario	Approximate Map-stage completion time
Without speculation	$\sim$ 200 s, bounded by the straggler
With speculation	$\sim$ 20 s: backup starts at 10 s and runs for 10 s

The example illustrates that a single straggler can dominate job completion time even when almost all other work has finished.

2.4.12 Speculation Trade-off

Speculation is not free:

- **benefit:** it can sharply reduce tail completion time when only a small number of tasks are anomalously slow;
- **cost:** backup attempts consume resources even though one attempt will eventually be discarded.

Speculation is therefore triggered selectively for sufficiently slow tasks when spare capacity is available.

Review Questions

1. Why is re-execution of a failed Map task safe?
2. What is the key difference between recovery from a failed Map task and a failed Reduce task?
3. Why do heartbeat timeouts not detect stragglers?
4. How does speculative execution work, and what is its principal cost?

2.5 Synthesis

2.5.1 Full MapReduce Pipeline



Figure 2.4: Complete MapReduce pipeline from distributed input blocks to distributed output shards.

The execution model can be read as a sequence:

$$\text{input blocks} \rightarrow \text{Map} \rightarrow \text{local partition} \rightarrow \text{shuffle} \rightarrow \text{sort/group} \rightarrow \text{Reduce} \rightarrow \text{output}.$$

Fault tolerance surrounds this pipeline through worker heartbeats, re-execution, output commit control, and speculative execution.

2.5.2 Storage versus Compute Layer

Table 2.4: Comparison between the storage and computation layers.

Aspect	Storage Layer	Compute Layer
Coordinator	Master / NameNode	Master / JobTracker
Worker unit	Chunkserver / DataNode	Map / Reduce task on workers
Failure detection	Heartbeats	Heartbeats
Recovery strategy	Re-replication	Re-execution
New mechanism	–	Speculative execution for stragglers

The two layers share a philosophy: centralized coordination, many workers, heartbeat-based detection, and automatic recovery. Their recovery operations differ because storage reconstructs replicas, while computation reconstructs work.

2.5.3 Strengths of MapReduce

The model's strengths include:

- **simplicity:** programmers define two sequential-looking functions;
- **built-in fault tolerance:** deterministic tasks can be transparently retried;
- **data locality:** Map scheduling exploits block placement from GFS/HDFS;
- **generality:** many batch-processing tasks can be expressed through the model.

2.5.4 Limitations

The same design choices introduce important limitations.

- **Disk-heavy execution:** intermediate data is materialized on disk between Map and Reduce.
- **Rigid two-stage structure:** complex workflows require chaining multiple jobs.
- **Repeated shuffle cost:** each chained job pays another partition/shuffle/sort overhead.
- **High latency:** the model is optimized for batch throughput rather than interactive response.

These limitations motivate in-memory DAG execution models such as Apache Spark.

2.5.5 Iterative Pipelines

Consider a pipeline of ten successive MapReduce jobs in which each job's output feeds the next. Such a workload repeatedly materializes intermediate data and repeatedly pays shuffle and disk-I/O costs.

This motivates retaining reusable intermediate state in memory and scheduling a more general computation graph, avoiding repeated materialization costs in later in-memory execution systems.

2.6 Key Terms

Table 2.5: Key MapReduce terminology.

Term	Meaning
Map task	Processes one input split and emits intermediate $(key, value)$ pairs.
Reduce task	Processes grouped values for its assigned intermediate-key partition.
Partition function	Maps an intermediate key to one Reduce task.
Shuffle	Transfers partitioned intermediate data from Map workers to Reduce workers.
Sort / group	Organizes intermediate records so that all values of one key are contiguous.
Combiner	Performs optional local pre-aggregation before shuffle.
Straggler	Task that remains alive but runs abnormally slowly.
Speculative execution	Runs a backup copy of a slow task on another worker and uses the first completed attempt.
Re-execution	Runs a failed task again on a healthy worker.
Atomic commit	Ensures only one concurrent task attempt becomes the official output.
Data locality	Schedules Map computation near its distributed input block.
Output shard	One output file produced by a Reduce task.

2.7 Chapter Summary

Table 2.6: Compact summary of the main relationships in the MapReduce execution model.

Concept	Main role	Key relationship / trade-off
Map	Transform independent input records	Stateless independence enables arbitrary parallel scheduling.
Reduce	Aggregate all values for one key	Framework-provided grouping hides distributed aggregation mechanics.
Determinism	Make retries reproduce the same logical result	Enables transparent re-execution after failure.
Input block	Natural Map split	Connects distributed storage directly to computation parallelism.
Data locality	Place Maps near their inputs	Minimizes network traffic in the Map phase.
Local Map output	Temporary intermediate storage	Avoids unnecessary DFS replication, but is lost if the worker fails.
Hash partitioning	Route equal keys to one reducer	Deterministic mapping ensures complete key groups.
Number of reducers	Control Reduce parallelism	Too few reducers limit parallelism; too many create coordination and output overhead.
Combiner	Pre-aggregate locally	Reduces shuffle volume when local combination is valid.
Shuffle	Move partitions from Maps to Reducers	All-to-all traffic makes network and disk I/O expensive.
Sort	Group equal keys contiguously	Simplifies the Reduce interface.
Failed Map	Re-execute task	Lost local intermediate output must be regenerated.
Failed Reduce	Re-execute and re-fetch	Existing Map output can generally be fetched again.
Atomic commit	Select one authoritative attempt	Protects output correctness when multiple attempts coexist.
Straggler	Slow but alive task	Not detected as failure because heartbeats continue.
Speculative execution	Duplicate slow work	Reduces tail latency at the cost of extra resources.
Master failure	Job-level coordinator failure	Original design favors restarting the job for simplicity.
MapReduce strength	Simple resilient batch abstraction	Hides distribution and failure handling.
MapReduce limitation	Disk-heavy two-stage model	Iterative pipelines repeatedly pay materialization and shuffle costs.
Spark motivation	General in-memory DAG execution	Addresses rigidity and repeated disk/shuffle overhead.

2.8 Quick Review

1. MapReduce turns simple Map and Reduce functions into distributed computations managed by a runtime.
2. Map processes one input record independently and may emit zero, one, or many intermediate pairs.
3. Reduce receives one intermediate key and the complete collection of values associated with that key.
4. The runtime, not the programmer, handles scheduling, networking, grouping, retry logic, and worker placement.
5. Deterministic, side-effect-free tasks make transparent re-execution safe.
6. Word count Maps words to $(word, 1)$ pairs and Reduce sums the values per word.
7. An inverted index uses the same execution structure with different intermediate values and aggregation.
8. Input blocks from GFS/HDFS naturally become units of Map parallelism.
9. Map tasks are scheduled with data locality whenever possible.
10. The number of Map tasks is commonly much larger than the number of machines, so tasks execute in waves.
11. Intermediate Map output is stored locally rather than immediately replicated into the distributed file system.
12. Hash partitioning uses $\text{hash}(key) \bmod R$ to route identical keys deterministically to the same reducer.
13. The number of Reduce tasks controls the balance between parallelism and coordination/output overhead.
14. A combiner performs optional local pre-aggregation and can reduce shuffle traffic.
15. Shuffle is an all-to-all pattern in which each mapper may send data to every reducer.
16. With M mappers and R reducers, the shuffle can involve up to $M \times R$ logical transfers.
17. Shuffle often dominates runtime because it combines network transfer with repeated disk I/O.
18. Sorting groups identical keys so the Reduce function receives one complete value collection at a time.
19. Final Reduce output is written back to GFS/HDFS as sharded output files.
20. Failed Map tasks are rerun because their local intermediate output may be lost with the failed worker.
21. Failed Reduce tasks can be rerun and normally re-fetch existing Map outputs.
22. Atomic commit prevents multiple concurrent attempts of the same task from producing multiple official outputs.
23. A straggler is slow but alive, so heartbeat-based failure detection does not classify it as failed.
24. Speculative execution launches backup attempts for slow tasks and accepts whichever attempt finishes first.
25. Speculation reduces tail latency but consumes additional cluster resources.
26. MapReduce's major limitations are disk-heavy execution, rigid two-stage jobs, repeated shuffle cost,

and high batch latency.

27. These limitations motivate the in-memory DAG model used by Apache Spark.

2.9 Further Reading

Further reading includes:

- Dean, J., & Ghemawat, S. (2004). *MapReduce: Simplified Data Processing on Large Clusters*. OSDI. A later version appeared in *Communications of the ACM*, 51(1), in 2008.
- Apache Hadoop. *MapReduce Tutorial*, including input splits, shuffle, and output commitment.

CHAPTER 3

Apache Spark and Dataflow Graph Architectures

MapReduce established that large distributed computations can survive routine worker failures through deterministic re-execution, but its execution model remains deliberately rigid and disk-oriented. Spark preserves the recoverability principle while changing the cost model: reusable intermediate state can remain in memory, computation is expressed as a general directed acyclic graph (DAG), and lost data can be reconstructed from recorded lineage instead of replicating every intermediate result.

This chapter develops the model through seven linked ideas: the limitations of chained MapReduce jobs, Resilient Distributed Datasets (RDDs), transformations and actions, lazy evaluation and execution DAGs, narrow versus wide dependencies, lineage-based fault tolerance, and the role of the DAG scheduler in turning a logical plan into physical stages and tasks.

Key Concept

Spark generalizes MapReduce from a fixed two-stage, disk-heavy job into an arbitrary DAG of operations. Its central performance mechanism is reuse of in-memory data; its central recovery mechanism is lineage-based recomputation.

3.1 Beyond Two-Stage Pipelines

3.1.1 The MapReduce Ceiling Revisited

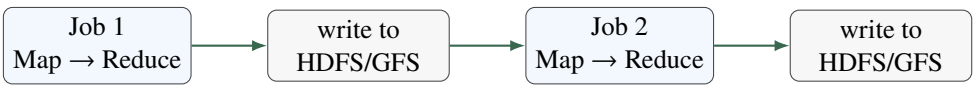
Chapter 1 established the storage substrate, while Chapter 2 showed how MapReduce executes reliable batch computations on top of it. The same MapReduce design choices that make the model simple also create three limitations for more complex workloads:

- **Disk-heavy execution:** intermediate results are materialized between jobs, so repeated pipelines repeatedly pay storage I/O.
- **Rigid two-stage structure:** each job has the fixed Map → Shuffle/Sort → Reduce shape.
- **High latency for iterative or interactive work:** each additional job incurs scheduling, materialization, and data-transfer overhead.

Spark was introduced through the Resilient Distributed Datasets paper by Zaharia et al. (NSDI 2012) specifically to improve data reuse across multiple operations and iterations.

3.1.2 Why Chaining MapReduce Jobs Is Expensive

A real analytics workflow may need filtering, transformations, joins, aggregation, and further filtering. Under the classic MapReduce model, a multi-step computation is represented as a sequence of independent jobs:



Each job is scheduled and torn down independently. Its complete output is written to the distributed file system and must be read again by the next job. In a ten-stage iterative pipeline, the motivating ten-stage example therefore pays ten full disk round-trips and ten shuffle phases even though only the initial input and final output truly require durable storage.

3.1.3 Arbitrary Computation Graphs

The natural structure of a multi-operation program is not a fixed two-stage pipeline but a graph of data dependencies. A **Directed Acyclic Graph (DAG)** represents this structure:

- nodes represent operations or the datasets those operations produce;
- directed edges represent dependencies between producers and consumers;
- acyclicity means execution flows forward through the dependency graph rather than forming a cyclic dependency.

A MapReduce job can be viewed as a very restricted DAG. Spark removes that restriction and allows the programmer to express arbitrary chains and branches of operations.

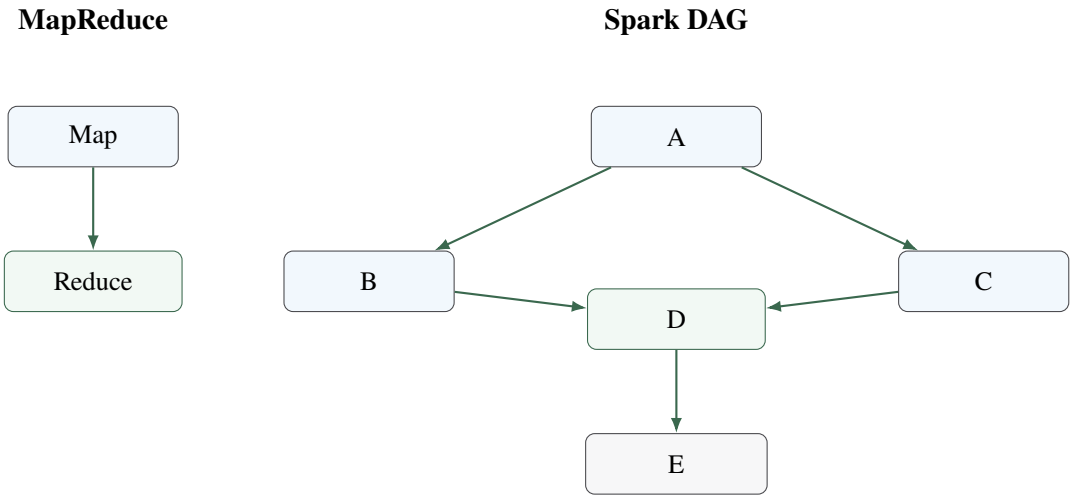


Figure 3.1: A fixed MapReduce pipeline compared with an arbitrary Spark computation DAG. Spark can represent branches, joins, and deeper operation chains rather than exactly one Map and one Reduce phase.

3.1.4 What "Memory-Centric" Means

Spark's second major shift is that intermediate results can remain in RAM across operations rather than being written to durable storage after every step. Data is written to durable storage when explicitly requested or when memory pressure requires another storage strategy.

The memory hierarchy discussed in Chapter 1 provides the performance intuition: RAM access is roughly two orders of magnitude faster than disk access for comparable data volumes. Reusing an in-memory working set is therefore especially valuable when an algorithm repeatedly scans the same dataset.

3.1.5 Motivating Workloads

Two workload classes motivate the design particularly strongly.

Iterative algorithms. Graph algorithms and machine-learning training loops repeatedly process the same data. In MapReduce, every iteration can require another full job, another read, and another materialization. Spark can retain the working dataset across iterations.

Interactive and exploratory analysis. An analyst may issue multiple successive queries over the same

dataset. Once that dataset is cached in cluster memory, later queries can avoid repeated disk I/O and complete much more quickly than a fresh batch job that reloads the same input every time.

3.1.6 MapReduce and Spark Side by Side

Table 3.1: Positioning Spark relative to the MapReduce execution model.

Aspect	MapReduce	Spark
Pipeline shape	Fixed two-stage Map → Reduce model	Arbitrary DAG of operations
Intermediate data	Materialized to disk between chained jobs	Held in memory when possible
Reuse across iterations	Re-read or recomputed through separate jobs	Cached in memory across iterations
Fault tolerance	Re-execution of failed tasks	Lineage-based recomputation of lost partitions
Best fit in this comparison	Large, one-pass batch jobs	Iterative, interactive, and multi-step pipelines

Review Questions

1. What three limitations of MapReduce does Spark directly address?
2. Why is chaining multiple MapReduce jobs expensive even when each individual job is efficient?
3. How does a DAG generalize the fixed two-stage MapReduce pipeline?
4. Why do iterative algorithms benefit disproportionately from memory-centric execution?

3.2 Resilient Distributed Datasets

3.2.1 RDD as the Core Abstraction

Definition – Resilient Distributed Dataset (RDD)

An RDD is an immutable collection of elements partitioned across the machines of a cluster. It is resilient because lost partitions can be reconstructed automatically from the recorded derivation history rather than requiring every intermediate dataset to be physically replicated.

An RDD behaves conceptually like a large distributed collection. Within the RDD programming model used here, every dataset manipulated by the Spark program is represented through this abstraction. Operations derive new RDDs from existing ones; an existing RDD is never modified in place. This intentionally hides much of the cluster-level complexity from the programmer.

3.2.2 Why Immutability Matters

Immutability is foundational rather than cosmetic. Because an RDD never changes after creation, the transformation that created it remains a complete and permanent description of how its contents can be reconstructed. This property supports lineage-based fault tolerance and also avoids many concurrency problems associated with shared mutable state distributed across machines.

Key Concept

For an immutable RDD, "how this dataset was created" is stable metadata. Spark can therefore retain the recipe for an intermediate dataset instead of maintaining a replicated physical copy of that dataset at all times.

3.2.3 Partitions: The Unit of Parallelism

RDDs are divided into **partitions**. Each partition contains a subset of the RDD's data and can be processed independently by a task. Consequently, the partition count bounds the maximum degree of parallelism available for operations on that RDD.

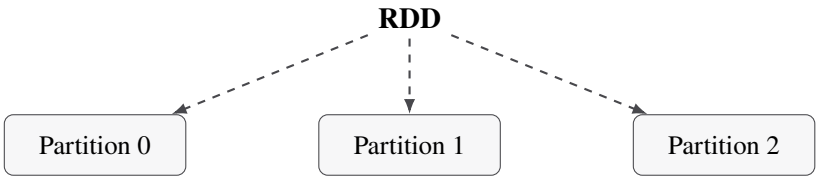


Figure 3.2: An RDD is logically one dataset but is physically processed as independent partitions distributed across the cluster.

3.2.4 RDD Partitions versus Storage Blocks

RDD partitions are related to, but distinct from, the GFS/HDFS blocks studied in Chapter 1. If an RDD is created from a distributed file, its partitions commonly correspond to underlying file blocks. However, an RDD can also be repartitioned explicitly or created from a non-file source such as an in-memory collection. Partitioning is therefore a computational concept at the RDD layer rather than a property tied permanently to one storage layout.

3.2.5 RDDs and Underlying Storage

An RDD is not a storage system. It is a computation abstraction layered on top of storage. Its initial data may come from HDFS, GFS, another backend, or a collection in the driver program that is distributed to the cluster.

Two primary RDD-creation patterns are:

```
# From distributed storage
rdd = sc.textFile("hdfs://path/to/data")

# From an existing in-memory collection
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

With `textFile`, partitions are derived from the underlying file organization. With `parallelize`, the driver's local collection is divided into partitions and distributed.

3.2.6 Logical Handle versus Materialized Data

An RDD should be understood as a *logical handle*: it describes a dataset, its partitioning, and the computation from which it is derived. The corresponding bytes do not necessarily exist in memory at the moment the RDD variable is created. Materialization depends on lazy evaluation and on whether the program chooses to cache the RDD.

3.2.7 The "Resilient" Property

Unlike GFS/HDFS, resilience for an intermediate RDD does not normally come from keeping several copies of every partition. Spark records the RDD's **lineage**: which parent RDD or RDDs it depends on and which transformation produced it. If a partition disappears because a worker fails, Spark can recompute that partition from its lineage.

3.2.8 RDD Interface Overview

The RDD interface exposes three families of operations:

- **Transformations**, which derive new RDDs from existing RDDs;
- **Actions**, which trigger computation and return a result to the driver or write output;
- **Persistence controls**, which request that computed partitions be retained for reuse.

This generalizes the MapReduce input model. An RDD can begin from storage blocks just as MapReduce begins from distributed input, but it can then pass through an arbitrary sequence of transformations instead of being restricted to one Map phase and one Reduce phase.

3.2.9 RDDs versus Regular Distributed Arrays

An RDD resembles a distributed array or list, but three properties distinguish it in this model:

- there are no random in-place updates to individual elements;
- the system tracks lineage describing how the RDD was produced;
- materialization is lazy, so the contents may not exist physically until an action requires them.

3.2.10 Caching and Persistence

Caching is the performance mechanism that turns an RDD from a recomputable logical description into a reusable in-memory working set.

```
rdd = sc.textFile("hdfs://path/to/data")
rdd.cache()       # mark the RDD for in-memory persistence
```

Without caching, an RDD may be recomputed from lineage when it is reused. With caching, computed partitions are retained and can be consumed directly by later operations, which is especially useful for iterative and interactive workloads.

Table 3.2: Common RDD persistence levels.

Storage level	Behaviour
Memory only	Fastest; partitions that do not fit are discarded and recomputed when needed.
Memory and disk	Keeps partitions in memory when possible and spills excess partitions to local disk.
Disk only	Avoids memory pressure but accesses persisted partitions at disk speed.

The appropriate persistence level depends on the size of the working dataset relative to available cluster memory.

Review Questions

1. What does it mean for an RDD to be immutable, and why does this matter for fault tolerance?
2. How do RDD partitions relate to, but differ from, the storage blocks studied in Chapter 1?
3. Why is RDD fault tolerance fundamentally different from GFS/HDFS replication?
4. What is the purpose of explicitly caching an RDD?

3.3 Transformations and Actions

3.3.1 Two Categories of RDD Operations

Every RDD operation considered here belongs to one of two categories.

Definition – Transformation

A transformation takes one or more RDDs and returns a new RDD describing a derived dataset. The transformation is recorded lazily rather than executed immediately.

Definition – Action

An action triggers the computation needed to produce a result outside the RDD abstraction, either by returning data to the driver program or by writing output to storage.

This division between *describing* computation and *triggering* computation is the foundation of Spark's lazy evaluation model.

3.3.2 Common Transformations

Table 3.3: Common RDD transformations.

Transformation	Effect
`map(f)`	Apply `f` to each element.
`filter(f)`	Keep elements for which `f` returns true.
`flatMap(f)`	Apply `f` and flatten the produced collections.
`distinct()`	Remove duplicate elements.
`union(other)`	Combine with another RDD.
`groupByKey()`	Group values by key.
`reduceByKey(f)`	Aggregate values per key using `f`.
`join(other)`	Join two key-value RDDs by key.

These transformations are deterministic functions over data, analogous in spirit to the deterministic Map operation studied in Chapter 2. Because RDDs are immutable, a transformation does not alter its input:

```
rdd1 = sc.textFile("data.txt")
rdd2 = rdd1.filter(lambda line: "error" in line)
# rdd1 is unchanged; rdd2 is a new RDD description
```

A chain of transformations therefore constructs a chain of new RDD descriptions.

Table 3.4: Common RDD actions.

Action	Effect
`count()`	Return the number of elements.
`collect()`	Return all elements to the driver.
`take(n)`	Return the first `n` elements.
`reduce(f)`	Aggregate all elements using `f`.
`saveAsTextFile(path)`	Write the RDD contents to storage.

3.3.3 Common Actions

Calling a transformation records another step in the plan. Calling an action forces Spark to execute the transformations required to obtain that action's result.

3.3.4 Worked Example: Word Count with RDDs

The MapReduce word-count example can be rewritten as a chain of RDD transformations:

```
lines  = sc.textFile("hdfs://documents/")
words  = lines.flatMap(lambda line: line.split())
pairs  = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

At this point, `lines`, `words`, `pairs`, and `counts` are logical RDD descriptions. No HDFS input necessarily has been read yet, and no word counting has happened.

Execution starts when an action is called:

```
result = counts.collect()   # ACTION: triggers the entire chain
```

This one action causes Spark to read the input, run `flatMap`, run `map`, perform `reduceByKey` including any required shuffle, and return the final result to the driver.

Key Concept

Transformations are not "free." Their execution cost is deferred until an action. Laziness changes *when* work happens and gives Spark visibility over the entire transformation chain before deciding how to execute it.

3.3.5 Why the Split Creates an Optimization Opportunity

If every transformation executed eagerly when called, the system would optimize operations one at a time. By delaying execution, Spark can inspect the complete chain and reason about the pipeline as a whole before physical work begins.

Review Questions

1. What is the fundamental difference between a transformation and an action?
2. In the word-count example, why has no actual computation happened until `collect()` is called?
3. Why does separating the description of computation from its execution create an optimization opportunity?
4. Give one transformation and one action that do not appear in the word-count chain.

Table 3.5: MapReduce's eager job model and Spark's lazy RDD model.

Aspect	MapReduce	Spark RDDs
When work happens	As each submitted job executes	Deferred until an action is called
Pipeline visibility	One job at a time	Entire transformation chain visible before execution
Multiple steps	Separate jobs with separate materialization	One logical chain that can be optimized and executed together

3.4 Lazy Evaluation and the Execution DAG

3.4.1 What Laziness Means

Lazy evaluation means that transformations update Spark's internal computation plan rather than immediately reading, transforming, or writing partitions. An action is the event that forces execution of the required part of that plan.

This delay has two direct benefits:

- Spark can optimize across the entire sequence of transformations rather than executing each step in isolation.
- Spark can sometimes avoid work that is irrelevant to the requested result, for example when an action such as `take(5)` requires only part of the dataset.

3.4.2 Building the Logical Execution Plan

Each derived RDD stores references to its parent RDD or RDDs and to the transformation that produced it. These references form a graph. In the simple word-count case the graph is a chain; operations such as `join` or `union` can create nodes with multiple incoming dependencies.

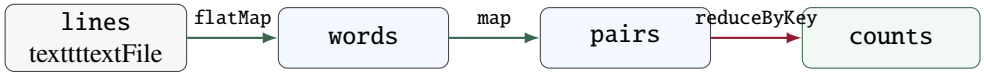


Figure 3.3: Logical RDD DAG for the word-count chain. The nodes are RDDs and the edges are transformations. The graph exists before `collect()` triggers physical execution.

3.4.3 Optimization Opportunities Enabled by Laziness

With the full DAG available before execution, Spark can exploit several optimization opportunities:

- **Operator fusion / pipelining:** compatible operations are applied in one pass without materializing complete intermediate datasets.
- **Avoiding unneeded computation:** parts of the DAG that do not contribute to the requested action can sometimes be skipped.
- **Choosing a physical execution structure:** the dependency graph determines where the logical chain must be broken into separate stages.

3.4.4 Operator Fusion within a Stage

Consider a narrow chain such as:

```
words.map(f).filter(g).map(h)
```

Spark can apply `f`, then `g`, then `h` to each element in one per-partition pass rather than fully materializing the output of each operation. This reduces extra scans and unnecessary intermediate memory usage.

The contrast with chained MapReduce jobs is fundamental: a MapReduce job boundary requires complete output materialization before the next job can consume it, while a compatible Spark transformation chain may never materialize those intermediates as standalone datasets.

3.4.5 From Logical DAG to Physical Jobs

When an action is called, the DAG scheduler converts the logical RDD graph into executable work:

1. inspect the dependencies required by the action;
2. divide the graph into **stages**;
3. create tasks within each stage, conventionally one task per partition;
4. schedule those tasks across the cluster.

Definition – Stage

A stage is a group of operations that can execute together without requiring a shuffle between them. Operations inside a stage can be pipelined; a new stage begins where the dependency graph requires cross-partition redistribution.

Operations such as `map` and `filter` can usually remain within a stage because they need only the corresponding input partition. Operations such as `reduceByKey` and `join` regroup data across partitions and therefore introduce a shuffle boundary.

3.4.6 Worked Example: Building and Triggering a Multi-Step DAG

The word-count example can be extended by filtering rare words after aggregation:

```
lines    = sc.textFile("hdfs://documents/")
words    = lines.flatMap(lambda line: line.split())
pairs    = words.map(lambda word: (word, 1))
counts   = pairs.reduceByKey(lambda a, b: a + b)
frequent = counts.filter(lambda kv: kv[1] > 100)
result   = frequent.collect()   # triggers the chain
```

The transformation chain is built first and executed only when `collect()` is called.

Table 3.6: Fixed MapReduce jobs versus a Spark execution DAG.

Aspect	MapReduce	Spark DAG
Pipeline depth	Exactly one Map phase followed by one Reduce phase per job	Arbitrary number of chained operations
Intermediate storage	Materialized between jobs	Pipelined in memory where possible
Multi-step pipelines	Multiple separately scheduled jobs	One DAG optimized as a whole
Shuffle position	Built into the Map-to-Reduce transition of every job	Only where dependency structure requires redistribution

Review Questions

1. What does it mean for Spark transformations to be lazy, and what triggers actual execution?

2. What is operator fusion or pipelining, and why does it reduce intermediate materialization?
3. What is a stage, and what causes a new stage to begin?
4. How does DAG-based execution differ from MapReduce's fixed two-stage model?

3.5 Narrow versus Wide Dependencies

3.5.1 Dependencies between RDDs

A dependency describes how a child RDD's partitions depend on partitions of its parent RDD or RDDs. This is the precise version of the transformation history introduced earlier. The dependency type determines both execution cost and stage structure.

Definition – Narrow Dependency

A dependency is narrow when each child partition depends on a small, fixed number of parent partitions, typically exactly one. Transformations such as `map`, `filter`, and `flatMap` are narrow in this model.

Because the required data remains local to the corresponding partition, narrow dependencies do not require cross-partition network transfer.

Definition – Wide Dependency

A dependency is wide when one child partition may depend on data from multiple, potentially all, parent partitions. Operations such as `groupByKey`, `reduceByKey`, and `join` are wide because records must be redistributed by key.

A wide dependency is structurally the same data-movement problem as MapReduce shuffle: records are partitioned, transferred across the network, and regrouped by key before downstream processing can continue.

3.5.2 Dependency Diagrams

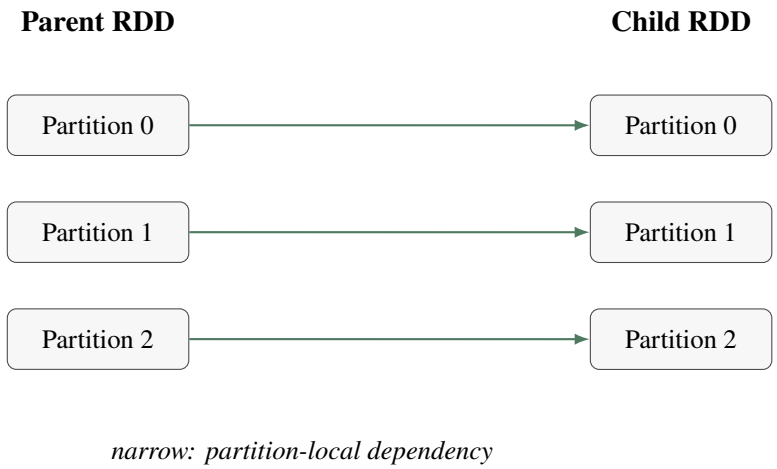


Figure 3.4: Narrow dependency: each child partition can be produced from a corresponding parent partition without network redistribution.

3.5.3 Why Dependency Type Determines Stage Boundaries

The narrow/wide distinction explains the stage concept precisely:

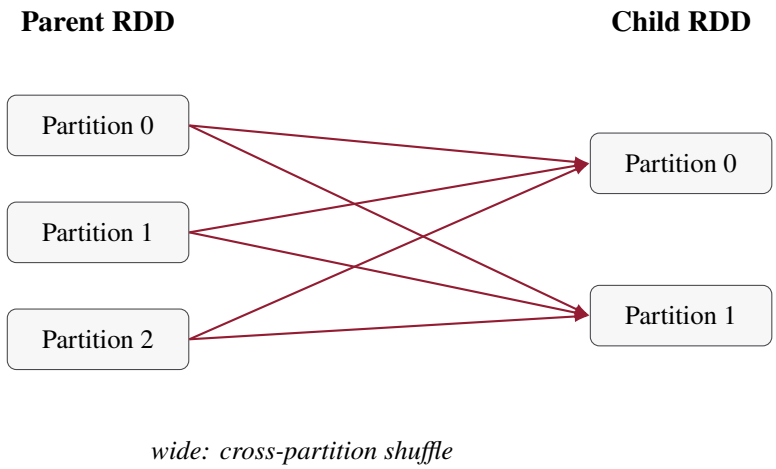


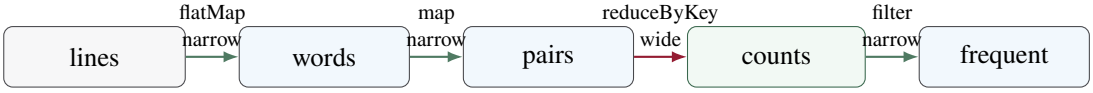
Figure 3.5: Wide dependency: each child partition may require records originating from several parent partitions, forcing network redistribution.

- a chain of narrow transformations can be pipelined on each partition inside one stage;
- a wide transformation requires a shuffle, so the next stage cannot proceed until the required redistributed data is available.

Therefore, a stage is a maximal run of narrow-dependency transformations. Wide dependencies form the boundaries between stages.

3.5.4 Worked DAG: Identifying Narrow and Wide Operations

For the extended word-count chain:



`flatMap`, `map`, and `filter` are narrow. `reduceByKey` is wide and therefore introduces the shuffle and stage boundary. The five-step logical chain compiles to two physical stages.

3.5.5 Performance Implications of Wide Dependencies

Wide dependencies inherit the same cost characteristics as MapReduce shuffle:

- network bandwidth proportional to the volume of redistributed data;
- possible disk I/O for shuffle spill and later reads;
- a synchronization point because the next stage depends on shuffle output from multiple partitions.

Spark does not eliminate shuffle cost. It confines shuffle to the points in the DAG where global redistribution is required.

3.5.6 Minimizing Wide Dependencies

Two practical guidelines follow:

- prefer `reduceByKey` over `groupByKey` followed by a separate aggregation when local partial aggregation is possible; this is analogous to a MapReduce combiner;
- push narrow operations such as filtering before a wide operation when possible, reducing the volume

of data that must cross the network.

Review Questions

1. What distinguishes a narrow dependency from a wide dependency?
2. Why do narrow dependencies permit pipelining while wide dependencies do not?
3. Why does the worked word-count chain compile to exactly two physical stages?
4. How is a wide dependency related to the MapReduce shuffle phase?

3.6 Lineage Tracking and Fault Tolerance

3.6.1 The Fault-Tolerance Problem

At cluster scale, the failure assumption from the previous chapters remains unchanged: machines holding RDD partitions can fail during computation. The recovery strategies differ by layer:

- GFS/HDFS protect stored data through physical replication;
- MapReduce re-executes failed tasks from available input;
- Spark recomputes specifically lost RDD partitions by following lineage.

3.6.2 What Lineage Records

Definition – Lineage

Lineage is the recorded chain of transformations that describes how an RDD's partitions were derived from their parent RDD or RDDs. The record includes the parent relationship and the transformation, including the relevant partitioning relationship, used to produce the child.

The same DAG therefore serves two roles: it is an execution plan before and during normal computation, and it is a recovery plan after data loss.

3.6.3 Lineage Stores a Recipe, Not Data Copies

This is the central difference from distributed-file-system replication. GFS/HDFS retain multiple physical copies of data. Spark can retain a compact derivation description and recreate missing data when required. The lineage metadata is small relative to the intermediate dataset it describes: it stores a compact derivation description rather than another physical copy of the full dataset.

3.6.4 How a Lost Partition Is Recomputed

If a partition disappears, Spark can rebuild exactly the required partition:

1. identify the parent partition or partitions recorded in its lineage;
2. reapply the transformation that produced the missing partition;
3. if a required parent partition is also missing, continue recursively backward through the lineage until available data is reached.

The important recovery granularity is the partition: unrelated partitions do not need to be recomputed.

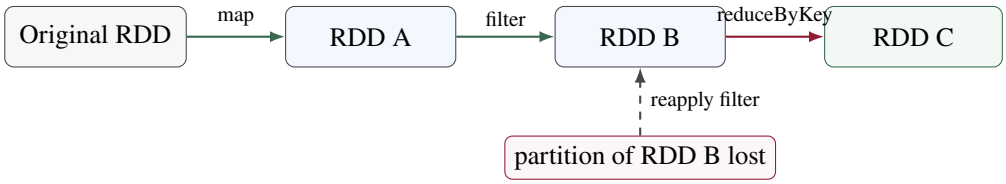


Figure 3.6: Lineage-based recovery. If a partition of RDD B is lost and the required RDD A partition still exists, Spark re-applies the recorded `filter` only for the missing partition.

Table 3.7: Recovery granularity in MapReduce and Spark.

Aspect	MapReduce	Spark RDDs
What is recomputed	Failed Map or Reduce task	Specifically lost RDD partition or partitions
Recovery granularity	Whole task	Individual partition following lineage
Multi-step pipelines	Failure handled within each separate job	One lineage chain can span the complete DAG

3.6.5 Comparison with MapReduce Re-Execution

Spark therefore generalizes MapReduce's "recompute the work" philosophy from a two-stage job to arbitrarily deep RDD transformation graphs.

3.6.6 Comparison with GFS/HDFS Replication

Table 3.8: Replication-based and lineage-based fault tolerance.

Aspect	GFS/HDFS	Spark RDDs
Fault-tolerance mechanism	Physical replication of data	Lineage-based recomputation
Storage overhead	Multiple physical copies; a typical comparison uses 3× replication	Minimal lineage metadata relative to the data itself
Recovery cost	Read an existing replica	Recompute the missing partition; cost can grow when lineage is long

This is a trade-off rather than an unconditional improvement. Replication spends storage continuously to make recovery fast. Lineage minimizes intermediate storage overhead but can make recovery computationally expensive when the derivation chain is deep.

3.6.7 Why Replicating Every Intermediate RDD Is Expensive

A Spark program may generate many intermediate RDDs. Replicating each one would require storing and maintaining multiple physical copies of every intermediate dataset. A ten-transformation pipeline makes the contrast concrete: a replication-based strategy would have to retain ten separate replicated intermediate datasets, while lineage stores a small amount of metadata per RDD.

3.6.8 Long Lineage and Checkpointing

Long lineage chains have a clear weakness. Losing a partition near the end of a deep DAG may require recomputing many preceding steps. Iterative algorithms can make this especially costly because each iteration lengthens the derivation history.

Definition – Checkpointing

Checkpointing writes an RDD's current materialized data to durable replicated storage such as HDFS. The checkpoint becomes a new recovery base so future recomputation does not need to traverse the complete earlier lineage chain.

Checkpointing therefore trades an explicit storage and replication cost now for a bounded recomputation cost later.

3.6.9 Recovery Reuses the DAG Scheduler

Fault recovery is integrated into ordinary scheduling. When a task or worker failure causes partitions to disappear, the DAG scheduler identifies the affected partitions and schedules replacement tasks using the same dependency and stage structure already used for normal execution. Spark does not require a completely separate recovery architecture.

Review Questions

1. How does lineage-based fault tolerance differ fundamentally from GFS/HDFS replication?
2. Walk through how Spark recovers one lost RDD partition.
3. Why can long lineage chains become a liability, and how does checkpointing address this?
4. In what sense does Spark reuse its normal DAG machinery for fault recovery?

3.7 Synthesis: From Transformations to Cluster Tasks

3.7.1 The Full Spark Execution Model

The mechanisms introduced throughout the chapter form one execution pipeline:

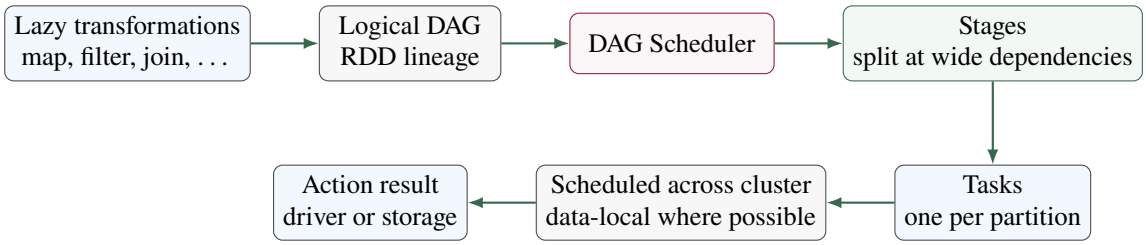


Figure 3.7: Spark execution model from lazy transformations to physical tasks. Narrow chains are pipelined within stages, while wide dependencies introduce shuffle boundaries between stages.

The action is the trigger. The logical lineage graph already exists before that action; execution converts the required part of the graph into physical stages and tasks.

3.7.2 Role of the DAG Scheduler

The DAG scheduler is Spark's coordinating component, conceptually analogous to the computation coordinator in MapReduce. Once an action is called, it:

1. walks the required lineage backward from the action's target RDD;
2. groups chains of narrow dependencies into pipelined stages;
3. orders those stages so that each stage's wide-dependency shuffle inputs are available;
4. creates one task per partition inside each stage and submits those tasks for execution.

3.7.3 Data Locality Reappears

Spark preserves the data-local scheduling principle from the earlier chapters. For an initial stage reading from HDFS/GFS, tasks are preferably placed on machines holding the corresponding storage blocks. For later stages, the scheduler prefers machines where required cached parent partitions already reside in memory whenever possible.

Key Concept

The same "move computation to data" principle appears at three layers of the course: distributed storage exposes block placement, MapReduce schedules Map tasks near those blocks, and Spark additionally tries to schedule later work near already materialized or cached RDD partitions.

3.7.4 Strengths of the Spark/RDD Model

Relative to the MapReduce model developed in Chapter 2, four strengths stand out:

- **Generality:** arbitrary DAGs replace a fixed two-stage shape.
- **Performance for iterative and interactive workloads:** in-memory caching avoids repeated disk round-trips.
- **Efficient fault tolerance:** lineage avoids full physical replication of every intermediate dataset.
- **Optimizable execution:** visibility of the full logical pipeline enables pipelining and avoidance of unnecessary materialization.

3.7.5 Remaining Limitations

The original RDD model still has limitations:

- the RDD API is relatively low-level for complex relational-style queries;
- RDDs do not inherently expose schema and column semantics that could enable richer structured-data optimization;
- the original model is batch-oriented and does not by itself represent continuous, unbounded streaming data.

3.7.6 Beyond the Original RDD Model

Several modern systems extend the broader design space:

- **Apache Flink:** continuous processing of unbounded streams;
- **Ray:** dynamic distributed task graphs for AI workloads;
- **Delta Lake:** a unified lakehouse storage abstraction connecting data-lake storage with structured tables.

These examples do not replace the RDD model developed here; they illustrate how modern infrastructure extends the design space beyond the original batch-oriented abstraction.

3.8 Key Terms

3.9 Chapter Summary

Table 3.9: Key Spark terminology.

Term	Meaning
RDD	Immutable, partitioned, distributed dataset with tracked lineage.
Transformation	Lazy operation that produces a new RDD, such as `map` or `filter`.
Action	Operation that triggers execution and returns or writes a result.
Narrow dependency	Child partition depends on a small fixed number of parent partitions; permits pipelining.
Wide dependency	Child partition depends on multiple parent partitions; requires a shuffle.
Lineage	Recorded transformation history that enables recomputation.
Stage	Maximal pipelinable group of operations between shuffle boundaries.
Checkpoint	Durable materialization used to bound the cost of recomputing a long lineage chain.
DAG scheduler	Component that derives stages and tasks from the logical lineage graph after an action triggers execution.

Table 3.10: Compact summary of the Spark/RDD execution model.

Concept	Main role	Key relationship / trade-off
RDD	Distributed immutable dataset abstraction	Partitioned for parallelism and described by lineage.
Transformation	Build a derived RDD lazily	Extends the DAG without immediately executing data processing.
Action	Trigger computation	Converts the required logical plan into physical execution.
Caching	Reuse computed partitions	Improves iterative and interactive performance but consumes memory or persistence capacity.
Logical DAG	Record dependencies	Gives Spark whole-pipeline visibility before execution.
Narrow dependency	Partition-local relationship	Enables pipelining inside one stage.
Wide dependency	Cross-partition relationship	Forces shuffle and creates a stage boundary.
Stage	Pipelined physical segment	Maximal sequence of operations not separated by a shuffle.
Task	Per-partition unit of execution	Stage parallelism is realized through tasks distributed across workers.
Lineage	Recovery recipe	Avoids replicating every intermediate RDD but can make recovery expensive when chains are long.
Checkpointing	Bound lineage depth	Pays durable-storage cost to reduce future recomputation cost.
DAG scheduler	Coordinate execution	Converts dependencies into ordered stages and tasks and reuses the same structure for recovery.

Concept	Main role	Key relationship / trade-off
Data locality	Place work near needed data	Extends the locality principle from DFS blocks to cached RDD partitions.

3.10 Quick Review

1. Spark was designed to address the disk-heavy, rigid, high-latency limitations of chained MapReduce jobs.
2. A Spark computation is represented as an arbitrary DAG rather than a fixed two-stage pipeline.
3. Memory-centric execution is especially useful when the same working dataset is reused across iterations or interactive queries.
4. An RDD is immutable, partitioned, distributed, and lineage-tracked.
5. RDD partitions are computation-level units and are related to, but not identical with, distributed-storage blocks.
6. `textFile` can build an RDD from distributed storage; `parallelize` can distribute a driver-side collection.
7. An RDD can exist as a logical description before its contents are materialized.
8. Caching retains computed partitions for fast reuse; persistence can use memory, disk, or a mixture.
9. Transformations create new RDD descriptions and are lazy.
10. Actions trigger the computation needed to return or write a result.
11. Lazy evaluation gives Spark visibility over the complete transformation chain before execution.
12. Compatible narrow transformations can be pipelined in one pass without full intermediate materialization.
13. A logical DAG is divided into stages when execution is triggered.
14. Narrow dependencies keep computation partition-local and therefore permit pipelining.
15. Wide dependencies require cross-partition redistribution and therefore cause shuffle.
16. Stage boundaries coincide with wide-dependency boundaries.
17. Spark does not eliminate shuffle; it limits shuffle to the points where redistribution is required by the DAG.
18. `reduceByKey` can reduce shuffle volume through local partial aggregation compared with grouping all values first.
19. Lineage records how RDD partitions were derived from parent RDDs.
20. If a partition is lost, Spark recomputes that partition by walking backward through lineage to available data.
21. Lineage avoids the storage cost of replicating every intermediate RDD, but recovery cost grows with lineage depth.

22. Checkpointing writes an RDD to durable replicated storage and resets the effective recovery base.
23. The DAG scheduler groups narrow chains into stages and creates one task per partition inside each stage.
24. Spark's scheduler preserves data locality both for initial DFS input and for later cached partitions when possible.
25. The original RDD model remains relatively low-level for structured relational queries and does not by itself model unbounded streaming data.

3.11 Further Reading

Further reading includes:

- Zaharia, M., et al. (2012). *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*. NSDI.
- Course reference materials on Spark execution internals.

CHAPTER 4

The Geometry of High-Dimensional Spaces

Module 1 developed the infrastructure that stores and processes large datasets. Module 2 changes the question: once data is represented numerically, what geometry do algorithms actually encounter? Text, images, and graphs routinely become vectors with hundreds, thousands, or millions of coordinates, and familiar geometric intuition from two or three dimensions can become actively misleading.

This chapter develops the mathematical chain behind the *curse of dimensionality*. The central phenomenon is concentration: volume moves toward boundaries, surface area concentrates near equators, random directions become nearly orthogonal, and pairwise distances become difficult to distinguish relative to their scale.

Key Concept

High-dimensional geometry is governed by concentration. As the dimension grows, "typical" points occupy increasingly thin geometric regions and many distance-based distinctions that are obvious in low dimensions become weak.

4.1 Why Low-Dimensional Intuition Fails

4.1.1 The Problem with Geometric Intuition

Human geometric intuition is built from direct experience in two and three dimensions. In that regime, a disk appears mostly filled by its interior, random directions are not usually close to perpendicular, and nearest-neighbor relationships are visibly meaningful. None of these intuitions extrapolates reliably to large dimension d.

The goal is therefore not to say vaguely that high-dimensional spaces are "strange", but to derive specific quantitative statements showing how volume, angle, and distance behave as $d \to \infty$.

4.1.2 Vectors, Norms, and the Two Reference Bodies

We work in the real vector space

$$\mathbb{R}^d = \{\mathbf{x} = (x_1, \ldots, x_d) : x_i \in \mathbb{R}\}.$$

A point and the vector from the origin to that point are used interchangeably. The Euclidean norm is

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2}.$$

Two geometric objects will be used repeatedly.

Definition – Unit Hypercube

The d-dimensional unit hypercube is

$$C_d = [0,1]^d = \{\mathbf{x} \in \mathbb{R}^d : 0 \le x_i \le 1 \text{ for all } i\}.$$

Definition – Unit Ball

The d-dimensional unit ball is

$$B_d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\|_2 \le 1\} = \left\{\mathbf{x} \in \mathbb{R}^d : \sum_{i=1}^{d} x_i^2 \le 1\right\}.$$

The chapter tracks two quantities as d grows:

- **volume:** where the mass of a cube or ball is located;
- **distance:** how much pairwise distances vary relative to their typical scale.

4.1.3 Why Big Data Naturally Produces Large d

High dimension is not an artificial mathematical regime. Typical examples include:

- text represented over vocabularies containing tens of thousands or millions of terms;
- images represented by raw pixels or learned feature vectors with hundreds or thousands of coordinates;
- graph and node embeddings represented in spaces with dozens or hundreds of dimensions.

The geometric effects derived below therefore directly affect similarity search, clustering, classification, and the representation methods developed later in this module.

Remark – Logical chain of the chapter

The results are cumulative: boundary concentration in cubes and balls leads to concentration near equators, which explains near-orthogonality; distance concentration then makes the loss of distance-based discrimination precise.

Review Questions

1. Why should geometric intuition learned in $\mathbb{R}^2$ or $\mathbb{R}^3$ be treated cautiously when d is large?
2. What are the two main geometric quantities studied in this chapter?
3. Give two real big-data representations whose natural ambient dimension can be very large.

4.2 Volume Concentration in the Hypercube

4.2.1 Volume of the Unit Cube

The unit cube has side length one in each of d coordinates, hence

$$\mathrm{Vol}(C_d) = 1^d = 1.$$

The interesting question is not the total volume, but where that volume lies.

4.2.2 A Thin Shell Near the Boundary

Fix $\varepsilon \in (0, 1/2)$. Consider the inner cube obtained by remaining at least ε away from every face:

$$C_d^{\text{inner}} = [\varepsilon, 1 - \varepsilon]^d.$$

Its side length is $1 - 2\varepsilon$, so

$$\text{Vol}\left(C_d^{\text{inner}}\right) = (1 - 2\varepsilon)^d.$$

The boundary shell is $C_d \setminus C_d^{\text{inner}}$. Since the full cube has volume one, the fraction of volume in the shell is

$$\frac{\text{Vol}(C_d) - \text{Vol}(C_d^{\text{inner}})}{\text{Vol}(C_d)} = 1 - (1 - 2\varepsilon)^d. \tag{4.1}$$

where d is the dimension and ε is the fixed shell width measured from each face.

For any fixed $\varepsilon \in (0, 1/2)$,

$$0 < 1 - 2\varepsilon < 1 \qquad \Longrightarrow \qquad (1 - 2\varepsilon)^d \to 0 \quad \text{as } d \to \infty.$$

Therefore,

$$1 - (1 - 2\varepsilon)^d \to 1. \tag{4.2}$$

Key Concept

Cube volume concentration. For every fixed $\varepsilon \in (0, 1/2)$, the fraction of the unit cube lying within distance ε of at least one face approaches 1 as $d \to \infty$.

In low dimensions, a thin border is genuinely a small fraction of the region. In high dimensions, almost the entire cube belongs to such a border shell.

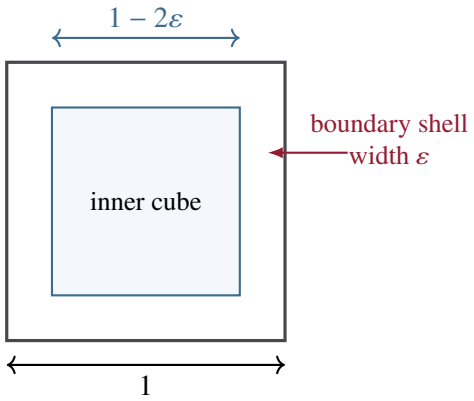


Figure 4.1: A two-dimensional picture of the construction used in every dimension. The formula, not the drawing, captures the high-dimensional effect.

4.2.3 The Diagonal Grows with Dimension

The main diagonal of C_d, from $(0, \ldots, 0)$ to $(1, \ldots, 1)$, has length

$$\sqrt{1^2 + \cdots + 1^2} = \sqrt{d}.$$

Thus the side length remains fixed at one while the diagonal grows without bound. Combined with boundary concentration, this already hints that random points will tend to lie far from one another on an absolute scale.

Review Questions

1. Derive Equation (4.1) from the volume of the inner cube.
2. Why does $(1-2\varepsilon)^d \to 0$ for fixed $\varepsilon > 0$?
3. What does "the cube has almost no interior" mean in this probabilistic volume sense?
4. Why is the growing diagonal $\sqrt{d}$ relevant to later distance results?

4.3 Volume Concentration in the Hypersphere

4.3.1 Volume of a d-Dimensional Ball

The volume of a d-dimensional ball of radius r is

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2}+1\right)} r^d, \tag{4.3}$$

where $\Gamma(\cdot)$ is the Gamma function, satisfying $\Gamma(n+1) = n!$ for integer n. The formula recovers familiar cases: $V_2(r) = \pi r^2$ and $V_3(r) = \frac{4}{3}\pi r^3$.

For the unit ball,

$$V_d(1) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2}+1\right)}.$$

Although $\pi^{d/2}$ grows exponentially, the Gamma term grows factorially. Stirling's approximation therefore implies

$$\lim_{d\to\infty} V_d(1) = 0. \tag{4.4}$$

The radius remains exactly one; it is the d-dimensional volume that vanishes.

4.3.2 A Numerical View of the Vanishing Volume

Table 4.1: Approximate unit-ball volumes across selected dimensions.

d	$V_d(1)$ (approx.)	Trend
2	3.14	familiar circle area
3	4.19	familiar sphere volume
5	5.26	near the maximum
10	2.55	already shrinking
20	2.6×10^{-2}	sharp decline
50	1.7×10^{-13}	effectively zero

The unit-ball volume is not monotone from $d = 1$ onward; it peaks at a small dimension and then decays rapidly. The asymptotic conclusion remains unambiguous.

4.3.3 Ball versus Circumscribing Cube

A radius-one ball fits inside the cube $[-1,1]^d$, whose volume is 2^d. Their volume ratio is

$$\frac{V_d(1)}{2^d} = \frac{\pi^{d/2}}{2^d \Gamma\left(\frac{d}{2}+1\right)} \longrightarrow 0. \tag{4.5}$$

Thus an inscribed ball occupies an increasingly negligible fraction of its circumscribing cube. Geometrically, more and more cube volume lies in regions that look like high-dimensional "corners" outside the ball.

4.3.4 A Thin Spherical Shell

Now consider the shell between radii $1-\varepsilon$ and 1, with fixed $\varepsilon \in (0,1)$. Since Equation (4.3) scales as r^d,

$$V_d(1-\varepsilon) = V_d(1)(1-\varepsilon)^d.$$

Therefore the fraction of ball volume in the outer shell is

$$\frac{V_d(1) - V_d(1-\varepsilon)}{V_d(1)} = 1 - (1-\varepsilon)^d \longrightarrow 1. \tag{4.6}$$

Key Concept

Sphere surface concentration. For any fixed $\varepsilon \in (0,1)$, almost all volume of the unit ball lies within distance ε of its surface as $d \to \infty$.

The cube and ball results are driven by the same elementary mechanism: a constant strictly between zero and one is raised to an exponent that grows with d.

Review Questions

1. State the volume formula for a d-ball and identify the role of the Gamma function.
2. Why can a ball keep radius one while its d-dimensional volume tends to zero?
3. Derive the spherical-shell fraction in Equation (4.6).
4. What does the ratio in Equation (4.5) say about the corners of a high-dimensional cube?

4.4 Concentration Near the Equator

4.4.1 From Surface Concentration to an Equatorial Question

Let

$$S_{d-1} = \left\{ \mathbf{x} \in \mathbb{R}^d : \sum_{i=1}^{d} x_i^2 = 1 \right\}$$

be the unit sphere. Fix a unit vector $\mathbf{e}$ as a "north-pole" direction. By rotational symmetry we may take $\mathbf{e} = \mathbf{e}_1$. The coordinate $x_1 = \mathbf{x} \cdot \mathbf{e}_1$ measures displacement from the equator perpendicular to this pole.

For a fixed width $\varepsilon > 0$, the equatorial band is

$$\{\mathbf{x} \in S_{d-1} : |x_1| \le \varepsilon\}.$$

4.4.2 Why Slices Shrink Away from the Equator

Fixing $x_1 = t$ in the sphere equation gives

$$x_2^2 + \cdots + x_d^2 = 1 - t^2.$$

The slice is therefore a sphere in the remaining coordinates with radius

$$r = \sqrt{1-t^2}.$$

A $(d-2)$-dimensional sphere of radius r has surface area proportional to r^{d-2}, so the slice area is proportional to

$$\left(\sqrt{1-t^2}\right)^{d-2} = (1-t^2)^{(d-2)/2}. \tag{4.7}$$

At the equator $t = 0$ this factor is one. For every fixed $0 < |t| < 1$, it tends to zero exponentially fast with d. To obtain surface probability per unit t, the slice area must also be multiplied by the surface Jacobian $1/\sqrt{1-t^2}$. Thus the coordinate density is proportional to $(1-t^2)^{(d-3)/2}$ for $d \geq 3$; slice area alone is not a probability density.

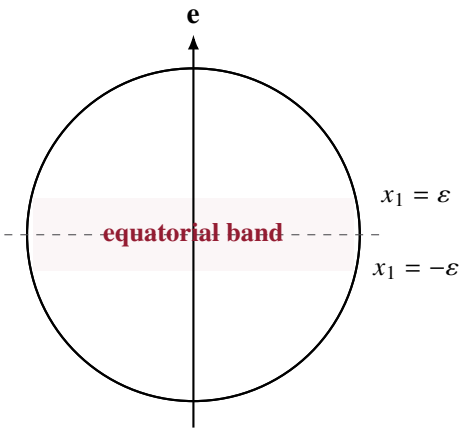


Figure 4.2: Low-dimensional cross-section of an equatorial band. The concentration result holds for the high-dimensional sphere.

4.4.3 A Concentration Bound

For a uniformly distributed point on S_{d-1}, the coordinate satisfies $\mathbb{E}[e^{tx_1}] \leq e^{t^2/(2d)}$. Applying the Chernoff bound to each tail and adding them gives

$$\Pr(|x_1| > \varepsilon) \leq 2e^{-d\varepsilon^2/2}. \tag{4.8}$$

For every fixed $\varepsilon > 0$,

$$e^{-d\varepsilon^2/2} \to 0, \qquad \Pr(|x_1| \leq \varepsilon) \to 1.$$

The band does not become wider; instead, a larger fraction of surface area moves into the same fixed-width band.

4.4.4 Random Unit Vectors Become Nearly Orthogonal

Because the sphere is rotationally symmetric, the pole direction can be any fixed unit vector. Let $\mathbf{x}$ and $\mathbf{y}$ be independent, uniformly random unit vectors. Condition on $\mathbf{x}$ and use it as the pole. Then $\mathbf{x} \cdot \mathbf{y}$ is the coordinate of $\mathbf{y}$ along that pole direction. Equation (4.8) therefore gives

$$\Pr(|\mathbf{x} \cdot \mathbf{y}| > \varepsilon) \leq 2e^{-d\varepsilon^2/2}. \tag{4.9}$$

Since $\mathbf{x} \cdot \mathbf{y} = \cos\theta$ for unit vectors, a dot product near zero means an angle near $90°$.

Key Concept

Near-orthogonality. Two independent random unit vectors in high-dimensional space are overwhelmingly likely to be almost perpendicular. Near-perpendicularity becomes typical rather than exceptional.

Review Questions

1. Why does fixing one coordinate of S_{d-1} produce a lower-dimensional sphere of radius $\sqrt{1-t^2}$?
2. Why does a small reduction in radius cause a large reduction in slice area when d is large?
3. How does rotational symmetry turn equatorial concentration into a statement about two random vectors?
4. Interpret the probability bound in Equation (4.9).

4.5 Distance Concentration and the Curse of Dimensionality

4.5.1 Random Points in the Unit Cube

Let $\mathbf{x}, \mathbf{y} \in [0,1]^d$ be drawn independently and uniformly. Their squared Euclidean distance is

$$\|\mathbf{x}-\mathbf{y}\|_2^2 = \sum_{i=1}^{d}(x_i - y_i)^2. \tag{4.10}$$

The summands are independent and identically distributed across coordinates, so expectation and variance can be analyzed coordinate by coordinate.

4.5.2 Expected Squared Distance

For independent $X, Y \sim \text{Uniform}(0,1)$,

$$\mathbb{E}[X] = \mathbb{E}[Y] = \frac{1}{2}, \qquad \mathbb{E}[X^2] = \mathbb{E}[Y^2] = \frac{1}{3}, \qquad \mathbb{E}[XY] = \frac{1}{4}.$$

Hence

$$\mathbb{E}[(X-Y)^2] = \mathbb{E}[X^2] - 2\mathbb{E}[XY] + \mathbb{E}[Y^2] \tag{4.11}$$

$$= \frac{1}{3} - 2\left(\frac{1}{4}\right) + \frac{1}{3} = \frac{1}{6}. \tag{4.12}$$

By linearity of expectation,

$$\mathbb{E}\left[\|\mathbf{x}-\mathbf{y}\|_2^2\right] = \frac{d}{6}. \tag{4.13}$$

Thus the typical squared distance grows linearly with d; the corresponding distance scale grows on the order of $\sqrt{d}$.

4.5.3 Relative Spread Shrinks

Let

$$Z = \|\mathbf{x}-\mathbf{y}\|_2^2.$$

Since Z is a sum of d i.i.d. coordinate contributions, its variance grows linearly:

$$\text{Var}(Z) = \frac{7d}{180}$$

because $\mathbb{E}[(X-Y)^4] = 1/15$ and $\text{Var}((X-Y)^2) = 1/15 - (1/6)^2 = 7/180$. Therefore

$$\text{Std}(Z) = O(\sqrt{d}), \qquad \mathbb{E}[Z] = O(d).$$

The coefficient of variation is

$$\frac{\mathrm{Std}(Z)}{\mathbb{E}[Z]} = \sqrt{\frac{7}{5d}} = O\left(\frac{1}{\sqrt{d}}\right) \longrightarrow 0. \tag{4.14}$$

The absolute spread can grow while the *relative* spread collapses.

4.5.4 Chebyshev Makes the Concentration Explicit

Chebyshev's inequality states that for a random variable with mean μ and standard deviation σ,

$$\Pr(|Z - \mu| \geq k\sigma) \leq \frac{1}{k^2}.$$

Using $\mu = d/6$ and $\sigma^2 = 7d/180$ gives, for every $k > 0$,

$$\Pr\left(\left|\|\mathbf{x} - \mathbf{y}\|_2^2 - \frac{d}{6}\right| \geq k\sqrt{d}\right) \leq \frac{7}{180k^2}, \tag{4.15}$$

The important scaling is that deviations grow like $\sqrt{d}$ while the mean grows like d. Equivalently, for any fixed $\delta > 0$, $\Pr(|Z/(d/6) - 1| \geq \delta) \leq 7/(5d\delta^2) \to 0$.

4.5.5 Nearest and Farthest Become Relatively Similar

For a fixed number $n \geq 2$ of independent uniform points in $[0, 1]^d$, a union bound over the finitely many pairs implies convergence in probability:

$$\frac{D_{\max}}{D_{\min}} \longrightarrow 1 \qquad \text{as } d \to \infty, \tag{4.16}$$

where $D_{\max}$ and $D_{\min}$ denote the largest and smallest pairwise distances in the set. The nearest pair and farthest pair become almost equally far apart *relative to the common distance scale*.

Table 4.2: Illustrative qualitative trend for random points in $[0, 1]^d$.

d	Mean squared distance $d/6$	Relative spread	Practical effect
2	0.33	High	"Near" and "far" remain visibly distinct.
10	1.67	Moderate	Some distance discrimination remains.
100	16.7	Low	Distances cluster around the mean.
1000	166.7	Very low	Nearest and farthest become relatively similar.

Definition – Curse of Dimensionality in Distance-Based Methods

As the feature-space dimension grows, the relative discriminative power of standard distance metrics degrades: "near" and "far" become progressively harder to distinguish relative to the scale of typical distances.

Review Questions

1. Derive $\mathbb{E}[(X - Y)^2] = 1/6$ for independent $X, Y \sim \mathrm{Uniform}(0, 1)$.
2. Why does $\mathbb{E}\|\mathbf{x} - \mathbf{y}\|_2^2$ grow as d while its standard deviation grows only as $\sqrt{d}$?

3. What does Equation (4.16) mean for a nearest-neighbor algorithm?
4. State the curse of dimensionality precisely in terms of distance discrimination.

4.6 Metric-Space Breakdown and Practical Consequences

4.6.1 What "Breakdown" Means

A metric does not stop being mathematically valid in high dimension. Euclidean distance still satisfies non-negativity, symmetry, identity of indiscernibles, and the triangle inequality. The breakdown is algorithmic: the metric loses its ability to produce a strong relative separation between similar and dissimilar points.

4.6.2 Why Euclidean Distance Is Vulnerable

Euclidean distance aggregates coordinate-wise contributions:

$$\|\mathbf{x}-\mathbf{y}\|_2 = \sqrt{\sum_{i=1}^{d}(x_i-y_i)^2}.$$

When many coordinates contribute independent variation, the aggregate behaves increasingly like its average value. A small subset of genuinely informative coordinates can therefore be diluted by the cumulative contribution of many weak or irrelevant coordinates.

4.6.3 Other Metrics

The same broad concentration issue affects other standard norms, although the exact rates differ:

$$\|\mathbf{x}-\mathbf{y}\|_1 = \sum_{i=1}^{d}|x_i-y_i|, \qquad \|\mathbf{x}-\mathbf{y}\|_\infty = \max_i |x_i-y_i|.$$

Manhattan distance exhibits qualitatively similar concentration; Chebyshev distance behaves differently because it depends only on the largest coordinate-wise difference. The course-level lesson is not that every metric fails identically, but that naive high-dimensional distance discrimination is broadly fragile.

4.6.4 Nearest-Neighbor Search

Nearest-neighbor search assumes that the nearest candidate is meaningfully closer than typical alternatives. Distance concentration weakens exactly this signal. Searching the raw space can therefore become less informative even before considering the separate computational cost of searching a large dataset.

4.6.5 Clustering and Classification

Algorithms such as k-means assign points to their nearest centroid. If all centroid distances are similar, assignments become more sensitive to noise and less geometrically meaningful. Likewise, k-nearest-neighbor classification can suffer when the selected neighbors are only marginally closer than many other training points.

Key Concept

High-dimensional data is not inherently unusable. The failure is in applying low-dimensional distance intuition unchanged. The natural response is to change the representation, reduce dimension, or use search methods designed for high-dimensional geometry.

This motivates the next chapter: the Johnson–Lindenstrauss lemma shows that one can project a finite

dataset into a much smaller Euclidean space while approximately preserving all pairwise distances.

4.7 Synthesis

4.7.1 The Chain of Results

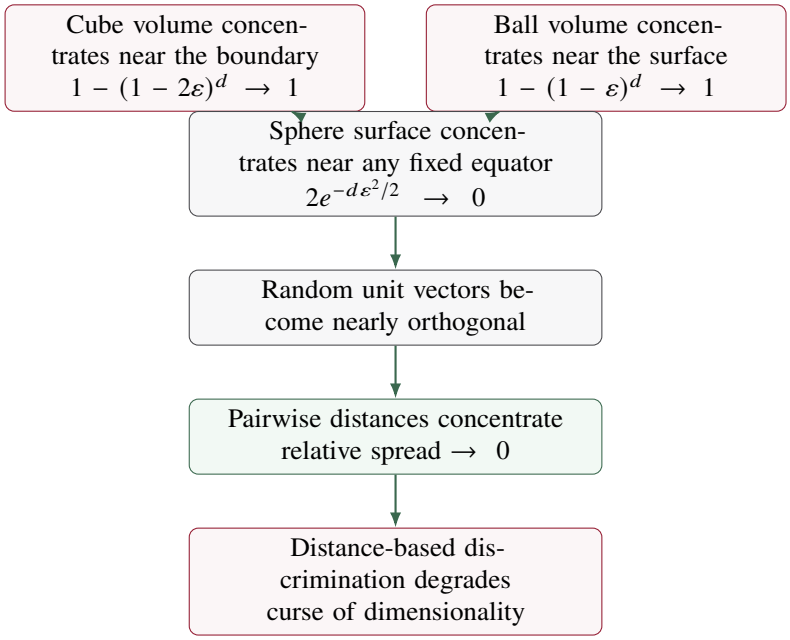


Figure 4.3: Logical chain connecting volume concentration to the algorithmic curse of dimensionality.

4.7.2 Connection Back to the Infrastructure Layer

These geometric effects describe exactly the vector data that the systems in Chapters 1–3 are designed to store and process. Scale-out infrastructure solves the systems problem of handling enormous datasets; it does not remove the mathematical behavior of the representations stored inside those systems.

4.8 Key Terms

Table 4.3: Key terminology for high-dimensional geometry.

Term	Meaning
Volume concentration	Most volume of a high-dimensional cube or ball lies near its boundary or surface.
Vanishing unit-ball volume	$V_d(1) \to 0$ even though the radius remains one.
Equatorial concentration	Surface area of a high-dimensional sphere concentrates near any fixed equator.
Near-orthogonality	Independent random unit vectors have dot product close to zero with high probability.
Distance concentration	Pairwise distances cluster tightly relative to their typical scale.
Curse of dimensionality	Loss of useful distance discrimination as dimension grows.

4.9 Chapter Summary

Table 4.4: Compact summary of high-dimensional concentration phenomena.

Concept	Mathematical form	Consequence
Cube shell	$1-(1-2\varepsilon)^d \to 1$	Almost all cube volume lies near at least one face.
Unit-ball volume	$\pi^{d/2}/\Gamma(d/2+1) \to 0$	A fixed-radius unit ball has vanishing d-volume.
Ball shell	$1-(1-\varepsilon)^d \to 1$	Remaining ball volume lies near the surface.
Equatorial band	$\Pr(\lvert x_1\rvert > \varepsilon) \le 2e^{-d\varepsilon^2/2}$	Most sphere surface lies near every fixed equator.
Near-orthogonality	$\Pr(\lvert \mathbf{x}\cdot\mathbf{y}\rvert > \varepsilon) \le 2e^{-d\varepsilon^2/2}$	Random directions are almost perpendicular.
Mean squared distance	$\mathbb{E}\lVert\mathbf{x}-\mathbf{y}\rVert_2^2 = d/6$	Typical squared distance grows linearly in d.
Relative spread	$O(1/\sqrt{d})$	Pairwise distances become relatively indistinguishable.
Algorithmic effect	$D_{\max}/D_{\min} \to 1$ qualitatively	Raw distance becomes a weaker similarity signal.

4.10 Quick Review

1. The unit hypercube $C_d = [0,1]^d$ always has total volume one.
2. Its inner cube at distance at least ε from every face has volume $(1-2\varepsilon)^d$.
3. Hence the boundary-shell fraction $1-(1-2\varepsilon)^d$ approaches one.
4. The cube diagonal has length $\sqrt{d}$.
5. A radius-r d-ball has volume $\pi^{d/2}r^d/\Gamma(d/2+1)$.
6. The unit-ball volume tends to zero as d grows.
7. An inscribed unit ball occupies a vanishing fraction of its circumscribing cube.
8. A fixed-width shell near a ball's surface contains asymptotically all of its volume.
9. Slices of the sphere away from the equator have radius $\sqrt{1-t^2}$ and exponentially smaller area.
10. Surface area therefore concentrates near any fixed equator.
11. Random unit vectors are nearly orthogonal with overwhelming probability.
12. For random $\mathbf{x},\mathbf{y} \in [0,1]^d$, the expected squared Euclidean distance is $d/6$.
13. The standard deviation of squared distance grows like $\sqrt{d}$ while the mean grows like d.
14. The relative spread of distances therefore decays as $O(1/\sqrt{d})$.
15. Nearest and farthest distances become relatively similar.

16. The curse of dimensionality is the resulting loss of discriminative power of naive distance-based methods.
17. Nearest-neighbor search, Euclidean clustering, and Euclidean-based classification are directly affected.
18. Dimensionality reduction is a natural response because it seeks a lower-dimensional representation that preserves the useful geometry.

4.11 Further Reading

Further reading includes:

- Blum, A., Hopcroft, J., & Kannan, R. (2020). *Foundations of Data Science*, Chapter 2.
- Course reference materials on high-dimensional geometry and concentration phenomena.

CHAPTER 5

Dimensionality Reduction and Random Projections

Chapter 4 showed why raw high-dimensional Euclidean geometry becomes problematic: pairwise distances concentrate, random directions become nearly orthogonal, and distance-based algorithms lose discriminative power. The natural response is to ask whether the data can be mapped into a much smaller space while preserving the geometric relationships that matter.

The Johnson–Lindenstrauss (JL) lemma gives a constructive answer. A random linear map into only $O(\log n/\varepsilon^2)$ dimensions approximately preserves all pairwise distances among n points, with high probability, independently of the original ambient dimension d.

Key Concept

The JL idea is data-oblivious dimensionality reduction: choose a random projection without inspecting the dataset, then rely on concentration of measure to guarantee that all pairwise Euclidean distances are approximately preserved.

5.1 The Dimensionality-Reduction Problem

5.1.1 The Core Goal

Let

$$S = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d.$$

We seek a map

$$f : \mathbb{R}^d \to \mathbb{R}^k, \qquad k \ll d,$$

that preserves the geometric relationships among the points. In this lecture, the structure of interest is pairwise Euclidean distance.

5.1.2 Exact Isometry

Definition – Isometry

A map f is an isometry on a set S if

$$\|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|_2 = \|\mathbf{x}_i - \mathbf{x}_j\|_2$$

for every pair $\mathbf{x}_i, \mathbf{x}_j \in S$.

Rotations, reflections, and translations are familiar isometries. However, an exact linear isometry from all of $\mathbb{R}^d$ into $\mathbb{R}^k$ with $k < d$ is impossible: every linear map $A : \mathbb{R}^d \to \mathbb{R}^k$ has a nontrivial null space, so some nonzero vector is mapped to zero.

5.1.3 Approximate Isometry

Exact preservation is stronger than most applications require. A controlled multiplicative distortion is sufficient:

$$(1-\varepsilon)\|\mathbf{x}_i-\mathbf{x}_j\|_2 \le \|f(\mathbf{x}_i)-f(\mathbf{x}_j)\|_2 \le (1+\varepsilon)\|\mathbf{x}_i-\mathbf{x}_j\|_2. \tag{5.1}$$

Here $\varepsilon \in (0,1)$ controls the allowed relative error. The lower bound prevents excessive contraction; the upper bound prevents excessive expansion.

Definition – $(1 \pm \varepsilon)$ Distortion

A map satisfies the $(1 \pm \varepsilon)$ distortion criterion on S if Equation (5.1) holds simultaneously for every pair in S.

The simultaneous requirement is important. A projection that preserves one pair with high probability is not enough when a dataset contains $\binom{n}{2}$ different pairs.

5.1.4 Data-Dependent versus Data-Oblivious Reduction

Two broad strategies illustrate different trade-offs.

Table 5.1: PCA/SVD and random projection solve different dimensionality-reduction objectives.

Aspect	PCA / SVD	Random Projection (JL)
Depends on data?	Yes; the projection directions are learned from the dataset.	No; the projection can be sampled before observing the data.
Optimizes for	Reconstruction error / explained variance.	Pairwise distance preservation.
Coordination	Requires global covariance/SVD-style computation.	Each point can be projected independently.
Target dimension	Chosen from retained variance or rank.	$k = O(\log n/\varepsilon^2)$, independent of d.
Best fit	Compression, denoising, interpretable principal axes.	Fast preprocessing before large-scale distance-based algorithms.

PCA/SVD is optimal for low-rank reconstruction, while JL is attractive when distributed scalability and distance preservation are the primary goals.

Review Questions

1. Why is exact linear dimensionality reduction impossible when $k < d$?
2. Interpret both inequalities in the $(1 \pm \varepsilon)$ criterion.
3. Why must a JL-style guarantee hold for all $\binom{n}{2}$ pairs simultaneously?
4. How do PCA/SVD and random projection differ in objective and scalability?

5.2 Random Projections

5.2.1 The Construction

Choose a random matrix

$$A \in \mathbb{R}^{k\times d}$$

whose entries are independent standard normal random variables:

$$A_{ij} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1), \qquad i = 1,\dots,k, \quad j = 1,\dots,d. \tag{5.2}$$

Define

$$f(\mathbf{x}) = \frac{1}{\sqrt{k}} A\mathbf{x} \in \mathbb{R}^k. \tag{5.3}$$

The $1/\sqrt{k}$ factor is not cosmetic: it normalizes the expected squared length.

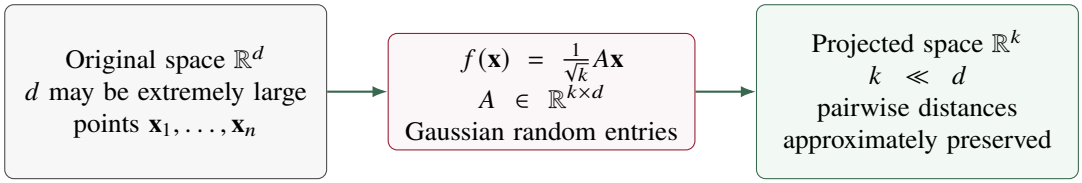


Figure 5.1: Random projection from a high-dimensional ambient space into a much smaller Euclidean space.

5.2.2 Why Gaussian Rows Are Isotropic

Fix a vector $\mathbf{x} \in \mathbb{R}^d$. For one row $\mathbf{a}_i$ of A,

$$(A\mathbf{x})_i = \mathbf{a}_i^\top \mathbf{x} = \sum_{j=1}^{d} A_{ij} x_j.$$

Because the A_{ij} are independent $\mathcal{N}(0,1)$ variables,

$$(A\mathbf{x})_i \sim \mathcal{N}\left(0, \|\mathbf{x}\|_2^2\right).$$

Thus the distribution depends only on the length of $\mathbf{x}$, not on its direction. This rotational invariance is the algebraic form of isotropy.

For a unit vector $\mathbf{u}$,

$$(A\mathbf{u})_i \sim \mathcal{N}(0,1)$$

for every row independently.

5.2.3 Why the Normalization Preserves Expected Squared Length

Since the k rows are independent,

$$\mathbb{E}\|A\mathbf{x}\|_2^2 = \sum_{i=1}^{k} \mathbb{E}(A\mathbf{x})_i^2 \tag{5.4}$$

$$= \sum_{i=1}^{k} \|\mathbf{x}\|_2^2 \tag{5.5}$$

$$= k\|\mathbf{x}\|_2^2. \tag{5.6}$$

Therefore, for the normalized projection,

$$\mathbb{E}\|f(\mathbf{x})\|_2^2 = \mathbb{E}\left\|\frac{1}{\sqrt{k}} A\mathbf{x}\right\|_2^2 = \|\mathbf{x}\|_2^2. \tag{5.7}$$

Key Concept

The normalized Gaussian projection is unbiased for squared length. The JL proof strengthens this expectation statement into a high-probability concentration statement that holds for every pair in the dataset simultaneously.

Review Questions

1. Why are Gaussian rows direction-agnostic?
2. Derive Equation (5.7).
3. What role does the $1/\sqrt{k}$ factor play?
4. Why does distance preservation reduce to length preservation of difference vectors?

5.3 The Johnson–Lindenstrauss Lemma

5.3.1 Formal Statement

Key Concept

Johnson–Lindenstrauss Lemma. Let $\varepsilon \in (0, 1/2]$, $n \geq 2$, and $S = \{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$. Set

$$k = \left\lceil \frac{24 \ln n}{\varepsilon^2} \right\rceil.$$

If $A \in \mathbb{R}^{k \times d}$ has i.i.d. $\mathcal{N}(0, 1)$ entries and $f(\mathbf{x}) = A\mathbf{x}/\sqrt{k}$, then with probability at least $1 - 1/n$,

$$(1 - \varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2 \leq \|f(\mathbf{x}_i) - f(\mathbf{x}_j)\|_2 \leq (1 + \varepsilon)\|\mathbf{x}_i - \mathbf{x}_j\|_2$$

for all $\binom{n}{2}$ pairs simultaneously.

The striking feature is what does *not* appear in the target dimension: the original dimension d. The required k depends only logarithmically on the number of points and quadratically on the inverse distortion tolerance.

5.3.2 Parameter Dependence

The target dimension obeys

$$k = O\left(\frac{\log n}{\varepsilon^2}\right). \tag{5.8}$$

The dependencies have a direct interpretation:

- increasing n increases k only logarithmically;
- demanding smaller distortion ε is expensive because k scales as $1/\varepsilon^2$;
- d does not appear in the bound.

5.3.3 Worked Numerical Example

For $n = 10^6$ points and $\varepsilon = 0.1$,

$$k = \left\lceil \frac{24 \ln(10^6)}{0.1^2} \right\rceil \approx 33{,}158.$$

Thus the same target dimension works whether the original vectors live in $\mathbb{R}^{10^5}$ or $\mathbb{R}^{10^7}$, provided the original space is large enough that reduction is actually useful.

Remark – The bound is conservative

The theorem gives a sufficient dimension with explicit constants for the proof route used here. Practical random projections often work with smaller k; the theorem's purpose is a worst-case probabilistic guarantee for all pairs.

Review Questions

1. State the JL lemma including the success probability and target dimension.
2. Why does k depend on $\log n$ rather than n itself?
3. Why is tight distortion expensive in terms of $1/\varepsilon^2$?
4. Why is independence from the original dimension d so important?

5.4 Proof of the JL Lemma

The proof follows the elementary route of Dasgupta and Gupta. Its four steps are: reduce one pair to one unit vector, identify a chi-squared random variable, prove an exponential concentration bound with an MGF/Chernoff argument, and apply a union bound over all pairs.

5.4.1 Step 1: Reduce One Pair to a Unit Vector

Fix a pair of distinct points $(\mathbf{x}_i, \mathbf{x}_j)$ and define

$$\mathbf{u} = \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|_2}, \qquad \|\mathbf{u}\|_2 = 1.$$

Because f is linear,

$$f(\mathbf{x}_i) - f(\mathbf{x}_j) = \frac{1}{\sqrt{k}} A(\mathbf{x}_i - \mathbf{x}_j).$$

Therefore the distortion condition for this pair becomes

$$(1-\varepsilon)^2 \le \frac{\|A\mathbf{u}\|_2^2}{k} \le (1+\varepsilon)^2. \tag{5.9}$$

The one-pair problem is now concentration of one scalar random quantity around one. It suffices to prove the stronger squared-length condition $1 - \varepsilon \le \|A\mathbf{u}\|_2^2/k \le 1 + \varepsilon$: taking square roots implies the desired length bounds since $\sqrt{1-\varepsilon} \ge 1 - \varepsilon$ and $\sqrt{1+\varepsilon} \le 1 + \varepsilon$.

5.4.2 Step 2: Identify the Chi-Squared Distribution

For a fixed unit vector $\mathbf{u}$, each component of $A\mathbf{u}$ is an independent standard normal:

$$(A\mathbf{u})_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1).$$

Write these variables as $g_1, \ldots, g_k$. Then

$$Z = \|A\mathbf{u}\|_2^2 = \sum_{i=1}^{k} g_i^2 \sim \chi_k^2. \tag{5.10}$$

Definition – Chi-Squared Distribution

If $g_1, \ldots, g_k \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1)$, then

$$Z = \sum_{i=1}^{k} g_i^2$$

has the chi-squared distribution with k degrees of freedom, written $Z \sim \chi_k^2$.

Its moments are

$$\mathbb{E}[Z] = k, \qquad \mathrm{Var}(Z) = 2k. \tag{5.11}$$

For the normalized quantity $Y = Z/k$,

$$\mathbb{E}[Y] = 1, \qquad \mathrm{Var}(Y) = \frac{2}{k}.$$

This already suggests concentration as k increases, but Chebyshev would give only polynomial tails. JL needs exponentially small failure probability so that a union bound over many pairs remains useful.

5.4.3 Step 3: Exponential Concentration via the MGF

For $Z \sim \chi_k^2$, the moment generating function is

$$\mathbb{E}[e^{tZ}] = (1-2t)^{-k/2}, \qquad t < \frac{1}{2}. \tag{5.12}$$

This follows because Z is a sum of independent squared standard normals and

$$\mathbb{E}[e^{tg^2}] = (1-2t)^{-1/2}.$$

For the upper tail, Markov's inequality applied to e^{tZ} gives, for $t > 0$,

$$\Pr\big(Z > (1+\varepsilon)k\big) \le \frac{\mathbb{E}[e^{tZ}]}{e^{t(1+\varepsilon)k}} \tag{5.13}$$

$$= \frac{(1-2t)^{-k/2}}{e^{t(1+\varepsilon)k}}. \tag{5.14}$$

Optimizing with

$$t^* = \frac{\varepsilon}{2(1+\varepsilon)}$$

yields

$$\Pr\big(Z > (1+\varepsilon)k\big) \le \left[(1+\varepsilon)e^{-\varepsilon}\right]^{k/2}. \tag{5.15}$$

A symmetric lower-tail calculation gives

$$\Pr\big(Z < (1-\varepsilon)k\big) \le \left[(1-\varepsilon)e^{\varepsilon}\right]^{k/2}. \tag{5.16}$$

For $0 < \varepsilon \le 1/2$, the inequalities $\ln(1+\varepsilon) \le \varepsilon - \varepsilon^2/2 + \varepsilon^3/3$ and $\ln(1-\varepsilon) \le -\varepsilon - \varepsilon^2/2$ bound each tail by $e^{-k\varepsilon^2/8}$. Adding the tails gives

$$\Pr\left(\left|\frac{Z}{k} - 1\right| > \varepsilon\right) \le 2e^{-k\varepsilon^2/8}. \tag{5.17}$$

This is the crucial one-pair concentration bound: failure probability decays exponentially in $k\varepsilon^2$.

5.4.4 Step 4: Union Bound over All Pairs

There are

$$\binom{n}{2} \leq \frac{n^2}{2}$$

pairs. Applying Equation (5.17) and the union bound,

$$\Pr(\text{any pair fails}) \leq \frac{n^2}{2} \cdot 2e^{-k\varepsilon^2/8} = n^2 e^{-k\varepsilon^2/8}. \tag{5.18}$$

To make this at most $1/n$, it suffices that

$$n^2 e^{-k\varepsilon^2/8} \leq \frac{1}{n}.$$

Taking logarithms gives

$$-\frac{k\varepsilon^2}{8} \leq -3\ln n \qquad \Longrightarrow \qquad k \geq \frac{24\ln n}{\varepsilon^2}.$$

We therefore choose

$$k = \left\lceil \frac{24\ln n}{\varepsilon^2} \right\rceil \tag{5.19}$$

which establishes the stated success probability. The squared-length guarantee already implies the required length guarantee without any further loss in probability.

5.4.5 Why the Proof Is Constructive

The argument does more than prove existence. It specifies an algorithm:

1. sample one matrix A with i.i.d. $\mathcal{N}(0,1)$ entries;
2. project every point with $f(\mathbf{x}) = A\mathbf{x}/\sqrt{k}$;
3. with probability at least $1 - 1/n$, all pairwise distances satisfy the required distortion bound.

A dense projection requires $O(kd)$ work per point and $O(nkd)$ for n points.

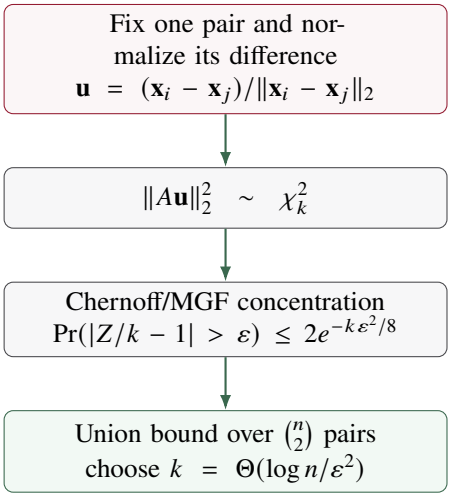


Figure 5.2: Proof architecture of the Johnson–Lindenstrauss lemma.

Review Questions

1. Why does one pair reduce to concentration of $\|A\mathbf{u}\|_2^2/k$?

2. Why is $\|A\mathbf{u}\|_2^2$ chi-squared with k degrees of freedom?
3. Why is an exponential Chernoff bound preferable to Chebyshev here?
4. Derive the union-bound expression in Equation (5.18).
5. What makes the proof constructive rather than purely existential?

5.5 Practical Implications and Limits

5.5.1 JL as a Preprocessing Step

A typical workflow is

$$\text{high-dimensional data in } \mathbb{R}^d \longrightarrow \text{random projection into } \mathbb{R}^k \longrightarrow \text{distance-based algorithm.}$$

The downstream method works in a much smaller space while pairwise Euclidean distances remain approximately preserved.

5.5.2 Sparse Random Projections

A dense Gaussian matrix needs kd random entries and $O(kd)$ multiplications per point. For very large sparse vectors, a sparse matrix can be substantially cheaper. One lecture example uses the Achlioptas-style distribution

$$A_{ij} = \begin{cases} +1, & \text{with probability } 1/6, \\ 0, & \text{with probability } 2/3, \\ -1, & \text{with probability } 1/6. \end{cases} \tag{5.20}$$

Here $\mathbb{E}[A_{ij}^2] = 1/3$, so use $f(\mathbf{x}) = \sqrt{3/k}\, A\mathbf{x}$ to preserve expected squared length. Equivalently, use nonzero entries $\pm\sqrt{3}$ with the usual $1/\sqrt{k}$ normalization. The same JL-style guarantee is retained while many matrix entries are zero.

5.5.3 Fast JL Transform

The Fast JL Transform replaces a dense Gaussian matrix with a structured product

$$A = PHD,$$

where:

- D is a random diagonal sign matrix with independent ± 1 entries;
- H is a Hadamard transform, computable in $O(d \log d)$ time;
- P is a random subsampling matrix.

This reduces projection time from $O(kd)$ to roughly $O(d \log d)$ per point in the lecture's comparison, at the cost of a more structured construction and modified constants.

5.5.4 What JL Does Not Guarantee

JL guarantees approximate pairwise Euclidean distances. It does not promise exact preservation of every geometric quantity:

- inner products are only approximately preserved;
- very close distance orderings can change when two candidates are near-ties;

- angles are preserved only approximately through their relationship to Euclidean geometry.

5.5.5 When JL Can Be Unhelpful

Several regimes limit its usefulness:

- **Very small ε:** because $k \propto 1/\varepsilon^2$, extremely tight distortion may require $k \geq d$, eliminating any reduction.
- **Extremely large n:** the logarithm grows slowly, but it is still unbounded.
- **Non-Euclidean metrics:** the classical JL guarantee is Euclidean; edit distance, Earth Mover's Distance, and other metrics require different embedding theory.

5.5.6 Accuracy–Compression Trade-off

For $n = 10^6$, the lecture's bound gives approximately:

Table 5.2: Illustrative trade-off between distortion and target dimension for $n = 10^6$.

ε	Required k (approx.)	Interpretation
0.5	1,327	Very compact, but 50% distortion can be too large.
0.1	33,158	Moderate accuracy–compression compromise.
0.01	3,315,723	Tight guarantee; k may exceed the original d.

Key Concept

JL is most compelling when the original ambient dimension is enormous, the dataset is finite, Euclidean distances are the relevant structure, and a moderate multiplicative distortion is acceptable.

5.6 Synthesis

5.6.1 From Curse to Cure

Chapter 4 and this chapter form one continuous argument:

Table 5.3: From high-dimensional pathology to random-projection remedy.

Chapter 4	This chapter
Volume and distance concentrate.	Use concentration to analyze a random linear map.
Random directions are nearly orthogonal.	Exploit Gaussian isotropy so no direction is systematically favored.
Distance discrimination degrades.	Preserve the finite dataset's original pairwise distances approximately.
Curse of dimensionality is identified.	A constructive probabilistic dimensionality-reduction method is obtained.

5.7 Key Terms

Table 5.4: Key terminology for dimensionality reduction and JL.

Term	Meaning
Isometry	Map that exactly preserves pairwise distances.
$(1 \pm \varepsilon)$ **distortion**	Multiplicative approximate preservation of every pairwise distance.
Random projection	Data-oblivious linear map defined by a random matrix.
Gaussian random matrix	Matrix whose entries are independent $\mathcal{N}(0, 1)$ variables.
Isotropy	Distributional symmetry that treats all input directions identically.
Chi-squared distribution	Distribution of a sum of squared independent standard normals.
MGF / Chernoff bound	Technique used to obtain exponentially decaying tail probabilities.
Union bound	Bounds the probability that any of many failure events occurs by summing individual failure probabilities.
JL lemma	Embeds n points into $O(\log n/\varepsilon^2)$ dimensions while approximately preserving all pairwise distances.
Sparse random projection	Projection with many zero matrix entries to reduce arithmetic cost.
Fast JL Transform	Structured random transform using sign flips, Hadamard mixing, and subsampling.

5.8 Chapter Summary

Table 5.5: Compact summary of dimensionality reduction by random projection.

Concept	Core statement	Significance
Approximate isometry	Distances preserved within $(1 \pm \varepsilon)$	Relaxation makes dimensionality reduction feasible.
Gaussian projection	$f(\mathbf{x}) = A\mathbf{x}/\sqrt{k}$	Data-oblivious and embarrassingly parallel.
Expected norm	$\mathbb{E}\|f(\mathbf{x})\|^2 = \|\mathbf{x}\|^2$	Normalization removes systematic scaling.
JL dimension	$k = O(\log n/\varepsilon^2)$	Independent of original dimension d.
One-pair reduction	$\|A\mathbf{u}\|^2 \sim \chi^2_k$	Converts geometry into probability.
Tail bound	$\Pr(\|Z/k - 1\| > \varepsilon) \leq 2e^{-k\varepsilon^2/8}$	Failure decreases exponentially in $k\varepsilon^2$.
Union bound	$\Pr(\text{any failure}) \leq n^2 e^{-k\varepsilon^2/8}$	Converts one-pair concentration into an all-pairs guarantee.
Sparse/Fast variants	Fewer arithmetic operations	Make random projection practical at large d.

5.9 Quick Review

1. Dimensionality reduction seeks $k \ll d$ while preserving useful structure.

2. Exact linear isometry of all of $\mathbb{R}^d$ into $\mathbb{R}^k$ is impossible when $k < d$.
3. JL relaxes exact preservation to a multiplicative $(1 \pm \varepsilon)$ guarantee.
4. PCA/SVD is data-dependent and reconstruction-oriented; JL is data-oblivious and distance-oriented.
5. A Gaussian JL matrix has independent $\mathcal{N}(0, 1)$ entries.
6. The normalized projection is $f(\mathbf{x}) = A\mathbf{x}/\sqrt{k}$.
7. Gaussian rotational invariance makes the construction isotropic.
8. Expected squared length is preserved exactly by the normalization.
9. The JL target dimension scales as $O(\log n/\varepsilon^2)$ and does not depend on d.
10. For one normalized difference vector $\mathbf{u}$, $\|A\mathbf{u}\|^2$ is chi-squared with k degrees of freedom.
11. A chi-squared variable has mean k and variance $2k$.
12. The MGF/Chernoff method yields exponentially small tail probability.
13. The one-pair bound is at most $2e^{-k\varepsilon^2/8}$ in the lecture's simplified form.
14. A union bound extends the guarantee to all $\binom{n}{2}$ pairs.
15. Choosing $k = \lceil 24 \ln n/\varepsilon^2 \rceil$ gives the stated high-probability guarantee.
16. The proof is constructive: sample a matrix and apply it to every point.
17. Dense projection costs $O(kd)$ per point.
18. Sparse random projections replace many entries by zero.
19. FJLT uses a structured product PHD to reduce projection cost.
20. JL does not preserve exact nearest-neighbor ordering, exact angles, or non-Euclidean metrics in general.

5.10 Further Reading

Further reading includes:

- Dasgupta, S., & Gupta, A. (2003). *An Elementary Proof of a Theorem of Johnson and Lindenstrauss*. Random Structures & Algorithms, 22(1).
- Course reference materials on random projections and dimensionality reduction.

CHAPTER 6

Vector Representations of Text and Embeddings

The previous two chapters treated data as vectors and studied the geometry of those vectors. This chapter asks the prerequisite question for text: how does raw symbolic language become a numerical vector representation in the first place? The answer evolves from sparse count-based vectors such as TF–IDF to learned dense embeddings such as Word2Vec, with scalability considerations at every step.

The progression is important. A vocabulary gives text a coordinate system; sparse representations make that coordinate system explicit but extremely high-dimensional; the distributional hypothesis motivates learning compact dense vectors from context; negative sampling makes those vectors trainable at web scale; and tokenization determines the vocabulary on which the entire pipeline depends.

Key Concept

Every text representation is a design choice about what structure to preserve. Sparse TF–IDF preserves explicit term identity and weighting; dense embeddings sacrifice coordinate-level interpretability in exchange for compact learned semantic geometry.

6.1 Text as High-Dimensional Data

6.1.1 Why Text Must Become Vectors

Machine-learning models, clustering methods, and similarity-search algorithms operate on numerical objects rather than raw strings. A text-processing pipeline therefore requires a map from words or documents into a vector space.

Let

$$V = \{w_1, w_2, \ldots, w_{|V|}\}$$

be the vocabulary of a corpus. The vocabulary acts as a coordinate system: each vocabulary item is associated with one dimension.

6.1.2 Bag of Words

For a document d, a bag-of-words vector records how many times each vocabulary term appears:

$$\mathbf{x}_d = (c_1, c_2, \ldots, c_{|V|}),$$

where c_i is the count of word w_i in d.

This representation deliberately discards word order. Consequently, sentences with the same multiset of words have the same bag-of-words vector even when their syntax or meaning differs.

6.1.3 One-Hot Word Vectors

A single word w_i can be represented as the standard basis vector

$$\mathbf{e}_i = (0, \ldots, 0, 1, 0, \ldots, 0) \in \mathbb{R}^{|V|}.$$

For distinct words $i \neq j$,

$$\mathbf{e}_i \cdot \mathbf{e}_j = 0.$$

Thus every pair of distinct words is exactly orthogonal. One-hot encoding captures identity but contains no graded semantic similarity.

6.1.4 Vocabulary Size at Web Scale

Realistic vocabularies are enormous. Wikipedia-scale corpora can contain millions of distinct tokens, while web crawls can reach tens of millions after including names, numbers, rare forms, multiple languages, and misspellings. The ambient dimension $d = |V|$ therefore lies directly in the high-dimensional regime studied in Chapter 4.

Remark – What bag of words loses

Count-based representations discard order and do not disambiguate polysemy. The same word form receives the same coordinate regardless of context, and phrases such as “not good” and “good not” contain the same counts.

Review Questions

1. Why must text be mapped to vectors before most algorithms can process it?
2. In what sense does the vocabulary define the coordinate system?
3. Why are distinct one-hot vectors always orthogonal?
4. Why does vocabulary size connect text representation directly to the curse of dimensionality?

6.2 Sparse Representations: TF–IDF

6.2.1 From Counts to Weighted Counts

Raw word counts overemphasize common function words that occur in nearly every document. TF–IDF balances two signals:

- a word should receive more weight when it occurs frequently in the current document;
- that weight should be reduced when the same word is common throughout the corpus.

6.2.2 Term Frequency

For term w and document d, the normalized term frequency is

$$\mathrm{TF}(w, d) = \frac{\mathrm{count}(w, d)}{\sum_{w' \in V} \mathrm{count}(w', d)}. \tag{6.1}$$

The denominator normalizes by document length so that longer documents do not dominate merely because they contain more tokens.

6.2.3 Inverse Document Frequency

Let N be the number of documents in the corpus and $\mathrm{df}(w)$ the number of documents containing w at least once. Define

$$\mathrm{IDF}(w) = \log\left(\frac{N}{\mathrm{df}(w)}\right). \tag{6.2}$$

Rare terms have large IDF; a term appearing in nearly every document receives weight close to zero.

6.2.4 The TF–IDF Formula

The combined weight is

$$\mathrm{TFIDF}(w, d) = \mathrm{TF}(w, d)\,\mathrm{IDF}(w). \tag{6.3}$$

The document vector is

$$\mathbf{x}_d \in \mathbb{R}^{|V|}, \qquad x_{d,i} = \mathrm{TFIDF}(w_i, d).$$

Example – Rare versus ubiquitous terms

For a corpus with $N = 1{,}000{,}000$ documents, a term appearing in 950,000 documents has

$$\mathrm{IDF} \approx \log(1.053) \approx 0.051,$$

while a term appearing in only 12 documents has

$$\mathrm{IDF} \approx \log(83{,}333) \approx 11.33.$$

The rare term therefore receives vastly more corpus-level discriminative weight.

6.2.5 Practical Variants

Implementations frequently use stabilized variants such as

$$\mathrm{IDF}_{\mathrm{smooth}}(w) = \log\left(\frac{N+1}{\mathrm{df}(w)+1}\right) + 1$$

and sublinear term frequency

$$\mathrm{TF}_{\mathrm{sublinear}}(w, d) = 1 + \log \mathrm{count}(w, d)$$

for terms that occur at least once. These variants improve numerical behavior without changing the central intuition.

6.2.6 Cosine Similarity

TF–IDF vectors vary in magnitude, so document similarity is commonly measured through the angle between them:

$$\cos\theta = \frac{\mathbf{x}_d \cdot \mathbf{x}_{d'}}{\|\mathbf{x}_d\|_2\,\|\mathbf{x}_{d'}\|_2}. \tag{6.4}$$

Cosine similarity normalizes vector length and focuses on directional composition.

6.2.7 Extreme Sparsity

A document may contain only hundreds or thousands of distinct terms even when $|V|$ is in the millions. Therefore

$$\frac{\mathrm{nnz}(\mathbf{x}_d)}{|V|} \ll 1,$$

where nnz is the number of nonzero coordinates. Practical systems store only (index, value) pairs for nonzero entries rather than materializing dense million-dimensional arrays.

Review Questions

1. What different information is captured by TF and IDF?
2. Why does IDF suppress function words appearing in almost every document?
3. Why is cosine similarity usually preferable to raw Euclidean distance for TF–IDF vectors?
4. Why is sparse storage essential at web-scale vocabulary sizes?

6.3 Scalability Limits of Sparse TF–IDF Spaces

6.3.1 Sparse Does Not Mean Low-Dimensional

A TF–IDF vector may contain few nonzero entries, but it still formally lives in $\mathbb{R}^{|V|}$. Sparsity reduces storage and arithmetic costs; it does not make the ambient vocabulary dimension disappear.

For two sparse documents, a simple overlap model gives

$$\mathbb{E}[\text{overlap}] \propto \frac{\text{nnz}(\mathbf{x}_d)\,\text{nnz}(\mathbf{x}_{d'})}{|V|}. \tag{6.5}$$

As $|V|$ grows with fixed document lengths, expected overlap shrinks. This sparsity-driven near-orthogonality is related to, but not identical with, the dense-vector concentration effects of Chapter 4.

6.3.2 Corpus-Scale Storage and Search

Even if one sparse vector is cheap, a corpus of N documents can contain an enormous total number of nonzero (index, value) pairs. The vocabulary-to-index mapping must also remain globally consistent across a distributed system.

Naively comparing every pair of documents costs $O(N^2)$ vector comparisons. Even a single query can require comparison with every corpus item unless an index or more compact representation is used.

6.3.3 Zipf's Law

Word frequencies empirically follow an approximate inverse-rank law:

$$\text{frequency}(w) \propto \frac{1}{\text{rank}(w)}. \tag{6.6}$$

A small set of terms accounts for a large fraction of occurrences, while a very long tail contains many rare terms and *hapax legomena* occurring exactly once.

6.3.4 Heaps' Law

Vocabulary size itself grows with corpus size. A common empirical model is

$$|V| \approx Kn^{\beta}, \qquad \beta \in (0, 1), \tag{6.7}$$

where n is the number of observed tokens and β is often around 0.4–0.6. Adding more text therefore keeps introducing new names, rare words, and misspellings rather than stabilizing the vocabulary.

Key Concept

Sparse TF–IDF faces two compounding scaling pressures: an enormous, continually growing vocabulary and expensive corpus-wide similarity computation. This motivates fixed-dimensional dense representations.

6.4 The Distributional Hypothesis

6.4.1 Meaning through Context

The distributional hypothesis can be summarized by Firth's dictum: a word is characterized by the company it keeps. Operationally, words appearing in similar linguistic contexts are expected to have related meanings or syntactic roles.

Rather than assigning meaning directly, the corpus supplies observable co-occurrence statistics. Words such as *dog* and *cat* often occur near related contextual terms; grammatical variants also share characteristic surrounding patterns.

6.4.2 Context Windows

For a token sequence

$$w_1, w_2, \ldots, w_T$$

and a target position t, choose a context radius c. The context consists of

$$\{w_{t+j} : -c \leq j \leq c,\ j \neq 0\}. \tag{6.8}$$

For example, with $c = 2$ in

the quick brown [fox] jumps over the lazy dog,

the target *fox* generates the context words *quick*, *brown*, *jumps*, and *over*.

6.4.3 From Co-Occurrence Counts to Learned Embeddings

Classical distributional methods can build an explicit word-context matrix and apply SVD. Word2Vec takes a different route: it learns dense vectors by solving a prediction problem directly, avoiding explicit materialization of the full co-occurrence matrix.

Review Questions

1. State the distributional hypothesis in operational terms.
2. What training pairs are produced by a context window of radius c?
3. Why can co-occurrence capture both semantic and syntactic similarity?
4. How does predictive learning differ from building a full co-occurrence matrix followed by SVD?

6.5 From Sparse to Dense: Word Embeddings

6.5.1 The Dense Representation

Instead of assigning one coordinate per vocabulary item, a word embedding maps each word to

$$\mathbf{v}_w \in \mathbb{R}^m,$$

where m is fixed and modest, commonly on the order of tens to hundreds of dimensions in the lecture examples. Most coordinates are nonzero, hence the term *dense embedding*.

Table 6.1: Sparse TF–IDF and dense word embeddings.

Aspect	Sparse TF–IDF	Dense Word2Vec-style embedding
Dimension	$\|V\|$, potentially millions and growing	Fixed, typically tens to hundreds
Nonzero entries	A small subset of coordinates	Essentially all coordinates
Coordinate meaning	Each coordinate corresponds to a vocabulary term	No single coordinate has a named semantic interpretation
Similarity signal	Based on weighted term overlap	Learned semantic proximity
Construction	Direct counting formula	Learned prediction objective

6.5.2 Distributed Representation

A dense embedding is called a *distributed representation* because meaning is encoded jointly across many continuous coordinates. No individual dimension is intended to represent a single named feature.

6.5.3 Semantic Geometry

Similarity is again measured naturally with cosine similarity:

$$\cos(\mathbf{v}_a, \mathbf{v}_b) = \frac{\mathbf{v}_a \cdot \mathbf{v}_b}{\|\mathbf{v}_a\|_2 \|\mathbf{v}_b\|_2}.$$

Now, however, proximity is learned from context rather than requiring direct vocabulary overlap.

6.5.4 Analogy Arithmetic

A celebrated empirical property of early Word2Vec embeddings is that some semantic and syntactic relationships appear approximately linear:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \tag{6.9}$$

The effect was not imposed directly as a rule; it emerged from the learned embedding geometry.

6.6 The Word2Vec Formulation

6.6.1 CBOW and Skip-Gram

Word2Vec introduced two closely related predictive architectures:

- **CBOW (Continuous Bag of Words):** use surrounding context words to predict the target word;
- **Skip-gram:** use the target word to predict each surrounding context word.

This chapter follows the lecture's emphasis on Skip-gram, which was reported to perform particularly well on rare words and large datasets.

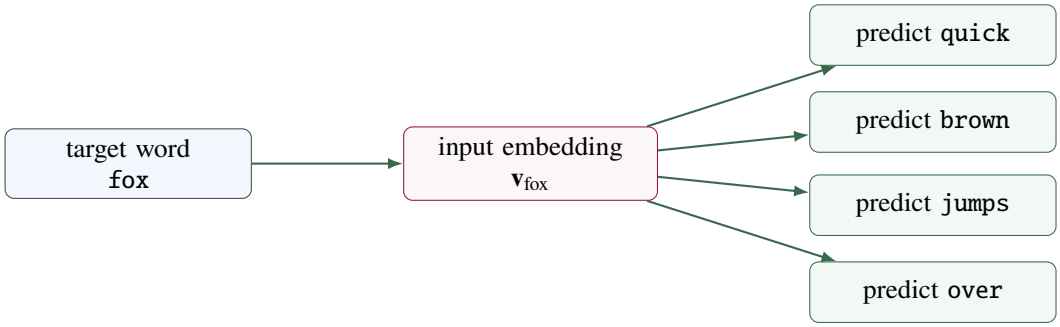


Figure 6.1: Skip-gram architecture: one target embedding is trained to predict surrounding context words.

6.6.2 The Skip-Gram Objective

Let the corpus be $w_1, \ldots, w_T$ and let c be the context radius. Skip-gram maximizes the average log probability of observed context words:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-c \le j \le c \\ j \neq 0,\ 1 \le t+j \le T}} \log p(w_{t+j} \mid w_t; \theta). \tag{6.10}$$

Here θ denotes the learned word-vector parameters.

6.6.3 Input and Output Vectors

Associate each word w with two parameter vectors:

- $\mathbf{v}_w$: input vector when w is the target;
- $\mathbf{v}'_w$: output vector when w is a context candidate.

The full softmax model is

$$p(w_O \mid w_I) = \frac{\exp\left((\mathbf{v}'_{w_O})^\top \mathbf{v}_{w_I}\right)}{\sum_{w=1}^{|V|} \exp\left((\mathbf{v}'_w)^\top \mathbf{v}_{w_I}\right)}. \tag{6.11}$$

The denominator converts dot-product compatibility scores into a probability distribution over the entire vocabulary.

After training, the input vectors $\mathbf{v}_w$ are typically used as the word embeddings for downstream tasks.

6.6.4 The Full-Softmax Bottleneck

Equation (6.11) requires a normalization over all $|V|$ vocabulary entries for every target-context pair. With vocabularies containing millions of words and corpora containing billions of tokens, an $O(|V|)$ update is computationally infeasible.

Hierarchical softmax reduces this to a sequence of binary decisions along a vocabulary tree, with $O(\log |V|)$ cost. The lecture emphasizes negative sampling as a simpler and highly effective alternative.

6.6.5 Negative Sampling

Define the logistic sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

For one observed positive pair (w_I, w_O), negative sampling optimizes

$$\log \sigma\left((\mathbf{v}'_{w_O})^\top \mathbf{v}_{w_I}\right) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma\left(-(\mathbf{v}'_{w_i})^\top \mathbf{v}_{w_I}\right)\right]. \tag{6.12}$$

The positive term pushes the target and observed context vectors toward alignment; the negative terms push noise-sampled word vectors away; a noise draw need not be a word that never occurs in the target's context.

Across the whole corpus, the lecture writes the Skip-gram with Negative Sampling objective as

$$J_{\text{SGNS}}(\theta) = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-c \le j \le c \\ j \neq 0,\ 1 \le t+j \le T}} \left[\log \sigma\left((\mathbf{v}'_{w_{t+j}})^\top \mathbf{v}_{w_t}\right) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \log \sigma\left(-(\mathbf{v}'_{w_i})^\top \mathbf{v}_{w_t}\right)\right]. \tag{6.13}$$

The update cost becomes $O(k)$ rather than $O(|V|)$, with k typically in the range 5–20 in the lecture material. The noise distribution P_n is commonly the unigram distribution raised to the $3/4$ power and renormalized.

Key Concept

Negative sampling changes the computational problem from "score every vocabulary word" to "separate one observed pair from a small number of sampled negative pairs." This is the step that makes Skip-gram practical at web scale.

Review Questions

1. What does CBOW predict, and what does Skip-gram predict?
2. Why does Word2Vec maintain separate input and output vectors?
3. Why is full softmax $O(|V|)$ per training update?
4. How does negative sampling reduce the cost to $O(k)$?
5. What is the role of the noise distribution P_n?

6.7 Training Word2Vec at Scale

6.7.1 Per-Update Cost

Table 6.2: Per-update complexity of three Word2Vec output strategies.

Method	Cost per update	Dependence on $\lvert V\rvert$
Full softmax	$O(\lvert V\rvert)$	Direct, linear
Hierarchical softmax	$O(\log \lvert V\rvert)$	Logarithmic
Negative sampling	$O(k)$, $k \approx 5$–20	None

For a million-word vocabulary and $k = 10$, replacing full softmax by negative sampling changes an update from roughly a million output comparisons to one positive and ten negative comparisons. The table treats embedding dimension m as fixed: including vector arithmetic, the costs are $O(m|V|)$, $O(m \log |V|)$, and $O(m(k+1))$, respectively.

6.7.2 Billion-Word Corpora

Cheap updates do not make the overall training run trivial. Hundreds of millions or billions of tokens produce enormous numbers of target-context pairs. Parallel training can therefore be useful, although distributed compute is not required for every corpus or hardware configuration.

6.7.3 Subsampling Frequent Words

A second optimization probabilistically discards extremely frequent words during training. The lecture gives

$$P(\text{discard } w) = \max\left\{0, 1 - \sqrt{\frac{t}{f(w)}}\right\}, \tag{6.14}$$

where $f(w)$ is the relative corpus frequency of w and t is a chosen threshold. Clipping at zero ensures a valid probability when $f(w) < t$. The purpose is to eliminate many low-information training pairs involving ubiquitous function words.

6.7.4 Data-Parallel Corpus Sharding

The token sequence can be split into independent corpus shards. Each worker processes its local text and computes SGNS updates for target-context-negative triples. This is directly analogous to the input-split idea from MapReduce: the bulk training data is naturally data-parallel.

6.7.5 Shared Parameters and Asynchronous Updates

The embedding parameters are global: every worker ultimately updates the same set

$$\{\mathbf{v}_w, \mathbf{v}'_w : w \in V\}.$$

The lecture connects this to later scale-out learning architectures such as Hogwild!-style asynchronous SGD and the Parameter Server. Negative sampling is especially helpful because each training example touches only a small subset of the full parameter table.

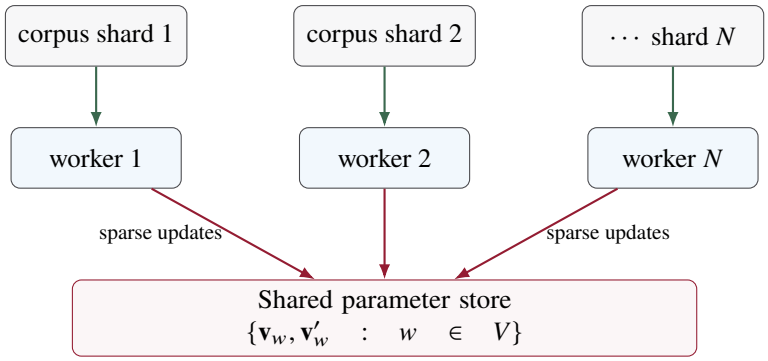


Figure 6.2: Conceptual distributed Word2Vec training pipeline: corpus shards feed parallel workers that update a shared embedding parameter store.

6.8 Tokenization at Scale

6.8.1 Tokenization Is a Representation Decision

Every method above assumes raw text has already been split into tokens. Tokenization determines what counts as an atomic symbol, which in turn determines vocabulary size and the units for TF–IDF or embedding training.

6.8.2 Word-Level Tokenization and OOV

The simplest tokenizer splits on whitespace and punctuation and treats complete words as tokens. A fixed word vocabulary then creates the out-of-vocabulary (OOV) problem: a new name, spelling, or word form observed after training has no corresponding vector.

Definition – Out-of-Vocabulary Token

Given a fixed training vocabulary V, an observed token $w \notin V$ is out of vocabulary and cannot be represented directly by a word-level lookup table.

6.8.3 Subword Tokenization

Subword methods decompose words into reusable pieces, for example

$$\texttt{unbelievable} \longrightarrow \texttt{un} + \texttt{believ} + \texttt{able}.$$

A previously unseen word can then be represented by known subword components, dramatically reducing the OOV problem.

6.8.4 Byte-Pair Encoding

BPE builds a fixed subword vocabulary through repeated frequency-based merges:

1. initialize each word as characters plus an end-of-word marker;
2. count frequencies of adjacent symbol pairs across the corpus;
3. choose the most frequent pair (A, B);
4. create a merged symbol AB, add it to the vocabulary, and replace occurrences of adjacent A, B;
5. repeat for a fixed number K of merges.

The output is a final vocabulary plus an ordered list of merge rules. At inference time, BPE replays those rules in the learned order.

Example – BPE merge sequence from the lecture

For the mini-corpus containing weighted forms `low_`, `lower_`, `newest_`, and `widest_`, early merges include

$$(\texttt{e}, \texttt{s}) \to \texttt{es}, \qquad (\texttt{es}, \texttt{t}) \to \texttt{est}, \qquad (\texttt{est}, \texttt{_}) \to \texttt{est_}, \qquad (\texttt{l}, \texttt{o}) \to \texttt{lo}.$$

An unseen word such as `lowest_` can then be segmented using learned pieces while any unmerged characters remain available as atomic fallback symbols.

6.8.5 WordPiece

WordPiece uses a likelihood-oriented merge score rather than raw pair count. The lecture relates the criterion to

$$\text{Score}(A, B) = \frac{\text{count}(AB)}{\text{count}(A)\,\text{count}(B)} \propto \frac{P(AB)}{P(A)P(B)}. \tag{6.15}$$

This resembles pointwise mutual information: frequent individual symbols are penalized unless their joint occurrence is disproportionately strong.

WordPiece also differs operationally at inference time: instead of replaying merge history, it uses greedy longest-match-first segmentation against the final vocabulary. Continuation pieces are conventionally

marked with a prefix such as ##.

6.8.6 Tokenization Trade-Offs

Table 6.3: Tokenization granularity trade-offs.

Granularity	Handles OOV?	Vocabulary size	Sequence length
Word-level	No	Very large and growing	Short
Subword (BPE/WordPiece)	Mostly	Fixed, moderate	Medium
Character-level	Yes	Small, fixed	Very long

Subword tokenization is the practical middle ground: it keeps vocabulary size bounded without inflating sequences as dramatically as pure character-level tokenization.

6.8.7 Distributed Tokenization

At web scale, tokenization itself is a data-parallel preprocessing stage. A large corpus can be split into shards, tokenized independently, and then combined to aggregate global pair counts or vocabulary statistics. The same MapReduce/Spark principles developed in Module 1 therefore reappear even before model training starts.

Review Questions

1. Why does tokenization directly control vocabulary size?
2. Why does word-level tokenization create an OOV problem?
3. Describe the iterative merge logic of BPE.
4. How does WordPiece differ from BPE in its merge criterion and inference procedure?
5. Why is tokenization naturally data-parallel at web scale?

6.9 Synthesis

6.9.1 The Representation Pipeline

6.9.2 Sparse versus Dense Recap

Table 6.4: Final side-by-side recap of TF–IDF and Word2Vec.

Aspect	TF–IDF	Word2Vec
Dimension	$\|V\|$, large and growing	Fixed, typically tens to hundreds
Sparsity	Extremely sparse	Dense
How obtained	Direct counting and corpus weighting	Learned predictive objective
Similarity	Weighted lexical overlap / cosine	Learned semantic geometry / cosine
Main scalability lever	Sparse storage and indexing	Negative sampling plus distributed training
Vocabulary issue	Directly tied to full term inventory	Still depends on tokenization; subwords reduce OOV

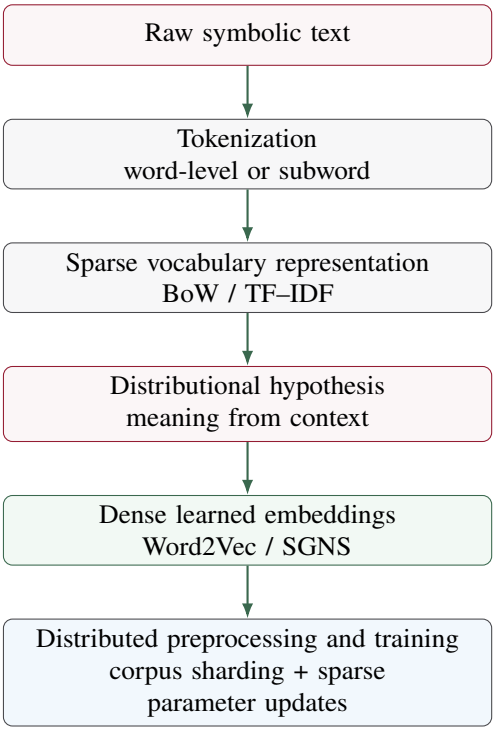


Figure 6.3: The chapter's progression from raw text to scalable dense vector representations.

6.10 Key Terms

6.11 Chapter Summary

Table 6.6: Compact summary of scalable text representation.

Concept	Core mechanism	Main trade-off / consequence
Vocabulary space	One coordinate per token	Explicit but dimension grows with corpus size.
TF–IDF	$\mathrm{TF}(w,d)\log(N/\mathrm{df}(w))$	Strong sparse baseline; still tied to lexical overlap.
Cosine similarity	Normalized dot product	Removes vector magnitude and compares composition.
Sparse overlap	Expected overlap decreases with $\lvert V\rvert$	Documents can become nearly orthogonal lexically.
Distributional hypothesis	Use context as a proxy for meaning	Enables learned semantic rather than purely lexical similarity.
Dense embedding	Fixed-dimensional continuous vector	Compact but individual dimensions are not directly interpretable.
Skip-gram softmax	Predict contexts from target word	Full normalization costs $O(\lvert V\rvert)$ per update.
Negative sampling	One positive plus k sampled negatives	Reduces update cost to $O(k)$.
Distributed training	Shard corpus, share parameters	Parallel data processing plus coordinated parameter updates.

Concept	Core mechanism	Main trade-off / consequence
Subword tokenization	Compose words from reusable pieces	Bounded vocabulary and reduced OOV at the cost of longer sequences.

6.12 Quick Review

1. A vocabulary defines the coordinate system for classical sparse text vectors.
2. Bag of Words preserves counts but discards word order.
3. Distinct one-hot vectors are exactly orthogonal and encode no graded similarity.
4. TF normalizes a term's count by document length.
5. IDF downweights words that occur in many documents.
6. TF–IDF multiplies local frequency by corpus-wide rarity.
7. Cosine similarity compares vector direction rather than magnitude.
8. TF–IDF vectors are extremely sparse but still live in the huge ambient space $\mathbb{R}^{|V|}$.
9. Expected sparse overlap decreases as vocabulary size grows.
10. Zipf's law produces a few very common words and a long tail of rare words.
11. Heaps' law says vocabulary size continues growing with the number of observed tokens.
12. The distributional hypothesis defines meaning operationally through linguistic context.
13. A context window generates target-context training pairs.
14. Dense embeddings use a fixed modest dimension and encode meaning across all coordinates jointly.
15. Learned embedding geometry can support approximate analogy arithmetic.
16. CBOW predicts the target from context; Skip-gram predicts context from the target.
17. Full softmax scores the entire vocabulary and therefore costs $O(|V|)$ per update.
18. Negative sampling reformulates training as discrimination between observed and sampled word pairs.
19. SGNS reduces each update to $O(k)$ with a small number of negatives.
20. Frequent-word subsampling removes many low-information training pairs.
21. Corpus sharding makes Word2Vec data-parallel, while shared vectors create a parameter-coordination problem.
22. Word-level tokenization suffers from OOV tokens.
23. BPE learns an ordered sequence of frequent adjacent-symbol merges.
24. WordPiece favors symbol pairs that co-occur disproportionately and uses longest-match-first inference.
25. Subword tokenization is the practical compromise between vocabulary size, OOV robustness, and sequence length.

Table 6.5: Key terminology for text representation and embeddings.

Term	Meaning
Vocabulary	Set of distinct tokens defining the coordinate system of sparse text vectors.
Bag of Words	Document representation based on token counts with word order discarded.
One-hot encoding	Basis-vector representation with one active coordinate per vocabulary item.
TF–IDF	Sparse weighting combining in-document frequency with corpus-wide rarity.
Cosine similarity	Normalized dot product measuring angular similarity between vectors.
Zipf's law	Approximate inverse relationship between word frequency and frequency rank.
Heaps' law	Empirical growth law $\|V\| \approx Kn^{\beta}$ for vocabulary size.
Distributional hypothesis	Words occurring in similar contexts tend to have related meanings.
Dense embedding	Fixed-dimensional learned continuous representation.
Distributed representation	Information encoded across many vector coordinates jointly.
CBOW	Predict target word from surrounding context words.
Skip-gram	Predict surrounding context words from the target word.
Negative sampling	Replace full-vocabulary softmax normalization by binary discrimination against sampled negatives.
SGNS	Skip-gram with Negative Sampling objective.
OOV	Token absent from a fixed training vocabulary.
BPE	Frequency-driven iterative subword merge algorithm.
WordPiece	Subword method using a likelihood/association-driven merge score and greedy longest-match inference.

6.13 Further Reading

Further reading includes:

- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. NeurIPS.
- Course reference materials on TF–IDF, tokenization, and distributed word embeddings.

Combined Quick Reference

The first six chapters now form two connected layers of the course. Module 1 develops the scale-out systems stack: distributed storage provides durable, location-aware blocks; MapReduce turns those blocks into data-local batch tasks connected by shuffle; Spark generalizes computation into a lazy DAG with in-memory reuse and lineage-based recovery. Module 2 then studies the vector representations that run on top of that infrastructure: high-dimensional geometry explains why naive distance degrades, Johnson–Lindenstrauss random projection provides a controlled dimensionality reduction, and text representation develops the path from sparse TF–IDF vectors to dense learned embeddings.

Module 1: Big Data Infrastructure

Table 6.7: Distributed storage, MapReduce, and Spark viewed as one scale-out architecture.

Concern	Distributed storage	MapReduce	Spark / RDDs
Basic unit	Block / chunk	Map or Reduce task	RDD partition / task
Coordinator	Master / NameNode	Master / JobTracker	DAG scheduler
Execution shape	Storage namespace and replica graph	Fixed Map → Shuffle → Reduce job	Arbitrary lazy DAG split into stages
Placement goal	Durable replicas across failure domains	Data-local Map execution	Data-local tasks and reuse of cached partitions
Failure detection	Heartbeats, block reports, checksums	Worker heartbeats and progress tracking	Task/worker failure handled through scheduler state
Recovery	Re-replication	Re-execution of failed tasks	Lineage-based recomputation of lost partitions
Network-sensitive operation	Cross-rack replication	Shuffle	Wide-dependency shuffle
Intermediate state	Durable replicated blocks	Local Map output plus job-boundary materialization	In-memory partitions when possible; checkpoint when needed
Main optimization	Placement and direct data transfer	Locality, combiners, speculation	Laziness, pipelining, caching, stage optimization

Module 2: High-Dimensional Data Representations

Key Concept

The course-wide pattern is now visible: scale-out systems make computation over enormous datasets feasible, while representation and approximation techniques make the mathematical operations performed on those datasets meaningful and tractable.

Table 6.8: Geometry, dimensionality reduction, and text embeddings as one representation pipeline.

Concern	High-dimensional geometry	Random projection / JL	Text representations
Starting object	Vectors in $\mathbb{R}^d$	Finite set of n vectors in $\mathbb{R}^d$	Raw tokens, documents, and vocabularies
Core problem	Volume/distance concentration weakens geometric intuition	Reduce d while preserving pairwise distances	Turn symbolic text into scalable numerical vectors
Key mechanism	Concentration near boundaries/equators	Isotropic random map $f(\mathbf{x}) = A\mathbf{x}/\sqrt{k}$	TF–IDF weighting or learned distributional embeddings
Central result	Relative distance spread $\to 0$	$k = O(\log n/\varepsilon^2)$ dimensions suffice	SGNS reduces Word2Vec update cost from $O(\lvert V\rvert)$ to $O(k)$
Main probability tool	Chebyshev / concentration estimates	Chi-squared MGF, Chernoff, union bound	Sampling-based training objective
Scalability issue	Raw distance loses discrimination as d grows	Dense projection costs $O(kd)$ per point	Vocabulary growth, full softmax, OOV tokens
Practical response	Change representation / reduce dimension	Sparse or Fast JL transforms	Sparse storage, negative sampling, subword tokenization
Connection to systems	Vectors are stored and processed on Module 1 infrastructure	Projection is embarrassingly parallel	Tokenization and training can be sharded across workers